STAR RX72N - rx_bno055 Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

File Index

2.1 File List

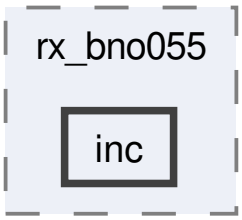
Here is a list of all files with brief descriptions:

Chapter 3

Directory Documentation

3.1 libs/rx_bno055/inc Directory Reference

Directory dependency graph for inc:

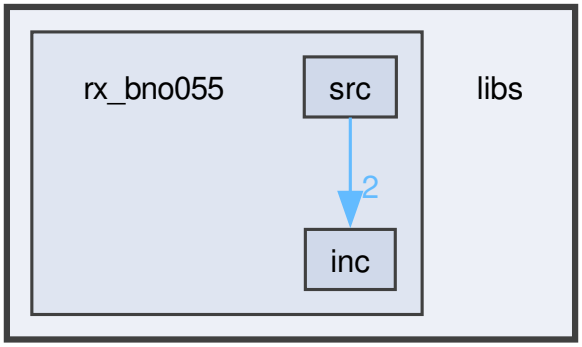


Files

- file rx_bno055.h
  BNO055 9-DOF Absolute Orientation Sensor Driver API.
- file rx_bno055_regs.h
  BNO055 9-DOF Absolute Orientation Sensor Register Map and Configuration Enumerations.

3.2 libs Directory Reference

Directory dependency graph for libs:

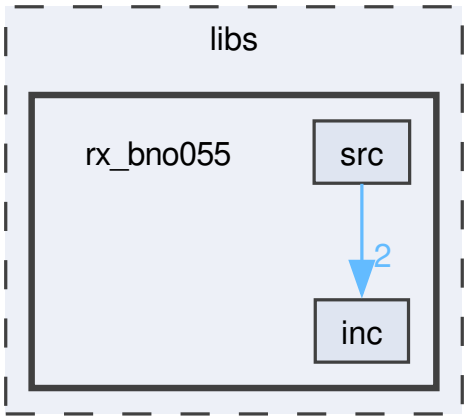


Directories

- directory rx_bno055

3.3 libs/rx_bno055 Directory Reference

Directory dependency graph for rx_bno055:

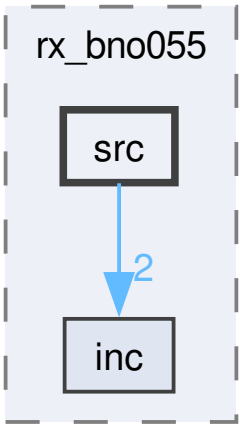


Directories

- directory inc
- directory src

3.4 libs/rx_bno055/src Directory Reference

Directory dependency graph for src:



Files

- file rx_bno055.c

  BNO055 9-DOF Absolute Orientation Sensor Driver Implementation.

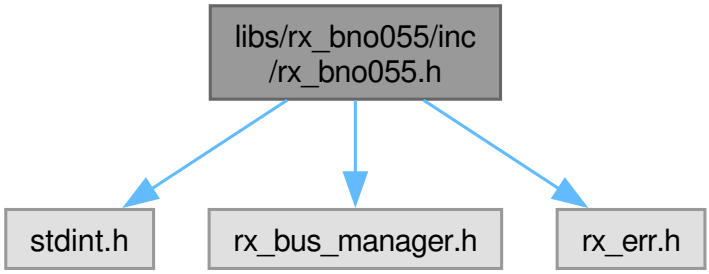
Chapter 4

File Documentation

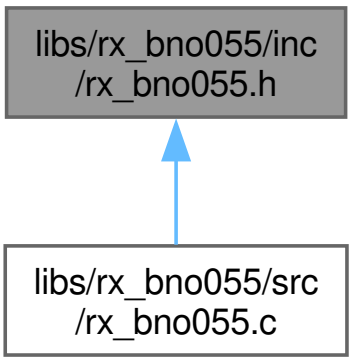
4.1 libs/rx_bno055/inc/rx_bno055.h File Reference

BNO055 9-DOF Absolute Orientation Sensor Driver API.

#include <stdint.h>
#include "rx_bus_manager.h"
#include "rx_err.h"
Include dependency graph for rx_bno055.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct bno055_config_t

  BNO055 driver configuration passed to rx_bno055_init().
- struct bno055_data_t

  BNO055 sensor output data (raw scaled integers).

Enumerations

- enum bno055_mode_t : uint8_t { k_bno055_mode_poll = 0U , k_bno055_mode_interrupt = 1U }

  BNO055 driver operating mode selection.
- enum bno055_calib_shift_t : uint8_t { k_bno055_calib_mag_shift = 0U , k_bno055_calib_acc_shift = 2U , k_bno055_calib_gyr_shift = 4U , k_bno055_calib_sys_shift = 6U }

  BNO055 CALIB_STAT register bit-shift positions for each subsystem.
- enum bno055_calib_mask_t : uint8_t { k_bno055_calib_level_mask = 0x03U , k_bno055_calib_full = 0x03U }

  BNO055 calibration level mask and fully-calibrated sentinel.
- enum bno055_scale_t : uint16_t { k_bno055_scale_heading , k_bno055_scale_quat = 16384U , k_bno055_scale_acc = 100U , k_bno055_scale_gyro = 16U }

  BNO055 output data scale factors (divisors for unit conversion).

Functions

- rx_err_t rx_bno055_init (rx_bus_manager_t *manager, const bno055_config_t *config)

  Initialize BNO055 sensor and configure for NDOF fusion mode.
- rx_err_t rx_bno055_read (bno055_data_t *out)

  Read current BNO055 fusion output data.
- rx_err_t rx_bno055_is_calibrated (bool *out_calibrated)

  Check whether all BNO055 subsystems are fully calibrated.
- rx_err_t rx_bno055_clear_int (void)

  Clear the BNO055 INT pin by reading the INT_STA register (Page 0, 0x37).

4.1.1 Detailed Description

BNO055 9-DOF Absolute Orientation Sensor Driver API.

4.1.2 Overview

Platform-independent driver for the Bosch BNO055 intelligent 9-axis absolute orientation sensor. The BNO055 integrates accelerometer, gyroscope, and geomagnetic sensor with an on-chip ARM Cortex-M0 that performs hardware sensor fusion to produce absolute orientation data (Euler angles, quaternions, linear acceleration).

This driver abstracts the BNO055 initialization sequence and data reading into three simple functions: init, read, and calibration check.

4.1.3 Key Features

- Absolute orientation: Euler angles (heading, roll, pitch) referenced to magnetic north and gravity
- Quaternion output: Unit quaternion for rotation-matrix computation
- Linear acceleration: Gravity-compensated acceleration in m/s^2
- Calibration status: Per-subsystem calibration quality (0-3)
- Temperature: On-chip temperature sensor

4.1.4 Hardware Configuration (STAR Project)

- Module: DFRobot BNO055 breakout
- I2C Address: 0x28 (COM3/ADR pulled LOW)
- I2C Bus: RIIC1 (SCL1=P2.1, SDA1=P2.0)
- Bus name in bus manager: "i2c1_imu"
- Reset pin: P8.3 (active-low, driven HIGH by hardware_init)

4.1.5 Operating Mode

The driver initializes the sensor in NDOF (Nine Degrees of Freedom) mode with:

- All three sensors active (accelerometer, gyroscope, magnetometer)
- Hardware sensor fusion running on the BNO055's ARM Cortex-M0
- Default measurement units: degrees, m/s^2, dps, Celsius

4.1.6 Initialization Sequence

Before calling rx_bno055_init(), the caller (imu_task.c) performs a hardware reset via P83 (IMU RST, active-low): assert LOW for >= 10 ms, then deassert HIGH and wait 650 ms for the BNO055 POR sequence to complete. See internal_imu_hardware_reset() in imu_task.c.

The rx_bno055_init() function then performs the following steps:

1. Software reset (SYS_TRIGGER = 0x20), wait 650 ms (belt-and-suspenders)
2. Set CONFIG mode (OPR_MODE = 0x00), wait 19 ms
3. Set normal power mode (PWR_MODE = 0x00)
4. Set default units (UNIT_SEL = 0x00)
5. Set default axis map (AXIS_MAP_CONFIG = 0x24, AXIS_MAP_SIGN = 0x00)
6. Clear system trigger (SYS_TRIGGER = 0x00)
7. Set NDOF fusion mode (OPR_MODE = 0x0C), wait 7 ms
8. Verify chip ID == 0xA0

4.1.7 Output Data Scales

Field	Raw Type	Scale	Unit
heading_deg16	int16_t	/ 16	degrees
roll_deg16	int16_t	/ 16	degrees
pitch_deg16	int16_t	/ 16	degrees
quat_w/x/y/z	int16_t	/ 16384	dimensionless
lin_acc_x/y/z	int16_t	/ 100	m/s^2
temp_degc	int8_t	1.0	deg C
calib_stat	uint8_t	raw	-

See also

rx_bno055_regs.h Register map and scale constants

rx_bno055.c Implementation

rx_bus_i2c.h I2C bus abstraction used

NASA Power of 10 Compliance:

- Rule 1 (No goto): [PASS] No goto, recursion, or setjmp
- Rule 3 (No heap): [PASS] Zero dynamic allocation
- Rule 5 (Assertions): [PASS] 2+ preconditions per function
- Rule 7 (Return checks): [PASS] All I2C returns validated
- Rule 8 (Preprocessor): [PASS] C23 typed enums, no magic numbers
- Rule 9 (Pointers): [WARN] Function pointers for DIP (bus manager)

SOLID Principles:

- S: Module handles ONLY BNO055 sensor operations
- O: Extensible via bus manager injection
- L: Any rx_bus_manager_t* is interchangeable
- I: Focused 3-function API
- D: Depends on rx_bus_manager_t abstraction, not RIIC directly

Author

STAR Team

Date

2026-03-04

Version

1.0.0

Copyright

Definition in file rx_bno055.h.

4.1.8 Data Structure Documentation

4.1.8.1 struct bno055_config_t

BNO055 driver configuration passed to rx_bno055_init().

Allows the caller to select polling or interrupt-driven mode. In interrupt mode, rx_bno055_init() programs the BNO055 Page 1 interrupt engine registers after chip ID verification so that the INT pin asserts on each new sample.

Invariant

mode must be a valid bno055_mode_t value

```
// Polling mode (no INT pin activity):
const bno055_config_t cfg = {.mode = k_bno055_mode_poll};
rx_bno055_init(&manager, &cfg);

// Interrupt mode (BNO055 INT -> RX72N IRQ12):
const bno055_config_t cfg = {.mode = k_bno055_mode_interrupt};
rx_bno055_init(&manager, &cfg);
```

See also

bno055_mode_t Mode selection enumeration

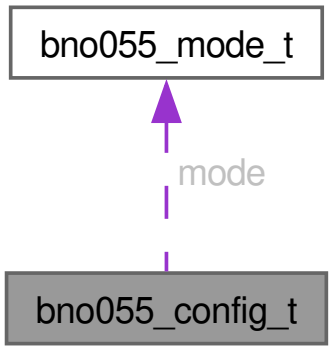
rx_bno055_init() Consumes this configuration struct

Since

Version 1.0.0

Definition at line 166 of file rx_bno055.h.

Collaboration diagram for bno055_config_t:



Data Fields

bno055_mode_t	mode	Operating mode: k_bno055_mode_poll or k_bno055_mode_interrupt

4.1.8.2 struct bno055_data_t

BNO055 sensor output data (raw scaled integers).

All values are stored as raw 16-bit integers to avoid floating-point operations in the driver. Convert to physical units using the scale factors defined in bno055_scale_t.

Conversion formulas:

```
float heading_deg = (float)data.heading_deg16 / (float)k_bno055_scale_heading;
float roll_deg    = (float)data.roll_deg16    / (float)k_bno055_scale_heading;
float pitch_deg   = (float)data.pitch_deg16   / (float)k_bno055_scale_heading;
float quat_w_f    = (float)data.quat_w        / (float)k_bno055_scale_quat;
float acc_x_mps2  = (float)data.lin_acc_x     / (float)k_bno055_scale_acc;
```

Invariant

quat_w^2 + quat_x^2 + quat_y^2 + quat_z^2 == k_bno055_scale_quat^2 (unit quaternion)

heading_deg16 in [0, 360 * k_bno055_scale_heading - 1] (0 to 359.9375 degrees * 16)

temp_degc in [-40, 85] (operating range of BNO055)

See also

bno055_scale_t Scale factor constants for unit conversion

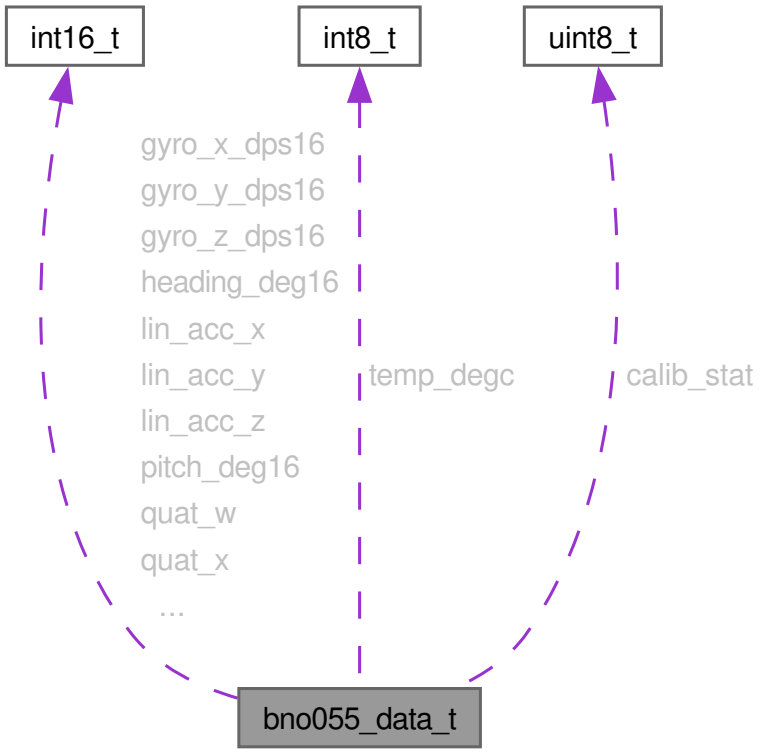
rx_bno055_read() Populates this structure

Since

Version 1.0.0

Definition at line 269 of file rx_bno055.h.

Collaboration diagram for bno055_data_t:



Data Fields

uint8_t	calib_stat	Raw CALIB_STAT byte: SYS[7:6] GYR[5:4] ACC[3:2] MAG[1:0]
int16_t	gyro_x_dps16	Gyroscope X in dps * k_bno055_scale_gyro (divide by 16 for deg/s)
int16_t	gyro_y_dps16	Gyroscope Y in dps * k_bno055_scale_gyro (divide by 16 for deg/s)
int16_t	gyro_z_dps16	Gyroscope Z in dps * k_bno055_scale_gyro (divide by 16 for deg/s)
int16_t	heading_deg16	Euler heading 0-359.9375 deg (divide by k_bno055_scale_heading for degrees)
int16_t	lin_acc_x	Linear accel X in m/s^2 * k_bno055_scale_acc (divide by k_bno055_scale_acc for m/s^2)
int16_t	lin_acc_y	Linear accel Y in m/s^2 * k_bno055_scale_acc (divide by k_bno055_scale_acc for m/s^2)
int16_t	lin_acc_z	Linear accel Z in m/s^2 * k_bno055_scale_acc (divide by k_bno055_scale_acc for m/s^2)
int16_t	pitch_deg16	Euler pitch -180 to +180 deg (divide by k_bno055_scale_heading for degrees)
int16_t	quat_w	Quaternion W component (divide by k_bno055_scale_quat for unit quaternion)
int16_t	quat_x	Quaternion X component (divide by k_bno055_scale_quat for unit quaternion)
int16_t	quat_y	Quaternion Y component (divide by k_bno055_scale_quat for unit quaternion)
int16_t	quat_z	Quaternion Z component (divide by k_bno055_scale_quat for unit quaternion)
int16_t	roll_deg16	Euler roll -90 to +90 deg (divide by k_bno055_scale_heading for degrees)
int8_t	temp_degc	On-chip temperature in degrees Celsius (1 deg C per LSB)

4.1.9 Enumeration Type Documentation

4.1.9.1 bno055_calib_mask_t

enum bno055_calib_mask_t : uint8_t

BNO055 calibration level mask and fully-calibrated sentinel.

After shifting a CALIB_STAT field into the LSB position, AND with k_bno055_calib_level_mask to isolate the 2-bit value. Compare against k_bno055_calib_full (== 3) to check for full calibration.

See also

bno055_calib_shift_t Shift constants for each subsystem

rx_bno055_is_calibrated() Uses both mask and full sentinel

Since

Version 1.0.0

Enumerator

k_bno055_calib_level_mask	2-bit mask for a single calibration level (0-3)
k_bno055_calib_full	Fully calibrated level

Definition at line 210 of file rx_bno055.h.

```
00210             : uint8_t {
00211   k_bno055_calib_level_mask = 0x03U, /**< 2-bit mask for a single calibration level (0-3) */
00212   k_bno055_calib_full       = 0x03U, /**< Fully calibrated level */
00213 } bno055_calib_mask_t;
```

4.1.9.2 bno055_calib_shift_t

enum bno055_calib_shift_t : uint8_t

BNO055 CALIB_STAT register bit-shift positions for each subsystem.

CALIB_STAT register (0x35) contains four 2-bit calibration level fields. Right-shift the raw byte by the appropriate constant, then AND with k_bno055_calib_level_mask to extract a value in [0, 3].

```
uint8_t sys_lvl = (calib_stat » k_bno055_calib_sys_shift) & k_bno055_calib_level_mask;
uint8_t mag_lvl = (calib_stat » k_bno055_calib_mag_shift) & k_bno055_calib_level_mask;
```

See also

bno055_calib_mask_t Mask and full-calibration sentinel values

rx_bno055_is_calibrated() Checks all four fields simultaneously

Since

Version 1.0.0

Enumerator

k_bno055_calib_mag_shift	Bit shift for magnetometer calibration level (bits 1:0)
k_bno055_calib_acc_shift	Bit shift for accelerometer calibration level (bits 3:2)
k_bno055_calib_gyr_shift	Bit shift for gyroscope calibration level (bits 5:4)
k_bno055_calib_sys_shift	Bit shift for system calibration level (bits 7:6)

Definition at line 189 of file rx_bno055.h.

```
00189             : uint8_t {
00190   k_bno055_calib_mag_shift = 0U, /**< Bit shift for magnetometer calibration level (bits 1:0) */
00191   k_bno055_calib_acc_shift = 2U, /**< Bit shift for accelerometer calibration level (bits 3:2) */
00192   k_bno055_calib_gyr_shift = 4U, /**< Bit shift for gyroscope calibration level (bits 5:4) */
00193   k_bno055_calib_sys_shift = 6U, /**< Bit shift for system calibration level (bits 7:6) */
00194 } bno055_calib_shift_t;
```

4.1.9.3 bno055_mode_t

enum bno055_mode_t : uint8_t

BNO055 driver operating mode selection.

Selects whether the BNO055 INT pin is configured to assert on each new data sample (interrupt-driven mode) or left unconfigured (polling mode).

In interrupt-driven mode, rx_bno055_init() programs five Page 1 registers after entering NDOF mode:

1. PAGE_ID = 1 (switch to Page 1)
2. ACC_AM_THRES = 0x01 (minimum threshold ~3.9 mg, fires at data rate)
3. ACC_INT_SETTINGS = 0x07 (enable any-motion on X+Y+Z axes)
4. INT_EN = 0x20 (enable accelerometer any-motion interrupt source)
5. PAGE_ID = 0 (restore Page 0 for normal sensor reads)

In polling mode, none of these writes occur and the INT pin never asserts.

See also

bno055_config_t Configuration struct passed to rx_bno055_init()

Since

Version 1.0.0

Enumerator

k_bno055_mode_poll	Polling mode: INT pin not configured; caller polls at fixed rate
k_bno055_mode_interrupt	Interrupt mode: configure BNO055 INT pin via ACC any-motion

Definition at line 135 of file rx_bno055.h.

```
00135             : uint8_t {
00136   k_bno055_mode_poll = 0U, /**< Polling mode: INT pin not configured; caller polls at fixed rate */
00137   k_bno055_mode_interrupt = 1U, /**< Interrupt mode: configure BNO055 INT pin via ACC any-motion */
00138 } bno055_mode_t;
```

4.1.9.4 bno055_scale_t

enum bno055_scale_t : uint16_t

BNO055 output data scale factors (divisors for unit conversion).

Scale factors to convert raw 16-bit integer fields in bno055_data_t to physical units. Divide the raw register value by the corresponding constant.

```
float heading_deg = (float)data.heading_deg16 / (float)k_bno055_scale_heading;
float quat_w_f    = (float)data.quat_w        / (float)k_bno055_scale_quat;
float acc_x_mps2  = (float)data.lin_acc_x     / (float)k_bno055_scale_acc;
```

See also

bno055_data_t Raw integer fields that use these scales

rx_bno055_read() Populates bno055_data_t with raw scaled integers

Since

Version 1.0.0

Enumerator

k_bno055_scale_heading	Degrees-per-unit for heading/roll/pitch (divide by 16 for degrees)
k_bno055_scale_quat	Quaternion unit (2^14; divide by 16384 for normalized float)
k_bno055_scale_acc	m/s^2 per unit for linear acceleration (divide by 100)
k_bno055_scale_gyro	deg/s per unit for gyro fields (divide by 16 for deg/s)

Definition at line 234 of file rx_bno055.h.

```
00234             : uint16_t {
00235   k_bno055_scale_heading =
00236     16U, /**< Degrees-per-unit for heading/roll/pitch (divide by 16 for degrees) */
00237   k_bno055_scale_quat = 16384U, /**< Quaternion unit (2^14; divide by 16384 for normalized float) */
00238   k_bno055_scale_acc  = 100U,   /**< m/s^2 per unit for linear acceleration (divide by 100) */
00239   k_bno055_scale_gyro = 16U,    /**< deg/s per unit for gyro fields (divide by 16 for deg/s) */
00240 } bno055_scale_t;
```

4.1.10 Function Documentation

4.1.10.1 rx_bno055_clear_int()

```
rx_err_t rx_bno055_clear_int (
                void ) [nodiscard]
```

Clear the BNO055 INT pin by reading the INT_STA register (Page 0, 0x37).

Per BNO055 datasheet section 3.6, the INT output pin is asserted when an enabled-and-unmasked interrupt source fires and stays asserted until firmware reads the INT_STA register. Without this read, the pin latches high after the first interrupt and never produces another edge, so edge-triggered external IRQs on the host MCU fire exactly once.

Call this after every sensor read (or at any other convenient cadence) so the pin re-triggers on the next ACC_BSX_DRDY event.

Returns

rx_err_t Operation result

Return values

k_rx_ok	INT_STA read successfully (pin cleared)
k_rx_err_invalid_state	Driver not yet initialized
k_rx_err_timeout	/ k_rx_err_nack I2C failure

Precondition

rx_bno055_init() completed with k_rx_ok

Postcondition

BNO055 INT pin is low; pending-interrupt bits in INT_STA are cleared

Note

Safe to call with no pending interrupts – the read is idempotent.

Not thread-safe.

Definition at line 1203 of file rx_bno055.c.

```
01204 {
01205   if (!s_initialized) {
01206     return k_rx_err_not_initialized;
01207   }
01208   uint8_t int_sta = 0U;
01209   return internal_read_regs(k_bno055_reg_int_sta, &int_sta, k_bno055_single_byte);
01210 }
```

References internal_read_regs(), k_bno055_reg_int_sta, k_bno055_single_byte, and s_initialized.

Here is the call graph for this function:

4.1.10.2 rx_bno055_init()

```
rx_err_t rx_bno055_init (
                rx_bus_manager_t * manager,
                const bno055_config_t * config)   [nodiscard]
```

Initialize BNO055 sensor and configure for NDOF fusion mode.

Performs the complete BNO055 initialization sequence:

1. Software reset via SYS_TRIGGER register (wait 650 ms for POR)
2. Enter CONFIG mode for register writes (wait 19 ms)
3. Configure power mode to Normal
4. Set measurement units to default (degrees, m/s^2, dps, Celsius)
5. Set default axis mapping (standard orientation)
6. Clear system trigger register
7. Enter NDOF fusion mode (wait 7 ms for mode transition)
8. Verify CHIP_ID register reads 0xA0
9. (If config->mode == k_bno055_mode_interrupt) Configure Page 1 interrupt engine so the INT pin asserts on each new accelerometer sample

The function stores the bus manager pointer in a module-static variable for use by subsequent rx_bno055_read() calls. This design assumes a single BNO055 instance (no multi-instance support needed for STAR hardware).

Parameters

in	manager	Pointer to initialized bus manager. Must have "i2c1_imu" bus registered and initialized. Must not be NULL.
in	config	Pointer to driver configuration selecting poll vs interrupt mode. Must not be NULL.

Returns

rx_err_t Initialization result

Return values

k_rx_ok	Sensor initialized, NDOF mode active, ready to read
k_rx_err_null_ptr	manager or config is NULL
k_rx_err_nack	I2C NACK - device not found (check address/wiring)
k_rx_err_invalid_state	CHIP_ID mismatch (wrong device or I2C issue)
k_rx_err_timeout	I2C transaction timeout

Precondition

Caller must have completed hardware reset via P83 (IMU RST, active-low): assert LOW for >= 10 ms, deassert HIGH, wait >= 650 ms for POR to complete. See internal_imu_hardware_reset() in imu_task.c.

manager must be initialized via rx_bus_manager_init()

"i2c1_imu" bus must be registered and initialized in manager

RIIC1 hardware initialized by hardware_init() (riic_init)

BNO055 powered and connected to I2C bus

Postcondition

Sensor in NDOF fusion mode

Internal s_manager pointer stored for subsequent reads

s_initialized flag set to true on success

Sensor actively running sensor fusion (accelerometer + gyro + mag)

INT pin configured to assert on data ready if config->mode == k_bno055_mode_interrupt

Note

Not thread-safe. Call from single-threaded initialization context.

Blocks for ~700 ms total due to required startup delays.

Warning

Do not call if hardware_init() has not been called first.

Performance:

- Execution time: ~700 ms (dominated by 650 ms POR delay)
- I2C transactions: ~12 writes + 1 read (polling); ~17 writes + 1 read (interrupt mode)

See also

rx_bno055_read() Read sensor data after initialization

bno055_mode_t Mode selection (poll vs interrupt)

bno055_config_t Configuration struct

bno055_delay_ms_t Required timing constants

Since

Version 1.0.0

Validates parameters, stores module state, then delegates the complete 8-step initialization sequence (BNO055 datasheet section 3.3.1) to internal_init_run_sequence(). Any failure sets s_manager = NULL before returning so the module is re-initializable.

Delay rationale:

- 650 ms after reset: BNO055 internal boot sequence (ARM Cortex-M0 startup)
- 19 ms after config mode: Fusion -> CONFIG mode transition
- 7 ms after NDOF mode: CONFIG -> NDOF mode transition

ThreadX tick conversion: 1 tick = 10 ms at 100 Hz tick rate (k_bno055_ms_per_tick = 10).

- 650 ms POR: 650/10 = 65 ticks
- 19 ms config: 19/10 + 1 = 2 ticks (20 ms, rounded up for margin)
- 7 ms NDOF: 1 tick (10 ms, rounded up for margin)

Precondition

manager non-NULL with "i2c1_imu" bus registered and initialized

config non-NULL with a valid bno055_mode_t value in config->mode

Postcondition

s_initialized == true on success

Sensor running NDOF fusion

s_manager == NULL on any failure path

INT pin asserts at accelerometer data rate when config->mode == k_bno055_mode_interrupt

Note

Not thread-safe

Blocks ~700 ms total

See also

internal_init_run_sequence() Delegated initialization sequence

bno055_delay_ms_t Timing constants

bno055_config_t Configuration struct

Since

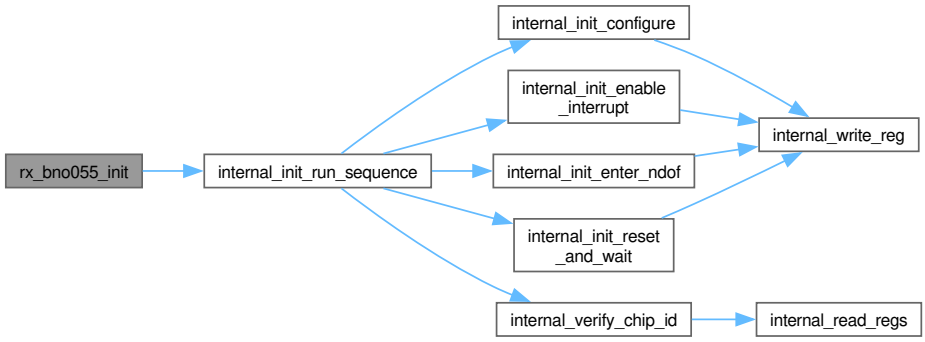
Version 1.0.0

Definition at line 874 of file rx_bno055.c.

```
00875 {
00876   static_assert((bool)((k_bno055_ms_per_tick != 0) != 0),
00877                 "k_bno055_ms_per_tick must be non-zero to avoid division by zero in tick delays");
00878   RX_CHECK_NULL_PTR(manager, s_tag, "Bus manager is NULL");
00879   RX_CHECK_NULL_PTR(config, s_tag, "Config is NULL");
00880
00881   if (s_initialized) {
00882     rx_log_error(s_tag, "BNO055 already initialized");
00883     return k_rx_err_invalid_state;
00884   }
00885
00886   s_manager     = manager;
00887   s_mode        = config->mode;
00888   s_initialized = false;
00889
00890   const rx_err_t err = internal_init_run_sequence();
00891   if (err != k_rx_ok) {
00892     s_manager = NULL;
00893     return err;
00894   }
00895
00896   s_initialized = true;
00897   rx_log_info(s_tag, "BNO055 initialized in NDOF mode");
00898
00899   return k_rx_ok;
00900 }
```

References internal_init_run_sequence(), k_bno055_ms_per_tick, bno055_config_t::mode, s_initialized, s_manager, s_mode, and s_tag.

Here is the call graph for this function:



4.1.10.3 rx_bno055_is_calibrated()

```
rx_err_t rx_bno055_is_calibrated (
                bool * out_calibrated)   [nodiscard]
```

Check whether all BNO055 subsystems are fully calibrated.

Reads the CALIB_STAT register (0x35) and checks whether all four calibration fields (SYS, GYR, ACC, MAG) are at level 3 (fully calibrated).

The BNO055 calibration process:

- Magnetometer: Move sensor in figure-8 pattern
- Gyroscope: Keep sensor stationary for a few seconds
- Accelerometer: Hold in 6 different orientations
- System: All subsystems must be calibrated

Calibration is stored in non-volatile EEPROM inside the BNO055.

Parameters

out	out_calibrated	Pointer to bool to receive result. Must not be NULL. Set to true if all four subsystems at level 3.

Returns

rx_err_t Read result

Return values

k_rx_ok	out_calibrated set (true or false)
k_rx_err_null_ptr	out_calibrated is NULL
k_rx_err_not_initialized	rx_bno055_init() not called
k_rx_err_nack	I2C communication failure
k_rx_err_timeout	I2C read timeout

Precondition

rx_bno055_init() called successfully

out_calibrated points to valid bool storage

Postcondition

*out_calibrated == true only if SYS==3 && GYR==3 && ACC==3 && MAG==3

CALIB_STAT register read over I2C (no state modified)

Note

Not thread-safe

Calibration status can be monitored in real-time via bno055_data_t.calib_stat

See also

bno055_calib_shift_t Bit-shift constants for each calibration subsystem field

bno055_calib_mask_t Mask and fully-calibrated sentinel for CALIB_STAT fields

rx_bno055_read() Reads calib_stat without separate I2C transaction

Since

Version 1.0.0

Reads CALIB_STAT register and checks all four 2-bit fields are at level 3 (fully calibrated).

Precondition

rx_bno055_init() succeeded

out_calibrated non-NULL

Postcondition

*out_calibrated valid only on k_rx_ok

CALIB_STAT register read (no state modified)

Note

Not thread-safe

See also

bno055_calib_shift_t Calibration shift constants

bno055_calib_mask_t Calibration mask and full-calibration sentinel

Since

Version 1.0.0

Definition at line 1173 of file rx_bno055.c.

```
01174 {
01175   RX_CHECK_NULL_PTR(out_calibrated, s_tag, "Output calibrated pointer is NULL");
01176
01177   if (!s_initialized) {
01178     return k_rx_err_not_initialized;
01179   }
01180
01181   uint8_t  calib_raw = 0;
01182   rx_err_t err = internal_read_regs(k_bno055_reg_calib_stat, &calib_raw, k_bno055_single_byte);
01183   if (err != k_rx_ok) {
01184     return err;
01185   }
01186
01187   const uint8_t mask = k_bno055_calib_level_mask;
```

```
01188   const uint8_t full = k_bno055_calib_full;
01189
01190   const uint8_t sys_cal = (calib_raw » k_bno055_calib_sys_shift) & mask;
01191   const uint8_t gyr_cal = (calib_raw » k_bno055_calib_gyr_shift) & mask;
01192   const uint8_t acc_cal = (calib_raw » k_bno055_calib_acc_shift) & mask;
01193   const uint8_t mag_cal = (calib_raw » k_bno055_calib_mag_shift) & mask;
01194
01195   /* Use bitwise & to evaluate all sub-expressions without short-circuit,
01196    * ensuring full branch coverage. */
01197   *out_calibrated =
01198     (bool)(((sys_cal == full) & (gyr_cal == full) & (acc_cal == full) & (mag_cal == full)) != 0);
01199
01200   return k_rx_ok;
01201 }
```

References internal_read_regs(), k_bno055_calib_acc_shift, k_bno055_calib_full, k_bno055_calib_gyr_shift, k_bno055_calib_level_mask, k_bno055_calib_mag_shift, k_bno055_calib_sys_shift, k_bno055_reg_calib_stat, k_bno055_single_byte, s_initialized, and s_tag.

Here is the call graph for this function:

4.1.10.4 rx_bno055_read()

```
rx_err_t rx_bno055_read (
                bno055_data_t * out)   [nodiscard]
```

Read current BNO055 fusion output data.

Performs six I2C read transactions to collect all fusion output data:

1. Read 6 bytes from 0x14: Gyroscope X(2) + Y(2) + Z(2)
2. Read 6 bytes from 0x1A: Euler heading(2) + roll(2) + pitch(2)
3. Read 8 bytes from 0x20: Quaternion W(2) + X(2) + Y(2) + Z(2)
4. Read 6 bytes from 0x28: Linear accel X(2) + Y(2) + Z(2)
5. Read 1 byte from 0x34: Temperature
6. Read 1 byte from 0x35: Calibration status

All multi-byte values are assembled in little-endian format (LSB | (MSB << 8)).

Parameters

out	out	Pointer to output data structure. Must not be NULL. Receives all sensor fusion data on success.

Returns

rx_err_t Read result

Return values

k_rx_ok	Data successfully read into *out
k_rx_err_null_ptr	out is NULL
k_rx_err_not_initialized	rx_bno055_init() not called yet
k_rx_err_nack	I2C NACK during read (sensor not responding)
k_rx_err_timeout	I2C read timeout

Precondition

rx_bno055_init() called successfully

out must point to valid bno055_data_t storage

Postcondition

out->gyro_x/y/z_dps16 contains gyroscope angular rate (divide by 16 for deg/s)

out->heading_deg16 contains Euler heading in 1/16 degree units

out->quat_w/x/y/z contains unit quaternion (divide by 16384)

out->lin_acc_x/y/z contains linear acceleration (divide by 100 for m/s^2)

out->calib_stat contains raw calibration status byte

Note

Not thread-safe; call from single task only

Calibration improves over time as sensor moves; check calib_stat

Performance:

- Execution time: ~2 ms at 400 kHz I2C (6 transactions)
- I2C bytes transferred: 28 bytes read

See also

rx_bno055_is_calibrated() Check calibration before relying on data

bno055_scale_t Scale factor constants for unit conversion

Since

Version 1.0.0

Performs six sequential I2C burst reads to collect all output data. All 16-bit values are assembled in little-endian format as specified in the BNO055 datasheet (LSB register first, MSB register second).

Register read sequence:

1. 0x14: 6 bytes -> Gyroscope X[2], Y[2], Z[2]
2. 0x1A: 6 bytes -> Euler heading[2], roll[2], pitch[2]
3. 0x20: 8 bytes -> Quaternion W[2], X[2], Y[2], Z[2]
4. 0x28: 6 bytes -> Linear accel X[2], Y[2], Z[2]
5. 0x34: 1 byte -> Temperature
6. 0x35: 1 byte -> Calibration status

Precondition

rx_bno055_init() succeeded

out non-NULL

Postcondition

All fields in *out populated on k_rx_ok

out->calib_stat reflects current calibration quality

Note

Not thread-safe

See also

bno055_scale_t Conversion factors for physical units

Since

Version 1.0.0

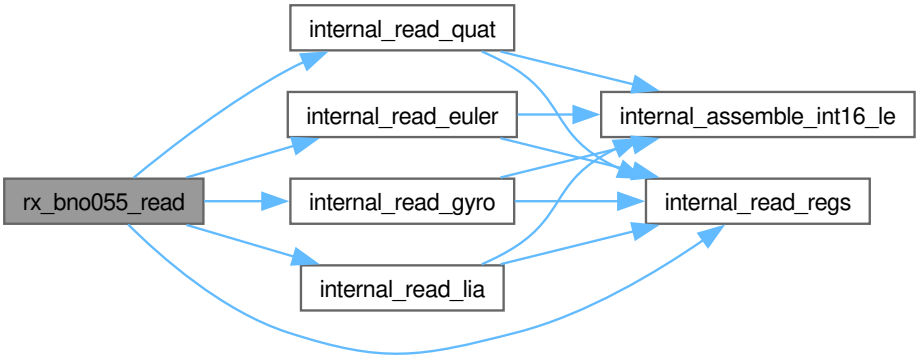
Definition at line 1103 of file rx_bno055.c.

```
01104 {
01105   RX_CHECK_NULL_PTR(out, s_tag, "Output data pointer is NULL");
01106
01107   if (!s_initialized) {
01108     return k_rx_err_not_initialized;
01109   }
01110
01111   rx_err_t err = internal_read_gyro(out);
01112   if (err != k_rx_ok) {
01113     return err;
01114   }
01115
01116   err = internal_read_euler(out);
01117   if (err != k_rx_ok) {
01118     return err;
01119   }
```

```
01120
01121   err = internal_read_quat(out);
01122   if (err != k_rx_ok) {
01123     return err;
01124   }
01125
01126   err = internal_read_lia(out);
01127   if (err != k_rx_ok) {
01128     return err;
01129   }
01130
01131   /* Read Temperature: 1 byte from 0x34 */
01132   uint8_t temp_raw = 0;
01133   err              = internal_read_regs(k_bno055_reg_temp, &temp_raw, k_bno055_single_byte);
01134   if (err != k_rx_ok) {
01135     rx_log_error(s_tag, "Temperature read failed");
01136     return err;
01137   }
01138
01139   /* Read Calibration status: 1 byte from 0x35 */
01140   uint8_t calib_raw = 0;
01141   err               = internal_read_regs(k_bno055_reg_calib_stat, &calib_raw, k_bno055_single_byte);
01142   if (err != k_rx_ok) {
01143     rx_log_error(s_tag, "Calib status read failed");
01144     return err;
01145   }
01146
01147   out->temp_degc  = (int8_t)temp_raw;
01148   out->calib_stat = calib_raw;
01149
01150   return k_rx_ok;
01151 }
```

References bno055_data_t::calib_stat, internal_read_euler(), internal_read_gyro(), internal_read_lia(), internal_read_quat(), internal_read_regs(), k_bno055_reg_calib_stat, k_bno055_reg_temp, k_bno055_single_byte, s_initialized, s_tag, and bno055_data_t::temp_degc.

Here is the call graph for this function:



4.2 rx_bno055.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_bno055.h
00003  * @brief BNO055 9-DOF Absolute Orientation Sensor Driver API
00004  *
00005  * @details
00006  * # Overview
00007  *
00008  * Platform-independent driver for the Bosch BNO055 intelligent 9-axis absolute
00009  * orientation sensor. The BNO055 integrates accelerometer, gyroscope, and
00010  * geomagnetic sensor with an on-chip ARM Cortex-M0 that performs hardware sensor
00011  * fusion to produce absolute orientation data (Euler angles, quaternions, linear
00012  * acceleration).
00013  *
00014  * This driver abstracts the BNO055 initialization sequence and data reading into
00015  * three simple functions: init, read, and calibration check.
00016  *
```

```
00017  * # Key Features
00018  *
00019  * - **Absolute orientation**: Euler angles (heading, roll, pitch) referenced to
00020  *   magnetic north and gravity
00021  * - **Quaternion output**: Unit quaternion for rotation-matrix computation
00022  * - **Linear acceleration**: Gravity-compensated acceleration in m/s^2
00023  * - **Calibration status**: Per-subsystem calibration quality (0-3)
00024  * - **Temperature**: On-chip temperature sensor
00025  *
00026  * # Hardware Configuration (STAR Project)
00027  *
00028  * - **Module**: DFRobot BNO055 breakout
00029  * - **I2C Address**: 0x28 (COM3/ADR pulled LOW)
00030  * - **I2C Bus**: RIIC1 (SCL1=P2.1, SDA1=P2.0)
00031  * - **Bus name in bus manager**: "i2c1_imu"
00032  * - **Reset pin**: P8.3 (active-low, driven HIGH by hardware_init)
00033  *
00034  * # Operating Mode
00035  *
00036  * The driver initializes the sensor in NDOF (Nine Degrees of Freedom) mode with:
00037  * - All three sensors active (accelerometer, gyroscope, magnetometer)
00038  * - Hardware sensor fusion running on the BNO055's ARM Cortex-M0
00039  * - Default measurement units: degrees, m/s^2, dps, Celsius
00040  *
00041  * # Initialization Sequence
00042  *
00043  * Before calling rx_bno055_init(), the caller (imu_task.c) performs a hardware
00044  * reset via P83 (IMU RST, active-low): assert LOW for >= 10 ms, then deassert
00045  * HIGH and wait 650 ms for the BNO055 POR sequence to complete. See
00046  * internal_imu_hardware_reset() in imu_task.c.
00047  *
00048  * The rx_bno055_init() function then performs the following steps:
00049  *
00050  * 1. Software reset (SYS_TRIGGER = 0x20), wait 650 ms (belt-and-suspenders)
00051  * 2. Set CONFIG mode (OPR_MODE = 0x00), wait 19 ms
00052  * 3. Set normal power mode (PWR_MODE = 0x00)
00053  * 4. Set default units (UNIT_SEL = 0x00)
00054  * 5. Set default axis map (AXIS_MAP_CONFIG = 0x24, AXIS_MAP_SIGN = 0x00)
00055  * 6. Clear system trigger (SYS_TRIGGER = 0x00)
00056  * 7. Set NDOF fusion mode (OPR_MODE = 0x0C), wait 7 ms
00057  * 8. Verify chip ID == 0xA0
00058  *
00059  * # Output Data Scales
00060  *
00061  * | Field | Raw Type | Scale | Unit |
00062  * |-------|----------|-------|------|
00063  * | heading_deg16 | int16_t | / 16 | degrees |
00064  * | roll_deg16 | int16_t | / 16 | degrees |
00065  * | pitch_deg16 | int16_t | / 16 | degrees |
00066  * | quat_w/x/y/z | int16_t | / 16384 | dimensionless |
00067  * | lin_acc_x/y/z | int16_t | / 100 | m/s^2 |
00068  * | temp_degc | int8_t | 1.0 | deg C |
00069  * | calib_stat | uint8_t | raw | - |
00070  *
00071  * @see rx_bno055_regs.h Register map and scale constants
00072  * @see rx_bno055.c Implementation
00073  * @see rx_bus_i2c.h I2C bus abstraction used
00074  *
00075  * @par NASA Power of 10 Compliance:
00076  * - **Rule 1** (No goto): [PASS] No goto, recursion, or setjmp
00077  * - **Rule 3** (No heap): [PASS] Zero dynamic allocation
00078  * - **Rule 5** (Assertions): [PASS] 2+ preconditions per function
00079  * - **Rule 7** (Return checks): [PASS] All I2C returns validated
00080  * - **Rule 8** (Preprocessor): [PASS] C23 typed enums, no magic numbers
00081  * - **Rule 9** (Pointers): [WARN] Function pointers for DIP (bus manager)
00082  *
00083  * @par SOLID Principles:
00084  * - **S**: Module handles ONLY BNO055 sensor operations
00085  * - **O**: Extensible via bus manager injection
00086  * - **L**: Any rx_bus_manager_t* is interchangeable
00087  * - **I**: Focused 3-function API
00088  * - **D**: Depends on rx_bus_manager_t abstraction, not RIIC directly
00089  *
00090  * @author STAR Team
00091  * @date 2026-03-04
00092  * @version 1.0.0
00093  * @copyright Copyright (c) 2026 Locked Inc.
00094  * SPDX-License-Identifier: MIT
00095  */
00096
00097 #pragma once
00098
00099 #ifdef __cplusplus
00100 extern "C" {
00101 #endif
00102
00103 #include <stdint.h>
```

```
00104
00105 #include "rx_bus_manager.h"
00106 #include "rx_err.h"
00107
00108 /*
      ================================================================================
00109 * Data Types
00110 *
      ================================================================================
00111 */
00112
00113 /**
00114 * @enum bno055_mode_t
00115 * @brief BNO055 driver operating mode selection
00116 *
00117 * @details
00118 * Selects whether the BNO055 INT pin is configured to assert on each new data
00119 * sample (interrupt-driven mode) or left unconfigured (polling mode).
00120 *
00121 * In interrupt-driven mode, rx_bno055_init() programs five Page 1 registers
00122 * after entering NDOF mode:
00123 * 1. PAGE_ID = 1 (switch to Page 1)
00124 * 2. ACC_AM_THRES = 0x01 (minimum threshold ~3.9 mg, fires at data rate)
00125 * 3. ACC_INT_SETTINGS = 0x07 (enable any-motion on X+Y+Z axes)
00126 * 4. INT_EN = 0x20 (enable accelerometer any-motion interrupt source)
00127 * 5. PAGE_ID = 0 (restore Page 0 for normal sensor reads)
00128 *
00129 * In polling mode, none of these writes occur and the INT pin never asserts.
00130 *
00131 * @see bno055_config_t Configuration struct passed to rx_bno055_init()
00132 *
00133 * @since Version 1.0.0
00134 */
00135 typedef enum : uint8_t {
00136   k_bno055_mode_poll = 0U, /**< Polling mode: INT pin not configured; caller polls at fixed rate */
00137   k_bno055_mode_interrupt = 1U, /**< Interrupt mode: configure BNO055 INT pin via ACC any-motion */
00138 } bno055_mode_t;
00139
00140 /**
00141 * @struct bno055_config_t
00142 * @brief BNO055 driver configuration passed to rx_bno055_init()
00143 *
00144 * @details
00145 * Allows the caller to select polling or interrupt-driven mode. In interrupt
00146 * mode, rx_bno055_init() programs the BNO055 Page 1 interrupt engine registers
00147 * after chip ID verification so that the INT pin asserts on each new sample.
00148 *
00149 * @invariant mode must be a valid bno055_mode_t value
00150 *
00151 * @code
00152 * // Polling mode (no INT pin activity):
00153 * const bno055_config_t cfg = {.mode = k_bno055_mode_poll};
00154 * rx_bno055_init(&manager, &cfg);
00155 *
00156 * // Interrupt mode (BNO055 INT -> RX72N IRQ12):
00157 * const bno055_config_t cfg = {.mode = k_bno055_mode_interrupt};
00158 * rx_bno055_init(&manager, &cfg);
00159 * @endcode
00160 *
00161 * @see bno055_mode_t Mode selection enumeration
00162 * @see rx_bno055_init() Consumes this configuration struct
00163 *
00164 * @since Version 1.0.0
00165 */
00166 typedef struct {
00167   bno055_mode_t mode; /**< Operating mode: k_bno055_mode_poll or k_bno055_mode_interrupt */
00168 } bno055_config_t;
00169
00170 /**
00171 * @enum bno055_calib_shift_t
00172 * @brief BNO055 CALIB_STAT register bit-shift positions for each subsystem
00173 *
00174 * @details
00175 * CALIB_STAT register (0x35) contains four 2-bit calibration level fields.
00176 * Right-shift the raw byte by the appropriate constant, then AND with
00177 * k_bno055_calib_level_mask to extract a value in [0, 3].
00178 *
00179 * @code
00180 * uint8_t sys_lvl = (calib_stat » k_bno055_calib_sys_shift) & k_bno055_calib_level_mask;
00181 * uint8_t mag_lvl = (calib_stat » k_bno055_calib_mag_shift) & k_bno055_calib_level_mask;
00182 * @endcode
00183 *
00184 * @see bno055_calib_mask_t Mask and full-calibration sentinel values
00185 * @see rx_bno055_is_calibrated() Checks all four fields simultaneously
00186 *
00187 * @since Version 1.0.0
00188 */
```

```
00189 typedef enum : uint8_t {
00190   k_bno055_calib_mag_shift = 0U, /**< Bit shift for magnetometer calibration level (bits 1:0) */
00191   k_bno055_calib_acc_shift = 2U, /**< Bit shift for accelerometer calibration level (bits 3:2) */
00192   k_bno055_calib_gyr_shift = 4U, /**< Bit shift for gyroscope calibration level (bits 5:4) */
00193   k_bno055_calib_sys_shift = 6U, /**< Bit shift for system calibration level (bits 7:6) */
00194 } bno055_calib_shift_t;
00195
00196 /**
00197  * @enum bno055_calib_mask_t
00198  * @brief BNO055 calibration level mask and fully-calibrated sentinel
00199  *
00200  * @details
00201  * After shifting a CALIB_STAT field into the LSB position, AND with
00202  * k_bno055_calib_level_mask to isolate the 2-bit value. Compare against
00203  * k_bno055_calib_full (== 3) to check for full calibration.
00204  *
00205  * @see bno055_calib_shift_t Shift constants for each subsystem
00206  * @see rx_bno055_is_calibrated() Uses both mask and full sentinel
00207  *
00208  * @since Version 1.0.0
00209  */
00210 typedef enum : uint8_t {
00211   k_bno055_calib_level_mask = 0x03U, /**< 2-bit mask for a single calibration level (0-3) */
00212   k_bno055_calib_full       = 0x03U, /**< Fully calibrated level */
00213 } bno055_calib_mask_t;
00214
00215 /**
00216  * @enum bno055_scale_t
00217  * @brief BNO055 output data scale factors (divisors for unit conversion)
00218  *
00219  * @details
00220  * Scale factors to convert raw 16-bit integer fields in bno055_data_t to
00221  * physical units. Divide the raw register value by the corresponding constant.
00222  *
00223  * @code
00224  * float heading_deg = (float)data.heading_deg16 / (float)k_bno055_scale_heading;
00225  * float quat_w_f    = (float)data.quat_w        / (float)k_bno055_scale_quat;
00226  * float acc_x_mps2  = (float)data.lin_acc_x     / (float)k_bno055_scale_acc;
00227  * @endcode
00228  *
00229  * @see bno055_data_t Raw integer fields that use these scales
00230  * @see rx_bno055_read() Populates bno055_data_t with raw scaled integers
00231  *
00232  * @since Version 1.0.0
00233  */
00234 typedef enum : uint16_t {
00235   k_bno055_scale_heading =
00236     16U, /**< Degrees-per-unit for heading/roll/pitch (divide by 16 for degrees) */
00237   k_bno055_scale_quat = 16384U, /**< Quaternion unit (2^14; divide by 16384 for normalized float) */
00238   k_bno055_scale_acc  = 100U,   /**< m/s^2 per unit for linear acceleration (divide by 100) */
00239   k_bno055_scale_gyro = 16U,    /**< deg/s per unit for gyro fields (divide by 16 for deg/s) */
00240 } bno055_scale_t;
00241
00242 /**
00243  * @struct bno055_data_t
00244  * @brief BNO055 sensor output data (raw scaled integers)
00245  *
00246  * @details
00247  * All values are stored as raw 16-bit integers to avoid floating-point
00248  * operations in the driver. Convert to physical units using the scale factors
00249  * defined in bno055_scale_t.
00250  *
00251  * **Conversion formulas:**
00252  * @code
00253  * float heading_deg = (float)data.heading_deg16 / (float)k_bno055_scale_heading;
00254  * float roll_deg    = (float)data.roll_deg16    / (float)k_bno055_scale_heading;
00255  * float pitch_deg   = (float)data.pitch_deg16   / (float)k_bno055_scale_heading;
00256  * float quat_w_f    = (float)data.quat_w        / (float)k_bno055_scale_quat;
00257  * float acc_x_mps2  = (float)data.lin_acc_x     / (float)k_bno055_scale_acc;
00258  * @endcode
00259  *
00260  * @invariant quat_w^2 + quat_x^2 + quat_y^2 + quat_z^2 == k_bno055_scale_quat^2 (unit quaternion)
00261  * @invariant heading_deg16 in [0, 360 * k_bno055_scale_heading - 1] (0 to 359.9375 degrees * 16)
00262  * @invariant temp_degc in [-40, 85] (operating range of BNO055)
00263  *
00264  * @see bno055_scale_t Scale factor constants for unit conversion
00265  * @see rx_bno055_read() Populates this structure
00266  *
00267  * @since Version 1.0.0
00268  */
00269 typedef struct {
00270   int16_t
00271     heading_deg16; /**< Euler heading 0-359.9375 deg (divide by k_bno055_scale_heading for degrees) */
00272   int16_t
00273     roll_deg16; /**< Euler roll -90 to +90 deg (divide by k_bno055_scale_heading for degrees) */
00274   int16_t
00275     pitch_deg16; /**< Euler pitch -180 to +180 deg (divide by k_bno055_scale_heading for degrees) */
```

```
00276   int16_t quat_w; /**< Quaternion W component (divide by k_bno055_scale_quat for unit quaternion) */
00277   int16_t quat_x; /**< Quaternion X component (divide by k_bno055_scale_quat for unit quaternion) */
00278   int16_t quat_y; /**< Quaternion Y component (divide by k_bno055_scale_quat for unit quaternion) */
00279   int16_t quat_z; /**< Quaternion Z component (divide by k_bno055_scale_quat for unit quaternion) */
00280   int16_t
00281     lin_acc_x; /**< Linear accel X in m/s^2 * k_bno055_scale_acc (divide by k_bno055_scale_acc for m/s^2) */
00282   int16_t
00283     lin_acc_y; /**< Linear accel Y in m/s^2 * k_bno055_scale_acc (divide by k_bno055_scale_acc for m/s^2) */
00284   int16_t
00285     lin_acc_z; /**< Linear accel Z in m/s^2 * k_bno055_scale_acc (divide by k_bno055_scale_acc for m/s^2) */
00286   int16_t gyro_x_dps16; /**< Gyroscope X in dps * k_bno055_scale_gyro (divide by 16 for deg/s) */
00287   int16_t gyro_y_dps16; /**< Gyroscope Y in dps * k_bno055_scale_gyro (divide by 16 for deg/s) */
00288   int16_t gyro_z_dps16; /**< Gyroscope Z in dps * k_bno055_scale_gyro (divide by 16 for deg/s) */
00289   int8_t  temp_degc;    /**< On-chip temperature in degrees Celsius (1 deg C per LSB) */
00290   uint8_t calib_stat;   /**< Raw CALIB_STAT byte: SYS[7:6] GYR[5:4] ACC[3:2] MAG[1:0] */
00291 } bno055_data_t;
00292
00293 /*
      ============================================================================
00294 * Public API
00295 *
      ============================================================================
00296 */
00297
00298 /**
00299 * @brief Initialize BNO055 sensor and configure for NDOF fusion mode
00300 *
00301 * @details
00302 * Performs the complete BNO055 initialization sequence:
00303 * 1. Software reset via SYS_TRIGGER register (wait 650 ms for POR)
00304 * 2. Enter CONFIG mode for register writes (wait 19 ms)
00305 * 3. Configure power mode to Normal
00306 * 4. Set measurement units to default (degrees, m/s^2, dps, Celsius)
00307 * 5. Set default axis mapping (standard orientation)
00308 * 6. Clear system trigger register
00309 * 7. Enter NDOF fusion mode (wait 7 ms for mode transition)
00310 * 8. Verify CHIP_ID register reads 0xA0
00311 * 9. (If config->mode == k_bno055_mode_interrupt) Configure Page 1 interrupt
00312 *    engine so the INT pin asserts on each new accelerometer sample
00313 *
00314 * The function stores the bus manager pointer in a module-static variable
00315 * for use by subsequent rx_bno055_read() calls. This design assumes a single
00316 * BNO055 instance (no multi-instance support needed for STAR hardware).
00317 *
00318 * @param[in] manager Pointer to initialized bus manager.
00319 *                    Must have "i2c1_imu" bus registered and initialized.
00320 *                    Must not be NULL.
00321 * @param[in] config  Pointer to driver configuration selecting poll vs interrupt
00322 *                    mode. Must not be NULL.
00323 *
00324 * @return rx_err_t Initialization result
00325 * @retval k_rx_ok Sensor initialized, NDOF mode active, ready to read
00326 * @retval k_rx_err_null_ptr manager or config is NULL
00327 * @retval k_rx_err_nack I2C NACK - device not found (check address/wiring)
00328 * @retval k_rx_err_invalid_state CHIP_ID mismatch (wrong device or I2C issue)
00329 * @retval k_rx_err_timeout I2C transaction timeout
00330 *
00331 * @pre Caller must have completed hardware reset via P83 (IMU RST, active-low):
00332 *      assert LOW for >= 10 ms, deassert HIGH, wait >= 650 ms for POR to complete.
00333 *      See internal_imu_hardware_reset() in imu_task.c.
00334 * @pre manager must be initialized via rx_bus_manager_init()
00335 * @pre "i2c1_imu" bus must be registered and initialized in manager
00336 * @pre RIIC1 hardware initialized by hardware_init() (riic_init)
00337 * @pre BNO055 powered and connected to I2C bus
00338 *
00339 * @post Sensor in NDOF fusion mode
00340 * @post Internal s_manager pointer stored for subsequent reads
00341 * @post s_initialized flag set to true on success
00342 * @post Sensor actively running sensor fusion (accelerometer + gyro + mag)
00343 * @post INT pin configured to assert on data ready if config->mode == k_bno055_mode_interrupt
00344 *
00345 * @note Not thread-safe. Call from single-threaded initialization context.
00346 * @note Blocks for ~700 ms total due to required startup delays.
00347 * @warning Do not call if hardware_init() has not been called first.
00348 *
00349 * @par Performance:
00350 * - Execution time: ~700 ms (dominated by 650 ms POR delay)
00351 * - I2C transactions: ~12 writes + 1 read (polling); ~17 writes + 1 read (interrupt mode)
00352 *
00353 * @see rx_bno055_read() Read sensor data after initialization
00354 * @see bno055_mode_t Mode selection (poll vs interrupt)
00355 * @see bno055_config_t Configuration struct
00356 * @see bno055_delay_ms_t Required timing constants
00357 *
00358 * @since Version 1.0.0
00359 */
00360 [[nodiscard]] rx_err_t rx_bno055_init(rx_bus_manager_t* manager, const bno055_config_t* config);
```

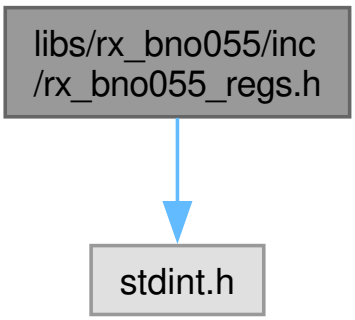
```
00361
00362 /**
00363  * @brief Read current BNO055 fusion output data
00364  *
00365  * @details
00366  * Performs six I2C read transactions to collect all fusion output data:
00367  * 1. Read 6 bytes from 0x14: Gyroscope X(2) + Y(2) + Z(2)
00368  * 2. Read 6 bytes from 0x1A: Euler heading(2) + roll(2) + pitch(2)
00369  * 3. Read 8 bytes from 0x20: Quaternion W(2) + X(2) + Y(2) + Z(2)
00370  * 4. Read 6 bytes from 0x28: Linear accel X(2) + Y(2) + Z(2)
00371  * 5. Read 1 byte from 0x34: Temperature
00372  * 6. Read 1 byte from 0x35: Calibration status
00373  *
00374  * All multi-byte values are assembled in little-endian format (LSB | (MSB « 8)).
00375  *
00376  * @param[out] out Pointer to output data structure. Must not be NULL.
00377  *                 Receives all sensor fusion data on success.
00378  *
00379  * @return rx_err_t Read result
00380  * @retval k_rx_ok Data successfully read into *out
00381  * @retval k_rx_err_null_ptr out is NULL
00382  * @retval k_rx_err_not_initialized rx_bno055_init() not called yet
00383  * @retval k_rx_err_nack I2C NACK during read (sensor not responding)
00384  * @retval k_rx_err_timeout I2C read timeout
00385  *
00386  * @pre rx_bno055_init() called successfully
00387  * @pre out must point to valid bno055_data_t storage
00388  *
00389  * @post out->gyro_x/y/z_dps16 contains gyroscope angular rate (divide by 16 for deg/s)
00390  * @post out->heading_deg16 contains Euler heading in 1/16 degree units
00391  * @post out->quat_w/x/y/z contains unit quaternion (divide by 16384)
00392  * @post out->lin_acc_x/y/z contains linear acceleration (divide by 100 for m/s^2)
00393  * @post out->calib_stat contains raw calibration status byte
00394  *
00395  * @note Not thread-safe; call from single task only
00396  * @note Calibration improves over time as sensor moves; check calib_stat
00397  *
00398  * @par Performance:
00399  * - Execution time: ~2 ms at 400 kHz I2C (6 transactions)
00400  * - I2C bytes transferred: 28 bytes read
00401  *
00402  * @see rx_bno055_is_calibrated() Check calibration before relying on data
00403  * @see bno055_scale_t Scale factor constants for unit conversion
00404  *
00405  * @since Version 1.0.0
00406  */
00407 [[nodiscard]] rx_err_t rx_bno055_read(bno055_data_t* out);
00408
00409 /**
00410  * @brief Check whether all BNO055 subsystems are fully calibrated
00411  *
00412  * @details
00413  * Reads the CALIB_STAT register (0x35) and checks whether all four
00414  * calibration fields (SYS, GYR, ACC, MAG) are at level 3 (fully calibrated).
00415  *
00416  * The BNO055 calibration process:
00417  * - **Magnetometer**: Move sensor in figure-8 pattern
00418  * - **Gyroscope**: Keep sensor stationary for a few seconds
00419  * - **Accelerometer**: Hold in 6 different orientations
00420  * - **System**: All subsystems must be calibrated
00421  *
00422  * Calibration is stored in non-volatile EEPROM inside the BNO055.
00423  *
00424  * @param[out] out_calibrated Pointer to bool to receive result. Must not be NULL.
00425  *                            Set to true if all four subsystems at level 3.
00426  *
00427  * @return rx_err_t Read result
00428  * @retval k_rx_ok out_calibrated set (true or false)
00429  * @retval k_rx_err_null_ptr out_calibrated is NULL
00430  * @retval k_rx_err_not_initialized rx_bno055_init() not called
00431  * @retval k_rx_err_nack I2C communication failure
00432  * @retval k_rx_err_timeout I2C read timeout
00433  *
00434  * @pre rx_bno055_init() called successfully
00435  * @pre out_calibrated points to valid bool storage
00436  *
00437  * @post *out_calibrated == true only if SYS==3 && GYR==3 && ACC==3 && MAG==3
00438  * @post CALIB_STAT register read over I2C (no state modified)
00439  *
00440  * @note Not thread-safe
00441  * @note Calibration status can be monitored in real-time via bno055_data_t.calib_stat
00442  *
00443  * @see bno055_calib_shift_t Bit-shift constants for each calibration subsystem field
00444  * @see bno055_calib_mask_t Mask and fully-calibrated sentinel for CALIB_STAT fields
00445  * @see rx_bno055_read() Reads calib_stat without separate I2C transaction
00446  *
00447  * @since Version 1.0.0
```

```
00448  */
00449 [[nodiscard]] rx_err_t rx_bno055_is_calibrated(bool* out_calibrated);
00450
00451 /**
00452  * @brief Clear the BNO055 INT pin by reading the INT_STA register (Page 0, 0x37)
00453  *
00454  * @details
00455  * Per BNO055 datasheet section 3.6, the INT output pin is asserted when an
00456  * enabled-and-unmasked interrupt source fires and stays asserted until
00457  * firmware reads the INT_STA register. Without this read, the pin latches
00458  * high after the first interrupt and never produces another edge, so
00459  * edge-triggered external IRQs on the host MCU fire exactly once.
00460  *
00461  * Call this after every sensor read (or at any other convenient cadence)
00462  * so the pin re-triggers on the next ACC_BSX_DRDY event.
00463  *
00464  * @return rx_err_t Operation result
00465  * @retval k_rx_ok INT_STA read successfully (pin cleared)
00466  * @retval k_rx_err_invalid_state Driver not yet initialized
00467  * @retval k_rx_err_timeout / k_rx_err_nack I2C failure
00468  *
00469  * @pre rx_bno055_init() completed with k_rx_ok
00470  * @post BNO055 INT pin is low; pending-interrupt bits in INT_STA are cleared
00471  *
00472  * @note Safe to call with no pending interrupts -- the read is idempotent.
00473  * @note Not thread-safe.
00474  */
00475 [[nodiscard]] rx_err_t rx_bno055_clear_int(void);
00476
00477 #ifdef UNIT_TEST
00478 /**
00479  * @brief Reset all driver static state to uninitialized (test-only)
00480  *
00481  * @details
00482  * Clears s_initialized and s_manager so that each unit test starts from a
00483  * clean, uninitialized state regardless of execution order. This function is
00484  * only compiled when UNIT_TEST is defined (host-side builds). It must NOT be
00485  * called from production code.
00486  *
00487  * @pre Driver may be in any state (initialized or not)
00488  * @pre Called only from test setUp() -- never from production firmware
00489  * @post s_initialized == false
00490  * @post s_manager == NULL
00491  *
00492  * @note For unit tests only; not available in firmware builds
00493  * @since Version 1.0.0
00494  */
00495 void rx_bno055_test_reset_state(void);
00496 #endif /* UNIT_TEST */
00497
00498 #ifdef ___cplusplus
00499 }
00500 #endif
```

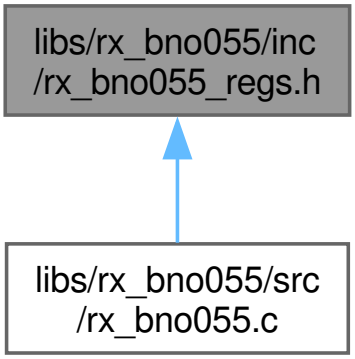
4.3 libs/rx_bno055/inc/rx_bno055_regs.h File Reference

BNO055 9-DOF Absolute Orientation Sensor Register Map and Configuration Enumerations.

#include <stdint.h>
Include dependency graph for rx_bno055_regs.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum bno055_reg_t : uint8_t { k_bno055_reg_chip_id = 0x00 , k_bno055_reg_acc_x_lsb = 0x08 , k_bno055_reg_acc_x_msb = 0x09 , k_bno055_reg_acc_y_lsb = 0x0A , k_bno055_reg_acc_y_msb = 0x0B , k_bno055_reg_acc_z_lsb = 0x0C , k_bno055_reg_acc_z_msb = 0x0D , k_bno055_reg_gyr_x_lsb = 0x14 , k_bno055_reg_gyr_x_msb = 0x15 , k_bno055_reg_gyr_y_lsb = 0x16 , k_bno055_reg_gyr_y_msb = 0x17 , k_bno055_reg_gyr_z_lsb = 0x18 , k_bno055_reg_gyr_z_msb = 0x19 , k_bno055_reg_eul_h_lsb = 0x1A , k_bno055_reg_eul_h_msb = 0x1B , k_bno055_reg_eul_r_lsb = 0x1C , k_bno055_reg_eul_r_msb = 0x1D , k_bno055_reg_eul_p_lsb = 0x1E , k_bno055_reg_eul_p_msb = 0x1F , k_bno055_reg_qua_w_lsb = 0x20 , k_bno055_reg_qua_w_msb = 0x21 , k_bno055_reg_qua_x_lsb = 0x22 , k_bno055_reg_qua_x_msb = 0x23 , k_bno055_reg_qua_y_lsb = 0x24 , k_bno055_reg_qua_y_msb = 0x25 , k_bno055_reg_qua_z_lsb = 0x26 , k_bno055_reg_qua_z_msb = 0x27 , k_bno055_reg_lia_x_lsb = 0x28 , k_bno055_reg_lia_x_msb = 0x29 , k_bno055_reg_lia_y_lsb = 0x2A , k_bno055_reg_lia_y_msb = 0x2B , k_bno055_reg_lia_z_lsb = 0x2C , k_bno055_reg_lia_z_msb = 0x2D , k_bno055_reg_grv_x_lsb = 0x2E , k_bno055_reg_grv_x_msb = 0x2F , k_bno055_reg_grv_y_lsb = 0x30 , k_bno055_reg_grv_y_msb = 0x31 , k_bno055_reg_grv_z_lsb = 0x32 , k_bno055_reg_grv_z_msb = 0x33 , k_bno055_reg_temp = 0x34 , k_bno055_reg_calib_stat = 0x35 , k_bno055_reg_int_sta , k_bno055_reg_unit_sel = 0x3B , k_bno055_reg_opr_mode = 0x3D , k_bno055_reg_pwr_mode = 0x3E , k_bno055_reg_sys_trigger = 0x3F , k_bno055_reg_axis_map_cfg = 0x41 , k_bno055_reg_axis_map_sgn = 0x42 }

  BNO055 register addresses (page 0).
- enum bno055_opr_mode_t : uint8_t { k_bno055_opr_config = 0x00 , k_bno055_opr_ndof = 0x0C }

  BNO055 operating mode selection.
- enum bno055_pwr_mode_t : uint8_t { k_bno055_pwr_normal = 0x00 }

  BNO055 power management modes.
- enum bno055_unit_sel_t : uint8_t { k_bno055_unit_sel_default = 0x00 }

  BNO055 unit selection register values.
- enum bno055_sys_trigger_t : uint8_t { k_bno055_sys_trigger_rst = 0x20 , k_bno055_sys_trigger_clear = 0x00 }

  BNO055 system trigger register values.
- enum bno055_axis_map_t : uint8_t { k_bno055_axis_map_default = 0x24 , k_bno055_axis_map_sign_pos = 0x00 }

BNO055 axis remapping configuration values.

- enum bno055_chip_id_t : uint8_t { k_bno055_chip_id_expected = 0xA0 }

  BNO055 chip identification constant.
- enum bno055_i2c_addr_t : uint8_t { k_bno055_i2c_addr = 0x28 }

  BNO055 I2C device address.
- enum bno055_delay_ms_t : uint16_t { k_bno055_delay_por_ms = 650 , k_bno055_delay_config_ms = 19 , k_bno055_delay_ndof_ms = 7 }

  BNO055 required startup and mode-change delay constants.
- enum bno055_delay_ticks_t : uint8_t { k_bno055_delay_ndof_ticks = 1 , k_bno055_delay_por_ticks = 65 , k_bno055_delay_config_ticks }

  BNO055 ThreadX tick delay constants for mode transitions.
- enum bno055_tick_round_t : uint8_t { k_bno055_tick_round_up }

  Extra tick for rounding fractional delay values.
- enum bno055_regs_scale_t : uint16_t { k_bno055_scale_euler_lsb_per_deg = 16 , k_bno055_scale_accel_lsb_per_mps2 = 100 , k_bno055_scale_gyro_lsb_per_dps = 16 , k_bno055_scale_quat_lsb = 16384 }

  BNO055 register-level output data scale factors (internal register naming).
- enum bno055_read_size_t : uint8_t { k_bno055_euler_bytes = 6U , k_bno055_quat_bytes = 8U , k_bno055_lia_bytes = 6U , k_bno055_gyro_bytes = 6U }

  BNO055 burst read buffer size constants.
- enum bno055_byte_idx_t : uint8_t { k_bno055_idx_lsb = 0U , k_bno055_idx_msb = 1U }

  Byte index constants for little-endian 16-bit assembly.
- enum bno055_shift_t : uint8_t { k_bno055_shift_msb = 8U }

  Bit-shift constant for assembling 16-bit little-endian values.
- enum bno055_tick_rate_t : uint8_t { k_bno055_ms_per_tick = 10 }

  ThreadX tick rate constant for delay conversion.
- enum bno055_time_const_t : uint16_t { k_bno055_ms_per_second = 1000U }

  Time conversion constants for tick-rate validation.
- enum bno055_page_t : uint8_t { k_bno055_page0 = 0x00U , k_bno055_page1 = 0x01U }

  BNO055 register page selection values for the PAGE_ID register.
- enum bno055_int_reg_t : uint8_t { k_bno055_reg_page_id = 0x07U , k_bno055_reg_int_msk = 0x0FU , k_bno055_reg_int_en = 0x10U }

  BNO055 interrupt engine register addresses (PAGE_ID and Page 1 registers).
- enum bno055_int_en_t : uint8_t { k_bno055_int_en_acc_bsx_drdy }

  INT_EN register bit values for enabling interrupt sources (Page 1, 0x10).
- enum bno055_int_msk_t : uint8_t { k_bno055_int_msk_acc_bsx_drdy = 0x01U }

  INT_MSK register bit values for routing interrupt sources to INT pin (Page 1, 0x0F).

4.3.1 Detailed Description

BNO055 9-DOF Absolute Orientation Sensor Register Map and Configuration Enumerations.

4.3.2 Overview

This file provides all register address definitions and configuration constants for the Bosch BNO055 9-DOF absolute orientation sensor. All constants are defined as C23 typed enumerations following STAR project conventions with zero magic numbers.

The BNO055 integrates a triaxial accelerometer, triaxial gyroscope, and triaxial geomagnetic sensor with an on-chip ARM Cortex-M0 that performs sensor fusion to produce absolute orientation quaternions and Euler angles in hardware.

4.3.3 Register Map Summary

Address Range	Contents
0x00	Chip ID (0xA0)
0x08-0x0D	Accelerometer XYZ (raw)
0x14-0x19	Gyroscope XYZ (raw)
0x1A-0x1F	Euler heading/roll/pitch
0x20-0x27	Quaternion W/X/Y/Z
0x28-0x2D	Linear acceleration XYZ
0x2E-0x33	Gravity vector XYZ
0x34	Temperature
0x35	Calibration status
0x3B	Unit selection
0x3D	Operating mode
0x3E	Power mode
0x3F	System trigger
0x41-0x42	Axis map config/sign

4.3.4 DFRobot Module Hardware

- I2C Address: 0x28 (COM3/ADR pulled LOW)
- Connected to RIIC1 (SCL1=P2.1, SDA1=P2.0)
- Power: 3.3V via J5 connector
- Reset: Active-low via P8.3

4.3.5 Scale Factors

Output	Scale	Unit
Euler angles	1/16 deg/LSB	degrees
Quaternion	1/16384/LSB	unit quaternion
Linear accel	1/100 m/s^2/LSB	m/s^2
Gyroscope	1/16 dps/LSB	deg/s

See also

rx_bno055.h Public driver API

rx_bno055.c Driver implementation

BNO055 datasheet section 3.6 for complete register descriptions

NASA Power of 10 Compliance:

- Rule 3: [PASS] No dynamic allocation (constants only)
- Rule 8: [PASS] C23 typed enums for all constants, no macros

Author

STAR Team

Date

2026-03-04

Version

1.0.0

Copyright

Definition in file rx_bno055_regs.h.

4.3.6 Enumeration Type Documentation

4.3.6.1 bno055_axis_map_t

enum bno055_axis_map_t : uint8_t

BNO055 axis remapping configuration values.

Controls axis remapping for mounting orientation compensation. Default values match the standard orientation (X=0, Y=1, Z=2) with positive signs on all axes.

See also

BNO055 datasheet section 3.4 for axis remapping details

Since

Version 1.0.0

Enumerator

k_bno055_axis_map_default	Default axis map: X=0,Y=1,Z=2 (0b00100100)
k_bno055_axis_map_sign_pos	All axes positive sign

Definition at line 251 of file rx_bno055_regs.h.

```
00251             : uint8_t {
00252   k_bno055_axis_map_default  = 0x24, /**< Default axis map: X=0,Y=1,Z=2 (0b00100100) */
00253   k_bno055_axis_map_sign_pos = 0x00, /**< All axes positive sign */
00254 } bno055_axis_map_t;
```

4.3.6.2 bno055_byte_idx_t

enum bno055_byte_idx_t : uint8_t

Byte index constants for little-endian 16-bit assembly.

The BNO055 outputs all multi-byte values in little-endian format (LSB first). These indices reference the LSB and MSB positions within a 2-byte register pair.

```
int16_t val = (int16_t)((uint16_t)buf[k_bno055_idx_lsb] |
                        ((uint16_t)buf[k_bno055_idx_msb] « k_bno055_shift_msb));
```

Since

Version 1.0.0

Enumerator

k_bno055_idx_lsb	Byte index of LSB in a little-endian 2-byte value
k_bno055_idx_msb	Byte index of MSB in a little-endian 2-byte value

Definition at line 428 of file rx_bno055_regs.h.

```
00428             : uint8_t {
00429   k_bno055_idx_lsb = 0U, /**< Byte index of LSB in a little-endian 2-byte value */
00430   k_bno055_idx_msb = 1U, /**< Byte index of MSB in a little-endian 2-byte value */
00431 } bno055_byte_idx_t;
```

4.3.6.3 bno055_chip_id_t

enum bno055_chip_id_t : uint8_t

BNO055 chip identification constant.

Fixed chip ID read from register 0x00 after power-on. Used to verify successful I2C communication and device identity during initialization.

Invariant

CHIP_ID register always reads k_bno055_chip_id_expected on genuine BNO055

Since

Version 1.0.0

Enumerator

k_bno055_chip_id_expected	Factory-programmed chip ID for BNO055

Definition at line 268 of file rx_bno055_regs.h.

```
00268             : uint8_t {
00269   k_bno055_chip_id_expected = 0xA0, /**< Factory-programmed chip ID for BNO055 */
00270 } bno055_chip_id_t;
```

4.3.6.4 bno055_delay_ms_t

enum bno055_delay_ms_t : uint16_t

BNO055 required startup and mode-change delay constants.

Timing requirements from BNO055 datasheet for power-on reset and operating mode transitions. Violating these delays will cause unreliable register reads and communication failures.

Transition	Required Delay
Power-on / software reset	650 ms minimum
CONFIG -> NDOF (fusion modes)	7 ms minimum
Any fusion -> CONFIG	19 ms minimum

See also

BNO055 datasheet table 0-2 (timing characteristics)

Since

Version 1.0.0

Enumerator

k_bno055_delay_por_ms	Power-on reset / software reset boot time (ms)
k_bno055_delay_config_ms	Fusion -> CONFIG mode transition delay (ms)
k_bno055_delay_ndof_ms	CONFIG -> NDOF mode transition delay (ms)

Definition at line 308 of file rx_bno055_regs.h.

```
00308             : uint16_t {
00309     k_bno055_delay_por_ms    = 650, /**< Power-on reset / software reset boot time (ms) */
00310     k_bno055_delay_config_ms = 19,  /**< Fusion -> CONFIG mode transition delay (ms) */
00311     k_bno055_delay_ndof_ms   = 7,   /**< CONFIG -> NDOF mode transition delay (ms) */
00312 } bno055_delay_ms_t;
```

4.3.6.5 bno055_delay_ticks_t

enum bno055_delay_ticks_t : uint8_t

BNO055 ThreadX tick delay constants for mode transitions.

Pre-computed tick counts for tx_thread_sleep() calls during the BNO055 initialization sequence. All values are rounded up to the next 10 ms tick boundary to ensure minimum required delays are met.

Transition	Min Delay	Ticks Used	Actual Delay
POR/reset	650 ms	65 ticks	650 ms exact
-> CONFIG	19 ms	2 ticks	20 ms (round-up)
-> NDOF	7 ms	1 tick	10 ms (round-up)

See also

bno055_delay_ms_t Delay values in milliseconds

rx_bno055_init() Uses these for tx_thread_sleep() calls

Since

Version 1.0.0

Enumerator

k_bno055_delay_ndof_ticks	CONFIG->NDOF transition: 7 ms rounds up to 1 tick (10 ms)
k_bno055_delay_por_ticks	POR/software reset boot: 650 ms / 10 ms per tick = 65 ticks
k_bno055_delay_config_ticks	Fusion->CONFIG transition: 19 ms rounds up to 2 ticks (20 ms)

Definition at line 334 of file rx_bno055_regs.h.

```
00334             : uint8_t {
00335   k_bno055_delay_ndof_ticks = 1, /**< CONFIG->NDOF transition: 7 ms rounds up to 1 tick (10 ms) */
00336   k_bno055_delay_por_ticks = 65, /**< POR/software reset boot: 650 ms / 10 ms per tick = 65 ticks */
00337   k_bno055_delay_config_ticks =
00338     2, /**< Fusion->CONFIG transition: 19 ms rounds up to 2 ticks (20 ms) */
00339 } bno055_delay_ticks_t;
```

4.3.6.6 bno055_i2c_addr_t

enum bno055_i2c_addr_t : uint8_t

BNO055 I2C device address.

The I2C address is determined by the COM3/ADR pin state:

- ADR = LOW (default/DFRobot module): 0x28
- ADR = HIGH: 0x29

The STAR project uses the DFRobot module with ADR pulled LOW.

Since

Version 1.0.0

Enumerator

k_bno055_i2c_addr	I2C device address (COM3/ADR=LOW, DFRobot module)

Definition at line 285 of file rx_bno055_regs.h.

```
00285             : uint8_t {
00286   k_bno055_i2c_addr = 0x28, /**< I2C device address (COM3/ADR=LOW, DFRobot module) */
00287 } bno055_i2c_addr_t;
```

4.3.6.7 bno055_int_en_t

enum bno055_int_en_t : uint8_t

INT_EN register bit values for enabling interrupt sources (Page 1, 0x10).

Each bit in INT_EN enables a specific interrupt source. Bit 0 (k_bno055_int_en_acc_bsx_drdy) enables the accelerometer BSX data-ready interrupt, which asserts on every new accelerometer sample.

INT_EN bit map (BNO055 datasheet Table 4-16):

- Bit 7: Reserved
- Bit 6: ACC_AM_EN – accelerometer any-motion
- Bit 5: ACC_HIGH_G_EN – accelerometer high-g
- Bit 4: Reserved
- Bit 3: GYR_HIGH_RATE_EN
- Bit 2: GYR_AM_EN
- Bit 1: Reserved
- Bit 0: ACC_BSX_DRDY – accelerometer data-ready (this constant)

See also

bno055_int_reg_t k_bno055_reg_int_en register address
bno055_int_msk_t INT_MSK mask values (same bit layout as INT_EN)
rx_bno055.c internal_init_enable_interrupt()

Since

Version 1.0.0

Enumerator

k_bno055_int_en_acc_bsx_drdy	INT_EN bit 0: enable accelerometer data-ready interrupt

Definition at line 569 of file rx_bno055_regs.h.

```
00569             : uint8_t {
00570   k_bno055_int_en_acc_bsx_drdy =
00571     0x01U, /**< INT_EN bit 0: enable accelerometer data-ready interrupt */
00572 } bno055_int_en_t;
```

4.3.6.8 bno055_int_msk_t

enum bno055_int_msk_t : uint8_t

INT_MSK register bit values for routing interrupt sources to INT pin (Page 1, 0x0F).

INT_MSK has the same bit layout as INT_EN (Table 4-15). Setting a bit here routes the corresponding enabled interrupt source to the physical INT pin. Both INT_EN and INT_MSK must be set for the INT pin to assert.

INT_MSK bit 0 (k_bno055_int_msk_acc_bsx_drdy) routes the accelerometer BSX data-ready interrupt to the INT pin so it asserts on every new sample.

See also

bno055_int_reg_t k_bno055_reg_int_msk register address

bno055_int_en_t INT_EN enable values (same bit positions)

rx_bno055.c internal_init_enable_interrupt()

Since

Version 1.0.0

Enumerator

k_bno055_int_msk_acc_bsx_drdy	INT_MSK bit 0: route ACC_BSX_DRDY to INT pin

Definition at line 592 of file rx_bno055_regs.h.

```
00592             : uint8_t {
00593   k_bno055_int_msk_acc_bsx_drdy = 0x01U, /**< INT_MSK bit 0: route ACC_BSX_DRDY to INT pin */
00594 } bno055_int_msk_t;
```

4.3.6.9 bno055_int_reg_t

enum bno055_int_reg_t : uint8_t

BNO055 interrupt engine register addresses (PAGE_ID and Page 1 registers).

These register addresses control the BNO055 interrupt output (INT pin). PAGE_ID (0x07) exists on both pages and selects the active page. All other addresses in this enum are Page 1 registers.

Interrupt configuration sequence (while in CONFIG mode):

1. Write PAGE_ID=1 to switch to Page 1
2. Write INT_MSK (0x0F): route ACC_BSX_DRDY to INT pin
3. Write INT_EN (0x10): enable ACC_BSX_DRDY interrupt source
4. Write PAGE_ID=0 to return to Page 0 for normal operation

Warning

All addresses in this enum (except PAGE_ID) are Page 1 addresses. Writing them without first setting PAGE_ID=1 will corrupt Page 0 sensor output registers (e.g., 0x0F = MAG_RADIUS_MSB on Page 0).

Invariant

All addresses are 8-bit I2C register addresses

See also

bno055_page_t Page selection values

rx_bno055.c internal_init_enable_interrupt()

Since

Version 1.0.0

Enumerator

k_bno055_reg_page_id	PAGE_ID register (both pages): 0=Page0, 1=Page1
k_bno055_reg_int_msk	INT_MSK (Page 1 0x0F): routes interrupt sources to INT pin
k_bno055_reg_int_en	INT_EN (Page 1 0x10): enables interrupt sources

Definition at line 538 of file rx_bno055_regs.h.

```
00538             : uint8_t {
00539   k_bno055_reg_page_id = 0x07U, /**< PAGE_ID register (both pages): 0=Page0, 1=Page1 */
00540   k_bno055_reg_int_msk = 0x0FU, /**< INT_MSK (Page 1 0x0F): routes interrupt sources to INT pin */
00541   k_bno055_reg_int_en  = 0x10U, /**< INT_EN (Page 1 0x10): enables interrupt sources */
00542 } bno055_int_reg_t;
```

4.3.6.10 bno055_opr_mode_t

enum bno055_opr_mode_t : uint8_t

BNO055 operating mode selection.

Controls sensor fusion algorithm. CONFIG mode is required for configuration changes. NDOF (Nine Degrees of Freedom) mode runs full sensor fusion with all three sensors (accelerometer, gyroscope, magnetometer).

Transition times:

- CONFIG -> any fusion mode: 7 ms minimum
- Any fusion mode -> CONFIG: 19 ms minimum

See also

BNO055 datasheet table 3-5 for all operating modes

bno055_delay_ms_t Timing constants

Since

Version 1.0.0

Enumerator

k_bno055_opr_config	Configuration mode (required for register writes)
k_bno055_opr_ndof	NDOF fusion: accel + gyro + mag absolute orientation

Definition at line 182 of file rx_bno055_regs.h.

```
00182             : uint8_t {
00183   k_bno055_opr_config = 0x00, /**< Configuration mode (required for register writes) */
00184   k_bno055_opr_ndof   = 0x0C, /**< NDOF fusion: accel + gyro + mag absolute orientation */
00185 } bno055_opr_mode_t;
```

4.3.6.11 bno055_page_t

enum bno055_page_t : uint8_t

BNO055 register page selection values for the PAGE_ID register.

The BNO055 organizes registers into two pages selected by writing to PAGE_ID (address 0x07 on both pages). Page 0 is the default after reset and contains all sensor output registers. Page 1 contains the interrupt engine configuration registers used to enable the INT output pin.

See also

bno055_int_reg_t Page 1 interrupt register addresses

rx_bno055.c internal_init_enable_interrupt() Uses PAGE_ID to switch pages

Since

Version 1.0.0

Enumerator

k_bno055_page0	Page 0 (default after reset): sensor output, system registers
k_bno055_page1	Page 1: interrupt engine configuration registers

Definition at line 508 of file rx_bno055_regs.h.

```
00508             : uint8_t {
00509   k_bno055_page0 = 0x00U, /**< Page 0 (default after reset): sensor output, system registers */
00510   k_bno055_page1 = 0x01U, /**< Page 1: interrupt engine configuration registers */
00511 } bno055_page_t;
```

4.3.6.12 bno055_pwr_mode_t

enum bno055_pwr_mode_t : uint8_t

BNO055 power management modes.

Normal mode keeps all sensors active. Low-power and suspend modes reduce power consumption at the cost of responsiveness.

See also

BNO055 datasheet section 3.2 for power management details

Since

Version 1.0.0

Enumerator

k_bno055_pwr_normal	Normal power mode: all sensors active

Definition at line 199 of file rx_bno055_regs.h.

```
00199             : uint8_t {
00200   k_bno055_pwr_normal = 0x00, /**< Normal power mode: all sensors active */
00201 } bno055_pwr_mode_t;
```

4.3.6.13 bno055_read_size_t

enum bno055_read_size_t : uint8_t

BNO055 burst read buffer size constants.

Defines byte counts for burst I2C reads of contiguous register blocks. Used to pre-size stack buffers in driver functions.

Since

Version 1.0.0

Enumerator

k_bno055_euler_bytes	Euler angles: heading(2) + roll(2) + pitch(2)
k_bno055_quat_bytes	Quaternion: W(2) + X(2) + Y(2) + Z(2)
k_bno055_lia_bytes	Linear acceleration: X(2) + Y(2) + Z(2)
k_bno055_gyro_bytes	Gyroscope: X(2) + Y(2) + Z(2)

Definition at line 406 of file rx_bno055_regs.h.

```
00406             : uint8_t {
00407   k_bno055_euler_bytes = 6U, /**< Euler angles: heading(2) + roll(2) + pitch(2) */
00408   k_bno055_quat_bytes  = 8U, /**< Quaternion: W(2) + X(2) + Y(2) + Z(2) */
00409   k_bno055_lia_bytes   = 6U, /**< Linear acceleration: X(2) + Y(2) + Z(2) */
00410   k_bno055_gyro_bytes  = 6U, /**< Gyroscope: X(2) + Y(2) + Z(2) */
00411 } bno055_read_size_t;
```

4.3.6.14 bno055_reg_t

enum bno055_reg_t : uint8_t

BNO055 register addresses (page 0).

Complete register address map for the BNO055 sensor. All register addresses reference page 0 (default page after reset).

The sensor organizes its output registers as follows:

- Raw sensor data: accelerometer (0x08-0x0D), gyroscope (0x14-0x19)
- Fusion output: Euler (0x1A-0x1F), quaternion (0x20-0x27), linear accel (0x28-0x2D), gravity (0x2E-0x33)
- System: temperature (0x34), calibration (0x35)
- Configuration: unit select (0x3B), operating mode (0x3D), power mode (0x3E), sys trigger (0x3F), axis map (0x41-0x42)

Invariant

All values are 8-bit I2C register addresses

k_bno055_reg_chip_id reads back k_bno055_chip_id_expected on power-up

Note

Enum constants use the STAR project k_ prefix convention for typed enum constants (as opposed to SCREAMING_SNAKE_CASE which applies to #define macros). This is intentional per STAR coding standards (CLAUDE.md).

See also

bno055_chip_id_t Expected chip ID value

BNO055 datasheet table 4-2 for complete page 0 register map

Since

Version 1.0.0

Enumerator

k_bno055_reg_chip_id	Chip ID register - reads 0xA0
k_bno055_reg_acc_x_lsb	Raw accel X low byte
k_bno055_reg_acc_x_msb	Raw accel X high byte
k_bno055_reg_acc_y_lsb	Raw accel Y low byte
k_bno055_reg_acc_y_msb	Raw accel Y high byte
k_bno055_reg_acc_z_lsb	Raw accel Z low byte

k_bno055_reg_acc_z_msb	Raw accel Z high byte
k_bno055_reg_gyr_x_lsb	Raw gyro X low byte
k_bno055_reg_gyr_x_msb	Raw gyro X high byte
k_bno055_reg_gyr_y_lsb	Raw gyro Y low byte
k_bno055_reg_gyr_y_msb	Raw gyro Y high byte
k_bno055_reg_gyr_z_lsb	Raw gyro Z low byte
k_bno055_reg_gyr_z_msb	Raw gyro Z high byte
k_bno055_reg_eul_h_lsb	Euler heading low byte (0-359.9375 deg)
k_bno055_reg_eul_h_msb	Euler heading high byte
k_bno055_reg_eul_r_lsb	Euler roll low byte (-90 to +90 deg)
k_bno055_reg_eul_r_msb	Euler roll high byte
k_bno055_reg_eul_p_lsb	Euler pitch low byte (-180 to +180 deg)
k_bno055_reg_eul_p_msb	Euler pitch high byte
k_bno055_reg_qua_w_lsb	Quaternion W low byte
k_bno055_reg_qua_w_msb	Quaternion W high byte
k_bno055_reg_qua_x_lsb	Quaternion X low byte
k_bno055_reg_qua_x_msb	Quaternion X high byte
k_bno055_reg_qua_y_lsb	Quaternion Y low byte
k_bno055_reg_qua_y_msb	Quaternion Y high byte
k_bno055_reg_qua_z_lsb	Quaternion Z low byte
k_bno055_reg_qua_z_msb	Quaternion Z high byte
k_bno055_reg_lia_x_lsb	Linear accel X low byte (m/s^2, gravity-free)
k_bno055_reg_lia_x_msb	Linear accel X high byte
k_bno055_reg_lia_y_lsb	Linear accel Y low byte
k_bno055_reg_lia_y_msb	Linear accel Y high byte
k_bno055_reg_lia_z_lsb	Linear accel Z low byte
k_bno055_reg_lia_z_msb	Linear accel Z high byte
k_bno055_reg_grv_x_lsb	Gravity vector X low byte
k_bno055_reg_grv_x_msb	Gravity vector X high byte
k_bno055_reg_grv_y_lsb	Gravity vector Y low byte
k_bno055_reg_grv_y_msb	Gravity vector Y high byte
k_bno055_reg_grv_z_lsb	Gravity vector Z low byte
k_bno055_reg_grv_z_msb	Gravity vector Z high byte
k_bno055_reg_temp	Temperature in deg C (1 degC/LSB)
k_bno055_reg_calib_stat	Calibration status: SYS[7:6] GYR[5:4] ACC[3:2] MAG[1:0]
k_bno055_reg_int_sta	Interrupt Status (Page 0) – reading clears INT pin per DS 3.6
k_bno055_reg_unit_sel	Unit selection register
k_bno055_reg_opr_mode	Operating mode register
k_bno055_reg_pwr_mode	Power mode register
k_bno055_reg_sys_trigger	System trigger (reset, self-test)
k_bno055_reg_axis_map_cfg	Axis remapping configuration
k_bno055_reg_axis_map_sgn	Axis remapping sign

Definition at line 107 of file rx_bno055_regs.h.

```
00107             : uint8_t {
00108   k_bno055_reg_chip_id    = 0x00, /**< Chip ID register - reads 0xA0 */
00109   k_bno055_reg_acc_x_lsb  = 0x08, /**< Raw accel X low byte */
00110   k_bno055_reg_acc_x_msb  = 0x09, /**< Raw accel X high byte */
00111   k_bno055_reg_acc_y_lsb  = 0x0A, /**< Raw accel Y low byte */
00112   k_bno055_reg_acc_y_msb  = 0x0B, /**< Raw accel Y high byte */
00113   k_bno055_reg_acc_z_lsb  = 0x0C, /**< Raw accel Z low byte */
00114   k_bno055_reg_acc_z_msb  = 0x0D, /**< Raw accel Z high byte */
00115   k_bno055_reg_gyr_x_lsb  = 0x14, /**< Raw gyro X low byte */
00116   k_bno055_reg_gyr_x_msb  = 0x15, /**< Raw gyro X high byte */
00117   k_bno055_reg_gyr_y_lsb  = 0x16, /**< Raw gyro Y low byte */
00118   k_bno055_reg_gyr_y_msb  = 0x17, /**< Raw gyro Y high byte */
00119   k_bno055_reg_gyr_z_lsb  = 0x18, /**< Raw gyro Z low byte */
00120   k_bno055_reg_gyr_z_msb  = 0x19, /**< Raw gyro Z high byte */
00121   k_bno055_reg_eul_h_lsb  = 0x1A, /**< Euler heading low byte (0-359.9375 deg) */
00122   k_bno055_reg_eul_h_msb  = 0x1B, /**< Euler heading high byte */
00123   k_bno055_reg_eul_r_lsb  = 0x1C, /**< Euler roll low byte (-90 to +90 deg) */
00124   k_bno055_reg_eul_r_msb  = 0x1D, /**< Euler roll high byte */
00125   k_bno055_reg_eul_p_lsb  = 0x1E, /**< Euler pitch low byte (-180 to +180 deg) */
00126   k_bno055_reg_eul_p_msb  = 0x1F, /**< Euler pitch high byte */
00127   k_bno055_reg_qua_w_lsb  = 0x20, /**< Quaternion W low byte */
00128   k_bno055_reg_qua_w_msb  = 0x21, /**< Quaternion W high byte */
00129   k_bno055_reg_qua_x_lsb  = 0x22, /**< Quaternion X low byte */
00130   k_bno055_reg_qua_x_msb  = 0x23, /**< Quaternion X high byte */
00131   k_bno055_reg_qua_y_lsb  = 0x24, /**< Quaternion Y low byte */
00132   k_bno055_reg_qua_y_msb  = 0x25, /**< Quaternion Y high byte */
00133   k_bno055_reg_qua_z_lsb  = 0x26, /**< Quaternion Z low byte */
00134   k_bno055_reg_qua_z_msb  = 0x27, /**< Quaternion Z high byte */
00135   k_bno055_reg_lia_x_lsb  = 0x28, /**< Linear accel X low byte (m/s^2, gravity-free) */
00136   k_bno055_reg_lia_x_msb  = 0x29, /**< Linear accel X high byte */
00137   k_bno055_reg_lia_y_lsb  = 0x2A, /**< Linear accel Y low byte */
00138   k_bno055_reg_lia_y_msb  = 0x2B, /**< Linear accel Y high byte */
00139   k_bno055_reg_lia_z_lsb  = 0x2C, /**< Linear accel Z low byte */
00140   k_bno055_reg_lia_z_msb  = 0x2D, /**< Linear accel Z high byte */
00141   k_bno055_reg_grv_x_lsb  = 0x2E, /**< Gravity vector X low byte */
00142   k_bno055_reg_grv_x_msb  = 0x2F, /**< Gravity vector X high byte */
00143   k_bno055_reg_grv_y_lsb  = 0x30, /**< Gravity vector Y low byte */
00144   k_bno055_reg_grv_y_msb  = 0x31, /**< Gravity vector Y high byte */
00145   k_bno055_reg_grv_z_lsb  = 0x32, /**< Gravity vector Z low byte */
00146   k_bno055_reg_grv_z_msb  = 0x33, /**< Gravity vector Z high byte */
00147   k_bno055_reg_temp       = 0x34, /**< Temperature in deg C (1 degC/LSB) */
00148   k_bno055_reg_calib_stat = 0x35, /**< Calibration status: SYS[7:6] GYR[5:4] ACC[3:2] MAG[1:0] */
00149   k_bno055_reg_int_sta =
00150     0x37, /**< Interrupt Status (Page 0) -- reading clears INT pin per DS 3.6 */
00151   k_bno055_reg_unit_sel     = 0x3B, /**< Unit selection register */
00152   k_bno055_reg_opr_mode     = 0x3D, /**< Operating mode register */
00153   k_bno055_reg_pwr_mode     = 0x3E, /**< Power mode register */
00154   k_bno055_reg_sys_trigger  = 0x3F, /**< System trigger (reset, self-test) */
00155   k_bno055_reg_axis_map_cfg = 0x41, /**< Axis remapping configuration */
00156   k_bno055_reg_axis_map_sgn = 0x42, /**< Axis remapping sign */
00157 } bno055_reg_t;
```

4.3.6.15 bno055_regs_scale_t

enum bno055_regs_scale_t : uint16_t

BNO055 register-level output data scale factors (internal register naming).

Scale factors to convert raw 16-bit integer output to physical units. Divide the raw register value by the corresponding scale constant. For the public API scale constants used with bno055_data_t, see bno055_scale_t in rx_bno055.h.

Output Register	Scale	Physical Unit
Euler angles	16	degrees
Quaternion W/X/Y/Z	16384	dimensionless (unit quaternion)
Linear acceleration	100	m/s^2
Gyroscope	16	deg/s

See also

bno055_scale_t Public API scale constants in rx_bno055.h

Since

Version 1.0.0

Enumerator

k_bno055_scale_euler_lsb_per_deg	Euler angle: 16 LSB per degree
k_bno055_scale_accel_lsb_per_mps2	Linear accel: 100 LSB per m/s^2
k_bno055_scale_gyro_lsb_per_dps	Gyroscope: 16 LSB per deg/s
k_bno055_scale_quat_lsb	Quaternion: 16384 LSB per unit

Definition at line 381 of file rx_bno055_regs.h.

```
00381             : uint16_t {
00382   k_bno055_scale_euler_lsb_per_deg  = 16,    /**< Euler angle: 16 LSB per degree */
00383   k_bno055_scale_accel_lsb_per_mps2 = 100,   /**< Linear accel: 100 LSB per m/s^2 */
00384   k_bno055_scale_gyro_lsb_per_dps   = 16,    /**< Gyroscope: 16 LSB per deg/s */
00385   k_bno055_scale_quat_lsb           = 16384, /**< Quaternion: 16384 LSB per unit */
00386 } bno055_regs_scale_t;
```

4.3.6.16 bno055_shift_t

enum bno055_shift_t : uint8_t

Bit-shift constant for assembling 16-bit little-endian values.

The MSB of a 16-bit little-endian pair must be shifted left by 8 bits to place it in the upper byte position when assembling the full value.

See also

bno055_byte_idx_t Byte index constants used with this shift

Since

Version 1.0.0

Enumerator

k_bno055_shift_msb	Bit-shift to place MSB into upper byte position

Definition at line 445 of file rx_bno055_regs.h.

```
00445             : uint8_t {
00446   k_bno055_shift_msb = 8U, /**< Bit-shift to place MSB into upper byte position */
00447 } bno055_shift_t;
```

4.3.6.17 bno055_sys_trigger_t

enum bno055_sys_trigger_t : uint8_t

BNO055 system trigger register values.

Controls system reset and self-test initiation. Writing k_bno055_sys_trigger_rst issues a software reset; the sensor reboots and takes ~650 ms to restart.

See also

BNO055 datasheet section 3.6.7 for SYS_TRIGGER details

bno055_delay_ms_t Boot delay constant

Since

Version 1.0.0

Enumerator

k_bno055_sys_trigger_rst	Software reset command (bit 5)
k_bno055_sys_trigger_clear	Clear trigger register (normal operation)

Definition at line 233 of file rx_bno055_regs.h.

```
00233             : uint8_t {
00234   k_bno055_sys_trigger_rst   = 0x20, /**< Software reset command (bit 5) */
00235   k_bno055_sys_trigger_clear = 0x00, /**< Clear trigger register (normal operation) */
00236 } bno055_sys_trigger_t;
```

4.3.6.18 bno055_tick_rate_t

enum bno055_tick_rate_t : uint8_t

ThreadX tick rate constant for delay conversion.

The ThreadX RTOS is configured at 100 Hz tick rate (10 ms per tick). Delay values in bno055_delay_ms_t (milliseconds) must be divided by k_bno055_ms_per_tick to compute the correct tx_thread_sleep() argument.

Conversion: ticks = delay_ms / k_bno055_ms_per_tick

Invariant

k_bno055_ms_per_tick > 0 (must be a valid divisor)

Warning

Assumes TX_TIMER_TICKS_PER_SECOND == 100. Must be updated if ThreadX tick rate changes (e.g., to 1000 Hz for 1 ms ticks).

See also

bno055_delay_ms_t Delay constants in milliseconds

rx_bno055_init() Uses these tick conversions

Since

Version 1.0.0

Enumerator

k_bno055_ms_per_tick	ThreadX tick period: 10 ms at 100 Hz tick rate

Definition at line 470 of file rx_bno055_regs.h.

```
00470             : uint8_t {
00471   k_bno055_ms_per_tick = 10, /**< ThreadX tick period: 10 ms at 100 Hz tick rate */
00472 } bno055_tick_rate_t;
```

4.3.6.19 bno055_tick_round_t

enum bno055_tick_round_t : uint8_t

Extra tick for rounding fractional delay values.

When a delay in milliseconds does not divide evenly by the tick period (k_bno055_ms_per_tick = 10 ms), add one extra tick to guarantee the minimum required delay is met. Used in the CONFIG mode transition (19 ms rounds up to 20 ms = 2 ticks).

See also

bno055_delay_ticks_t Delay tick constants

rx_bno055_init() Uses this in tx_thread_sleep() calls

Since

Version 1.0.0

Enumerator

k_bno055_tick_round_up	Extra tick added to round fractional delays up to next tick boundary

Definition at line 356 of file rx_bno055_regs.h.

```
00356             : uint8_t {
00357   k_bno055_tick_round_up =
00358     1, /**< Extra tick added to round fractional delays up to next tick boundary */
00359 } bno055_tick_round_t;
```

4.3.6.20 bno055_time_const_t

enum bno055_time_const_t : uint16_t

Time conversion constants for tick-rate validation.

Named constant for milliseconds per second, used in the compile-time assertion that verifies k_bno055_ms_per_tick matches TX_TIMER_TICKS_PER_SECOND.

Since

Version 1.0.0

Enumerator

k_bno055_ms_per_second	Milliseconds per second (for tick-rate conversion)

Definition at line 484 of file rx_bno055_regs.h.

```
00484             : uint16_t {
00485   k_bno055_ms_per_second = 1000U, /**< Milliseconds per second (for tick-rate conversion) */
00486 } bno055_time_const_t;
```

4.3.6.21 bno055_unit_sel_t

enum bno055_unit_sel_t : uint8_t

BNO055 unit selection register values.

Selects measurement units for each sensor output. Default configuration uses degrees for Euler angles, m/s^2 for acceleration, dps for gyroscope, and Celsius for temperature.

See also

BNO055 datasheet section 3.6.5 for unit selection details

Since

Version 1.0.0

Enumerator

k_bno055_unit_sel_default	Degrees, m/s^2, dps, Celsius (factory default)

Definition at line 216 of file rx_bno055_regs.h.

```
00216             : uint8_t {
00217   k_bno055_unit_sel_default = 0x00, /**< Degrees, m/s^2, dps, Celsius (factory default) */
00218 } bno055_unit_sel_t;
```

4.4 rx_bno055_regs.h

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_bno055_regs.h
00003  * @brief BNO055 9-DOF Absolute Orientation Sensor Register Map and Configuration Enumerations
00004  *
00005  * @details
00006  * # Overview
00007  *
00008  * This file provides all register address definitions and configuration constants for the
00009  * Bosch BNO055 9-DOF absolute orientation sensor. All constants are defined as C23 typed
00010  * enumerations following STAR project conventions with zero magic numbers.
00011  *
00012  * The BNO055 integrates a triaxial accelerometer, triaxial gyroscope, and triaxial
00013  * geomagnetic sensor with an on-chip ARM Cortex-M0 that performs sensor fusion to
00014  * produce absolute orientation quaternions and Euler angles in hardware.
00015  *
00016  * # Register Map Summary
00017  *
00018  * | Address Range | Contents |
00019  * |---------------|----------|
00020  * | 0x00 | Chip ID (0xA0) |
00021  * | 0x08-0x0D | Accelerometer XYZ (raw) |
00022  * | 0x14-0x19 | Gyroscope XYZ (raw) |
00023  * | 0x1A-0x1F | Euler heading/roll/pitch |
00024  * | 0x20-0x27 | Quaternion W/X/Y/Z |
00025  * | 0x28-0x2D | Linear acceleration XYZ |
00026  * | 0x2E-0x33 | Gravity vector XYZ |
00027  * | 0x34 | Temperature |
00028  * | 0x35 | Calibration status |
00029  * | 0x3B | Unit selection |
00030  * | 0x3D | Operating mode |
00031  * | 0x3E | Power mode |
00032  * | 0x3F | System trigger |
00033  * | 0x41-0x42 | Axis map config/sign |
00034  *
00035  * # DFRobot Module Hardware
00036  *
00037  * - I2C Address: 0x28 (COM3/ADR pulled LOW)
00038  * - Connected to RIIC1 (SCL1=P2.1, SDA1=P2.0)
00039  * - Power: 3.3V via J5 connector
00040  * - Reset: Active-low via P8.3
00041  *
00042  * # Scale Factors
00043  *
00044  * | Output | Scale | Unit |
00045  * |--------|-------|------|
00046  * | Euler angles | 1/16 deg/LSB | degrees |
00047  * | Quaternion | 1/16384/LSB | unit quaternion |
00048  * | Linear accel | 1/100 m/s^2/LSB | m/s^2 |
00049  * | Gyroscope | 1/16 dps/LSB | deg/s |
00050  *
00051  * @see rx_bno055.h Public driver API
00052  * @see rx_bno055.c Driver implementation
00053  * @see BNO055 datasheet section 3.6 for complete register descriptions
00054  *
00055  * @par NASA Power of 10 Compliance:
00056  * - **Rule 3:** [PASS] No dynamic allocation (constants only)
00057  * - **Rule 8:** [PASS] C23 typed enums for all constants, no macros
00058  *
00059  * @author STAR Team
00060  * @date 2026-03-04
00061  * @version 1.0.0
00062  * @copyright Copyright (c) 2026 Locked Inc.
00063  * SPDX-License-Identifier: MIT
00064  */
00065
00066 #pragma once
00067
00068 #include <stdint.h>
00069
00070 #ifdef __cplusplus
00071 extern "C" {
00072 #endif
00073
00074 /*
      ================================================================================
00075  * Register Address Map
00076  *
      ================================================================================
00077  */
00078
00079 /**
00080  * @enum bno055_reg_t
```

```
00081  * @brief BNO055 register addresses (page 0)
00082  *
00083  * @details
00084  * Complete register address map for the BNO055 sensor. All register
00085  * addresses reference page 0 (default page after reset).
00086  *
00087  * The sensor organizes its output registers as follows:
00088  * - Raw sensor data: accelerometer (0x08-0x0D), gyroscope (0x14-0x19)
00089  * - Fusion output: Euler (0x1A-0x1F), quaternion (0x20-0x27),
00090  *   linear accel (0x28-0x2D), gravity (0x2E-0x33)
00091  * - System: temperature (0x34), calibration (0x35)
00092  * - Configuration: unit select (0x3B), operating mode (0x3D),
00093  *   power mode (0x3E), sys trigger (0x3F), axis map (0x41-0x42)
00094  *
00095  * @invariant All values are 8-bit I2C register addresses
00096  * @invariant k_bno055_reg_chip_id reads back k_bno055_chip_id_expected on power-up
00097  *
00098  * @note Enum constants use the STAR project k_ prefix convention for typed enum
00099  *       constants (as opposed to SCREAMING_SNAKE_CASE which applies to \#define macros).
00100  *       This is intentional per STAR coding standards (CLAUDE.md).
00101  *
00102  * @see bno055_chip_id_t Expected chip ID value
00103  * @see BNO055 datasheet table 4-2 for complete page 0 register map
00104  *
00105  * @since Version 1.0.0
00106  */
00107 typedef enum : uint8_t {
00108   k_bno055_reg_chip_id    = 0x00, /**< Chip ID register - reads 0xA0 */
00109   k_bno055_reg_acc_x_lsb  = 0x08, /**< Raw accel X low byte */
00110   k_bno055_reg_acc_x_msb  = 0x09, /**< Raw accel X high byte */
00111   k_bno055_reg_acc_y_lsb  = 0x0A, /**< Raw accel Y low byte */
00112   k_bno055_reg_acc_y_msb  = 0x0B, /**< Raw accel Y high byte */
00113   k_bno055_reg_acc_z_lsb  = 0x0C, /**< Raw accel Z low byte */
00114   k_bno055_reg_acc_z_msb  = 0x0D, /**< Raw accel Z high byte */
00115   k_bno055_reg_gyr_x_lsb  = 0x14, /**< Raw gyro X low byte */
00116   k_bno055_reg_gyr_x_msb  = 0x15, /**< Raw gyro X high byte */
00117   k_bno055_reg_gyr_y_lsb  = 0x16, /**< Raw gyro Y low byte */
00118   k_bno055_reg_gyr_y_msb  = 0x17, /**< Raw gyro Y high byte */
00119   k_bno055_reg_gyr_z_lsb  = 0x18, /**< Raw gyro Z low byte */
00120   k_bno055_reg_gyr_z_msb  = 0x19, /**< Raw gyro Z high byte */
00121   k_bno055_reg_eul_h_lsb  = 0x1A, /**< Euler heading low byte (0-359.9375 deg) */
00122   k_bno055_reg_eul_h_msb  = 0x1B, /**< Euler heading high byte */
00123   k_bno055_reg_eul_r_lsb  = 0x1C, /**< Euler roll low byte (-90 to +90 deg) */
00124   k_bno055_reg_eul_r_msb  = 0x1D, /**< Euler roll high byte */
00125   k_bno055_reg_eul_p_lsb  = 0x1E, /**< Euler pitch low byte (-180 to +180 deg) */
00126   k_bno055_reg_eul_p_msb  = 0x1F, /**< Euler pitch high byte */
00127   k_bno055_reg_qua_w_lsb  = 0x20, /**< Quaternion W low byte */
00128   k_bno055_reg_qua_w_msb  = 0x21, /**< Quaternion W high byte */
00129   k_bno055_reg_qua_x_lsb  = 0x22, /**< Quaternion X low byte */
00130   k_bno055_reg_qua_x_msb  = 0x23, /**< Quaternion X high byte */
00131   k_bno055_reg_qua_y_lsb  = 0x24, /**< Quaternion Y low byte */
00132   k_bno055_reg_qua_y_msb  = 0x25, /**< Quaternion Y high byte */
00133   k_bno055_reg_qua_z_lsb  = 0x26, /**< Quaternion Z low byte */
00134   k_bno055_reg_qua_z_msb  = 0x27, /**< Quaternion Z high byte */
00135   k_bno055_reg_lia_x_lsb  = 0x28, /**< Linear accel X low byte (m/s^2, gravity-free) */
00136   k_bno055_reg_lia_x_msb  = 0x29, /**< Linear accel X high byte */
00137   k_bno055_reg_lia_y_lsb  = 0x2A, /**< Linear accel Y low byte */
00138   k_bno055_reg_lia_y_msb  = 0x2B, /**< Linear accel Y high byte */
00139   k_bno055_reg_lia_z_lsb  = 0x2C, /**< Linear accel Z low byte */
00140   k_bno055_reg_lia_z_msb  = 0x2D, /**< Linear accel Z high byte */
00141   k_bno055_reg_grv_x_lsb  = 0x2E, /**< Gravity vector X low byte */
00142   k_bno055_reg_grv_x_msb  = 0x2F, /**< Gravity vector X high byte */
00143   k_bno055_reg_grv_y_lsb  = 0x30, /**< Gravity vector Y low byte */
00144   k_bno055_reg_grv_y_msb  = 0x31, /**< Gravity vector Y high byte */
00145   k_bno055_reg_grv_z_lsb  = 0x32, /**< Gravity vector Z low byte */
00146   k_bno055_reg_grv_z_msb  = 0x33, /**< Gravity vector Z high byte */
00147   k_bno055_reg_temp       = 0x34, /**< Temperature in deg C (1 degC/LSB) */
00148   k_bno055_reg_calib_stat = 0x35, /**< Calibration status: SYS[7:6] GYR[5:4] ACC[3:2] MAG[1:0] */
00149   k_bno055_reg_int_sta =
00150     0x37, /**< Interrupt Status (Page 0) -- reading clears INT pin per DS 3.6 */
00151   k_bno055_reg_unit_sel     = 0x3B, /**< Unit selection register */
00152   k_bno055_reg_opr_mode     = 0x3D, /**< Operating mode register */
00153   k_bno055_reg_pwr_mode     = 0x3E, /**< Power mode register */
00154   k_bno055_reg_sys_trigger  = 0x3F, /**< System trigger (reset, self-test) */
00155   k_bno055_reg_axis_map_cfg = 0x41, /**< Axis remapping configuration */
00156   k_bno055_reg_axis_map_sgn = 0x42, /**< Axis remapping sign */
00157 } bno055_reg_t;
00158
00159 /*
      ============================================================================
00160  * Configuration Enumerations
00161  *
      ============================================================================
00162  */
00163
00164 /**
00165  * @enum bno055_opr_mode_t
```

```
00166  * @brief BNO055 operating mode selection
00167  *
00168  * @details
00169  * Controls sensor fusion algorithm. CONFIG mode is required for configuration
00170  * changes. NDOF (Nine Degrees of Freedom) mode runs full sensor fusion with
00171  * all three sensors (accelerometer, gyroscope, magnetometer).
00172  *
00173  * @par Transition times:
00174  * - CONFIG -> any fusion mode: 7 ms minimum
00175  * - Any fusion mode -> CONFIG: 19 ms minimum
00176  *
00177  * @see BNO055 datasheet table 3-5 for all operating modes
00178  * @see bno055_delay_ms_t Timing constants
00179  *
00180  * @since Version 1.0.0
00181  */
00182 typedef enum : uint8_t {
00183   k_bno055_opr_config = 0x00, /**< Configuration mode (required for register writes) */
00184   k_bno055_opr_ndof   = 0x0C, /**< NDOF fusion: accel + gyro + mag absolute orientation */
00185 } bno055_opr_mode_t;
00186
00187 /**
00188  * @enum bno055_pwr_mode_t
00189  * @brief BNO055 power management modes
00190  *
00191  * @details
00192  * Normal mode keeps all sensors active. Low-power and suspend modes
00193  * reduce power consumption at the cost of responsiveness.
00194  *
00195  * @see BNO055 datasheet section 3.2 for power management details
00196  *
00197  * @since Version 1.0.0
00198  */
00199 typedef enum : uint8_t {
00200   k_bno055_pwr_normal = 0x00, /**< Normal power mode: all sensors active */
00201 } bno055_pwr_mode_t;
00202
00203 /**
00204  * @enum bno055_unit_sel_t
00205  * @brief BNO055 unit selection register values
00206  *
00207  * @details
00208  * Selects measurement units for each sensor output. Default configuration
00209  * uses degrees for Euler angles, m/s^2 for acceleration, dps for gyroscope,
00210  * and Celsius for temperature.
00211  *
00212  * @see BNO055 datasheet section 3.6.5 for unit selection details
00213  *
00214  * @since Version 1.0.0
00215  */
00216 typedef enum : uint8_t {
00217   k_bno055_unit_sel_default = 0x00, /**< Degrees, m/s^2, dps, Celsius (factory default) */
00218 } bno055_unit_sel_t;
00219
00220 /**
00221  * @enum bno055_sys_trigger_t
00222  * @brief BNO055 system trigger register values
00223  *
00224  * @details
00225  * Controls system reset and self-test initiation. Writing k_bno055_sys_trigger_rst
00226  * issues a software reset; the sensor reboots and takes ~650 ms to restart.
00227  *
00228  * @see BNO055 datasheet section 3.6.7 for SYS_TRIGGER details
00229  * @see bno055_delay_ms_t Boot delay constant
00230  *
00231  * @since Version 1.0.0
00232  */
00233 typedef enum : uint8_t {
00234   k_bno055_sys_trigger_rst   = 0x20, /**< Software reset command (bit 5) */
00235   k_bno055_sys_trigger_clear = 0x00, /**< Clear trigger register (normal operation) */
00236 } bno055_sys_trigger_t;
00237
00238 /**
00239  * @enum bno055_axis_map_t
00240  * @brief BNO055 axis remapping configuration values
00241  *
00242  * @details
00243  * Controls axis remapping for mounting orientation compensation. Default
00244  * values match the standard orientation (X=0, Y=1, Z=2) with positive signs
00245  * on all axes.
00246  *
00247  * @see BNO055 datasheet section 3.4 for axis remapping details
00248  *
00249  * @since Version 1.0.0
00250  */
00251 typedef enum : uint8_t {
00252   k_bno055_axis_map_default  = 0x24, /**< Default axis map: X=0,Y=1,Z=2 (0b00100100) */
```

```
00253   k_bno055_axis_map_sign_pos = 0x00, /**< All axes positive sign */
00254 } bno055_axis_map_t;
00255
00256 /**
00257  * @enum bno055_chip_id_t
00258  * @brief BNO055 chip identification constant
00259  *
00260  * @details
00261  * Fixed chip ID read from register 0x00 after power-on. Used to verify
00262  * successful I2C communication and device identity during initialization.
00263  *
00264  * @invariant CHIP_ID register always reads k_bno055_chip_id_expected on genuine BNO055
00265  *
00266  * @since Version 1.0.0
00267  */
00268 typedef enum : uint8_t {
00269   k_bno055_chip_id_expected = 0xA0, /**< Factory-programmed chip ID for BNO055 */
00270 } bno055_chip_id_t;
00271
00272 /**
00273  * @enum bno055_i2c_addr_t
00274  * @brief BNO055 I2C device address
00275  *
00276  * @details
00277  * The I2C address is determined by the COM3/ADR pin state:
00278  * - ADR = LOW (default/DFRobot module): 0x28
00279  * - ADR = HIGH: 0x29
00280  *
00281  * The STAR project uses the DFRobot module with ADR pulled LOW.
00282  *
00283  * @since Version 1.0.0
00284  */
00285 typedef enum : uint8_t {
00286   k_bno055_i2c_addr = 0x28, /**< I2C device address (COM3/ADR=LOW, DFRobot module) */
00287 } bno055_i2c_addr_t;
00288
00289 /**
00290  * @enum bno055_delay_ms_t
00291  * @brief BNO055 required startup and mode-change delay constants
00292  *
00293  * @details
00294  * Timing requirements from BNO055 datasheet for power-on reset and
00295  * operating mode transitions. Violating these delays will cause
00296  * unreliable register reads and communication failures.
00297  *
00298  * | Transition | Required Delay |
00299  * |------------|----------------|
00300  * | Power-on / software reset | 650 ms minimum |
00301  * | CONFIG -> NDOF (fusion modes) | 7 ms minimum |
00302  * | Any fusion -> CONFIG | 19 ms minimum |
00303  *
00304  * @see BNO055 datasheet table 0-2 (timing characteristics)
00305  *
00306  * @since Version 1.0.0
00307  */
00308 typedef enum : uint16_t {
00309   k_bno055_delay_por_ms    = 650, /**< Power-on reset / software reset boot time (ms) */
00310   k_bno055_delay_config_ms = 19,  /**< Fusion -> CONFIG mode transition delay (ms) */
00311   k_bno055_delay_ndof_ms   = 7,   /**< CONFIG -> NDOF mode transition delay (ms) */
00312 } bno055_delay_ms_t;
00313
00314 /**
00315  * @enum bno055_delay_ticks_t
00316  * @brief BNO055 ThreadX tick delay constants for mode transitions
00317  *
00318  * @details
00319  * Pre-computed tick counts for tx_thread_sleep() calls during the BNO055
00320  * initialization sequence. All values are rounded up to the next 10 ms tick
00321  * boundary to ensure minimum required delays are met.
00322  *
00323  * | Transition | Min Delay | Ticks Used | Actual Delay |
00324  * |------------|-----------|------------|--------------|
00325  * | POR/reset  | 650 ms    | 65 ticks   | 650 ms exact |
00326  * | -> CONFIG  | 19 ms     | 2 ticks    | 20 ms (round-up) |
00327  * | -> NDOF    | 7 ms      | 1 tick     | 10 ms (round-up) |
00328  *
00329  * @see bno055_delay_ms_t Delay values in milliseconds
00330  * @see rx_bno055_init() Uses these for tx_thread_sleep() calls
00331  *
00332  * @since Version 1.0.0
00333  */
00334 typedef enum : uint8_t {
00335   k_bno055_delay_ndof_ticks = 1, /**< CONFIG->NDOF transition: 7 ms rounds up to 1 tick (10 ms) */
00336   k_bno055_delay_por_ticks = 65, /**< POR/software reset boot: 650 ms / 10 ms per tick = 65 ticks */
00337   k_bno055_delay_config_ticks =
00338     2, /**< Fusion->CONFIG transition: 19 ms rounds up to 2 ticks (20 ms) */
00339 } bno055_delay_ticks_t;
```

```
00340
00341 /**
00342  * @enum bno055_tick_round_t
00343  * @brief Extra tick for rounding fractional delay values
00344  *
00345  * @details
00346  * When a delay in milliseconds does not divide evenly by the tick period
00347  * (k_bno055_ms_per_tick = 10 ms), add one extra tick to guarantee the
00348  * minimum required delay is met. Used in the CONFIG mode transition
00349  * (19 ms rounds up to 20 ms = 2 ticks).
00350  *
00351  * @see bno055_delay_ticks_t Delay tick constants
00352  * @see rx_bno055_init() Uses this in tx_thread_sleep() calls
00353  *
00354  * @since Version 1.0.0
00355  */
00356 typedef enum : uint8_t {
00357   k_bno055_tick_round_up =
00358     1, /**< Extra tick added to round fractional delays up to next tick boundary */
00359 } bno055_tick_round_t;
00360
00361 /**
00362  * @enum bno055_regs_scale_t
00363  * @brief BNO055 register-level output data scale factors (internal register naming)
00364  *
00365  * @details
00366  * Scale factors to convert raw 16-bit integer output to physical units.
00367  * Divide the raw register value by the corresponding scale constant.
00368  * For the public API scale constants used with bno055_data_t, see bno055_scale_t in rx_bno055.h.
00369  *
00370  * | Output Register | Scale | Physical Unit |
00371  * |-----------------|-------|---------------|
00372  * | Euler angles | 16 | degrees |
00373  * | Quaternion W/X/Y/Z | 16384 | dimensionless (unit quaternion) |
00374  * | Linear acceleration | 100 | m/s^2 |
00375  * | Gyroscope | 16 | deg/s |
00376  *
00377  * @see bno055_scale_t Public API scale constants in rx_bno055.h
00378  *
00379  * @since Version 1.0.0
00380  */
00381 typedef enum : uint16_t {
00382   k_bno055_scale_euler_lsb_per_deg  = 16,    /**< Euler angle: 16 LSB per degree */
00383   k_bno055_scale_accel_lsb_per_mps2 = 100,   /**< Linear accel: 100 LSB per m/s^2 */
00384   k_bno055_scale_gyro_lsb_per_dps   = 16,    /**< Gyroscope: 16 LSB per deg/s */
00385   k_bno055_scale_quat_lsb           = 16384, /**< Quaternion: 16384 LSB per unit */
00386 } bno055_regs_scale_t;
00387
00388 /* NOTE: Calibration bit-shift and mask constants have been moved to rx_bno055.h
00389  * as bno055_calib_shift_t and bno055_calib_mask_t (public API types). */
00390
00391 /*
      ================================================================================
00392  * Read Buffer Size Constants
00393  *
      ================================================================================
00394  */
00395
00396 /**
00397  * @enum bno055_read_size_t
00398  * @brief BNO055 burst read buffer size constants
00399  *
00400  * @details
00401  * Defines byte counts for burst I2C reads of contiguous register blocks.
00402  * Used to pre-size stack buffers in driver functions.
00403  *
00404  * @since Version 1.0.0
00405  */
00406 typedef enum : uint8_t {
00407   k_bno055_euler_bytes = 6U, /**< Euler angles: heading(2) + roll(2) + pitch(2) */
00408   k_bno055_quat_bytes  = 8U, /**< Quaternion: W(2) + X(2) + Y(2) + Z(2) */
00409   k_bno055_lia_bytes   = 6U, /**< Linear acceleration: X(2) + Y(2) + Z(2) */
00410   k_bno055_gyro_bytes  = 6U, /**< Gyroscope: X(2) + Y(2) + Z(2) */
00411 } bno055_read_size_t;
00412
00413 /**
00414  * @enum bno055_byte_idx_t
00415  * @brief Byte index constants for little-endian 16-bit assembly
00416  *
00417  * @details
00418  * The BNO055 outputs all multi-byte values in little-endian format (LSB first).
00419  * These indices reference the LSB and MSB positions within a 2-byte register pair.
00420  *
00421  * @code
00422  * int16_t val = (int16_t)((uint16_t)buf[k_bno055_idx_lsb] |
00423  *                         ((uint16_t)buf[k_bno055_idx_msb] << k_bno055_shift_msb));
00424  * @endcode
```

```
00425  *
00426  * @since Version 1.0.0
00427  */
00428 typedef enum : uint8_t {
00429   k_bno055_idx_lsb = 0U, /**< Byte index of LSB in a little-endian 2-byte value */
00430   k_bno055_idx_msb = 1U, /**< Byte index of MSB in a little-endian 2-byte value */
00431 } bno055_byte_idx_t;
00432
00433 /**
00434  * @enum bno055_shift_t
00435  * @brief Bit-shift constant for assembling 16-bit little-endian values
00436  *
00437  * @details
00438  * The MSB of a 16-bit little-endian pair must be shifted left by 8 bits
00439  * to place it in the upper byte position when assembling the full value.
00440  *
00441  * @see bno055_byte_idx_t Byte index constants used with this shift
00442  *
00443  * @since Version 1.0.0
00444  */
00445 typedef enum : uint8_t {
00446   k_bno055_shift_msb = 8U, /**< Bit-shift to place MSB into upper byte position */
00447 } bno055_shift_t;
00448
00449 /**
00450  * @enum bno055_tick_rate_t
00451  * @brief ThreadX tick rate constant for delay conversion
00452  *
00453  * @details
00454  * The ThreadX RTOS is configured at 100 Hz tick rate (10 ms per tick).
00455  * Delay values in bno055_delay_ms_t (milliseconds) must be divided by
00456  * k_bno055_ms_per_tick to compute the correct tx_thread_sleep() argument.
00457  *
00458  * Conversion: ticks = delay_ms / k_bno055_ms_per_tick
00459  *
00460  * @invariant k_bno055_ms_per_tick > 0 (must be a valid divisor)
00461  *
00462  * @warning Assumes TX_TIMER_TICKS_PER_SECOND == 100. Must be updated if
00463  *          ThreadX tick rate changes (e.g., to 1000 Hz for 1 ms ticks).
00464  *
00465  * @see bno055_delay_ms_t Delay constants in milliseconds
00466  * @see rx_bno055_init() Uses these tick conversions
00467  *
00468  * @since Version 1.0.0
00469  */
00470 typedef enum : uint8_t {
00471   k_bno055_ms_per_tick = 10, /**< ThreadX tick period: 10 ms at 100 Hz tick rate */
00472 } bno055_tick_rate_t;
00473
00474 /**
00475  * @enum bno055_time_const_t
00476  * @brief Time conversion constants for tick-rate validation
00477  *
00478  * @details
00479  * Named constant for milliseconds per second, used in the compile-time
00480  * assertion that verifies k_bno055_ms_per_tick matches TX_TIMER_TICKS_PER_SECOND.
00481  *
00482  * @since Version 1.0.0
00483  */
00484 typedef enum : uint16_t {
00485   k_bno055_ms_per_second = 1000U, /**< Milliseconds per second (for tick-rate conversion) */
00486 } bno055_time_const_t;
00487
00488 /*
      ============================================================================
00489  * Page 1 Interrupt Engine Registers
00490  *
      ============================================================================
00491  */
00492
00493 /**
00494  * @enum bno055_page_t
00495  * @brief BNO055 register page selection values for the PAGE_ID register
00496  *
00497  * @details
00498  * The BNO055 organizes registers into two pages selected by writing to PAGE_ID
00499  * (address 0x07 on both pages). Page 0 is the default after reset and contains
00500  * all sensor output registers. Page 1 contains the interrupt engine configuration
00501  * registers used to enable the INT output pin.
00502  *
00503  * @see bno055_int_reg_t Page 1 interrupt register addresses
00504  * @see rx_bno055.c internal_init_enable_interrupt() Uses PAGE_ID to switch pages
00505  *
00506  * @since Version 1.0.0
00507  */
00508 typedef enum : uint8_t {
00509   k_bno055_page0 = 0x00U, /**< Page 0 (default after reset): sensor output, system registers */
```

```
00510   k_bno055_page1 = 0x01U, /**< Page 1: interrupt engine configuration registers */
00511 } bno055_page_t;
00512
00513 /**
00514  * @enum bno055_int_reg_t
00515  * @brief BNO055 interrupt engine register addresses (PAGE_ID and Page 1 registers)
00516  *
00517  * @details
00518  * These register addresses control the BNO055 interrupt output (INT pin).
00519  * PAGE_ID (0x07) exists on both pages and selects the active page.
00520  * All other addresses in this enum are Page 1 registers.
00521  *
00522  * Interrupt configuration sequence (while in CONFIG mode):
00523  * 1. Write PAGE_ID=1 to switch to Page 1
00524  * 2. Write INT_MSK (0x0F): route ACC_BSX_DRDY to INT pin
00525  * 3. Write INT_EN (0x10): enable ACC_BSX_DRDY interrupt source
00526  * 4. Write PAGE_ID=0 to return to Page 0 for normal operation
00527  *
00528  * @warning All addresses in this enum (except PAGE_ID) are Page 1 addresses.
00529  *          Writing them without first setting PAGE_ID=1 will corrupt Page 0
00530  *          sensor output registers (e.g., 0x0F = MAG_RADIUS_MSB on Page 0).
00531  *
00532  * @invariant All addresses are 8-bit I2C register addresses
00533  * @see bno055_page_t Page selection values
00534  * @see rx_bno055.c internal_init_enable_interrupt()
00535  *
00536  * @since Version 1.0.0
00537  */
00538 typedef enum : uint8_t {
00539   k_bno055_reg_page_id = 0x07U, /**< PAGE_ID register (both pages): 0=Page0, 1=Page1 */
00540   k_bno055_reg_int_msk = 0x0FU, /**< INT_MSK (Page 1 0x0F): routes interrupt sources to INT pin */
00541   k_bno055_reg_int_en  = 0x10U, /**< INT_EN (Page 1 0x10): enables interrupt sources */
00542 } bno055_int_reg_t;
00543
00544 /**
00545  * @enum bno055_int_en_t
00546  * @brief INT_EN register bit values for enabling interrupt sources (Page 1, 0x10)
00547  *
00548  * @details
00549  * Each bit in INT_EN enables a specific interrupt source.
00550  * Bit 0 (k_bno055_int_en_acc_bsx_drdy) enables the accelerometer BSX
00551  * data-ready interrupt, which asserts on every new accelerometer sample.
00552  *
00553  * INT_EN bit map (BNO055 datasheet Table 4-16):
00554  * - Bit 7: Reserved
00555  * - Bit 6: ACC_AM_EN -- accelerometer any-motion
00556  * - Bit 5: ACC_HIGH_G_EN -- accelerometer high-g
00557  * - Bit 4: Reserved
00558  * - Bit 3: GYR_HIGH_RATE_EN
00559  * - Bit 2: GYR_AM_EN
00560  * - Bit 1: Reserved
00561  * - Bit 0: ACC_BSX_DRDY -- accelerometer data-ready (this constant)
00562  *
00563  * @see bno055_int_reg_t k_bno055_reg_int_en register address
00564  * @see bno055_int_msk_t INT_MSK mask values (same bit layout as INT_EN)
00565  * @see rx_bno055.c internal_init_enable_interrupt()
00566  *
00567  * @since Version 1.0.0
00568  */
00569 typedef enum : uint8_t {
00570   k_bno055_int_en_acc_bsx_drdy =
00571     0x01U, /**< INT_EN bit 0: enable accelerometer data-ready interrupt */
00572 } bno055_int_en_t;
00573
00574 /**
00575  * @enum bno055_int_msk_t
00576  * @brief INT_MSK register bit values for routing interrupt sources to INT pin (Page 1, 0x0F)
00577  *
00578  * @details
00579  * INT_MSK has the same bit layout as INT_EN (Table 4-15). Setting a bit here
00580  * routes the corresponding enabled interrupt source to the physical INT pin.
00581  * Both INT_EN and INT_MSK must be set for the INT pin to assert.
00582  *
00583  * INT_MSK bit 0 (k_bno055_int_msk_acc_bsx_drdy) routes the accelerometer BSX
00584  * data-ready interrupt to the INT pin so it asserts on every new sample.
00585  *
00586  * @see bno055_int_reg_t k_bno055_reg_int_msk register address
00587  * @see bno055_int_en_t INT_EN enable values (same bit positions)
00588  * @see rx_bno055.c internal_init_enable_interrupt()
00589  *
00590  * @since Version 1.0.0
00591  */
00592 typedef enum : uint8_t {
00593   k_bno055_int_msk_acc_bsx_drdy = 0x01U, /**< INT_MSK bit 0: route ACC_BSX_DRDY to INT pin */
00594 } bno055_int_msk_t;
00595
00596 #ifdef __cplusplus
```

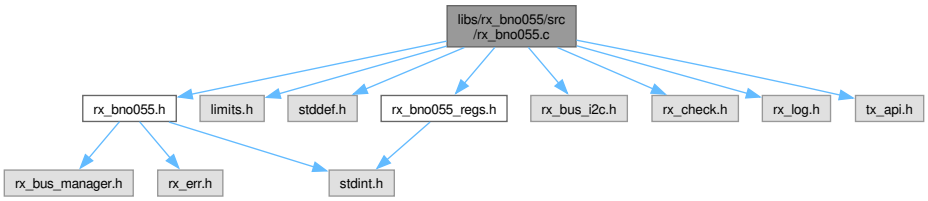
```
00597 }
00598 #endif
```

4.5 libs/rx_bno055/src/rx_bno055.c File Reference

BNO055 9-DOF Absolute Orientation Sensor Driver Implementation.

```
#include "rx_bno055.h"
#include <limits.h>
#include <stddef.h>
#include "rx_bno055_regs.h"
#include "rx_bus_i2c.h"
#include "rx_check.h"
#include "rx_log.h"
#include "tx_api.h"
```

Include dependency graph for rx_bno055.c:



Enumerations

- enum bno055_i2c_size_t : uint8_t { k_bno055_write_buf_size = 2 , k_bno055_read_cmd_size = 1 }

  I2C write/read buffer size constants for register access.
- enum bno055_assert_val_t : uint8_t { k_bno055_expected_msb_shift = 8U , k_bno055_expected_uint16_bits = 16U }

  Compile-time assertion reference values for internal_assemble_int16_le.
- enum bno055_i2c_write_idx_t : uint8_t { k_bno055_write_idx_reg = 0 , k_bno055_write_idx_val = 1 }

  Byte indices into the 2-byte I2C write buffer.
- enum bno055_single_byte_size_t : uint8_t { k_bno055_single_byte = 1 }

  Single byte read/write sizes.
- enum euler_offsets_t : uint8_t {
  k_eul_h_lsb_off = 0 , k_eul_h_msb_off = 1 , k_eul_r_lsb_off = 2 , k_eul_r_msb_off = 3 ,
  k_eul_p_lsb_off = 4 , k_eul_p_msb_off = 5 }

  Byte offsets in the 6-byte Euler angle burst-read buffer.
- enum quat_offsets_t : uint8_t {
  k_qua_w_lsb_off = 0 , k_qua_w_msb_off = 1 , k_qua_x_lsb_off = 2 , k_qua_x_msb_off = 3 ,
  k_qua_y_lsb_off = 4 , k_qua_y_msb_off = 5 , k_qua_z_lsb_off = 6 , k_qua_z_msb_off = 7 }

  Byte offsets in the 8-byte quaternion burst-read buffer.
- enum lia_offsets_t : uint8_t {
  k_lia_x_lsb_off = 0 , k_lia_x_msb_off = 1 , k_lia_y_lsb_off = 2 , k_lia_y_msb_off = 3 ,
  k_lia_z_lsb_off = 4 , k_lia_z_msb_off = 5 }

  Byte offsets in the 6-byte linear acceleration burst-read buffer.

- enum gyro_offsets_t : uint8_t {
  k_gyr_x_lsb_off = 0 , k_gyr_x_msb_off = 1 , k_gyr_y_lsb_off = 2 , k_gyr_y_msb_off = 3 ,
  k_gyr_z_lsb_off = 4 , k_gyr_z_msb_off = 5 }

  Byte offsets within a gyroscope burst read buffer (6 bytes).
- enum bno055_gyro_size_t : uint8_t { k_bno055_gyro_expected_bytes = 6U }

  Expected byte count for a gyroscope burst read.
- enum bno055_quat_size_t : uint8_t { k_bno055_quat_expected_bytes }

  Expected byte count for a quaternion burst read.
- enum bno055_euler_size_t : uint8_t { k_bno055_euler_expected_bytes }

  Expected byte count for an Euler angle burst read.
- enum bno055_lia_size_t : uint8_t { k_bno055_lia_expected_bytes }

  Expected byte count for a linear acceleration burst read.

Functions

- RX_STATIC_TESTABLE rx_err_t internal_write_reg (uint8_t reg, uint8_t val)

  Write a single byte to a BNO055 register via I2C.
- RX_STATIC_TESTABLE rx_err_t internal_read_regs (uint8_t reg, uint8_t *buf, uint8_t len)

  Burst-read consecutive registers from BNO055 via I2C write-read.
- static int16_t internal_assemble_int16_le (uint8_t low, uint8_t high)

  Assemble a little-endian 16-bit signed integer from two bytes.
- RX_STATIC_TESTABLE rx_err_t internal_read_euler (bno055_data_t *out)

  Read Euler angle registers from BNO055 and populate heading/roll/pitch.
- RX_STATIC_TESTABLE rx_err_t internal_read_quat (bno055_data_t *out)

  Read quaternion registers from BNO055 and populate quat_w/x/y/z.
- RX_STATIC_TESTABLE rx_err_t internal_read_lia (bno055_data_t *out)

  Read linear acceleration registers from BNO055 and populate lin_acc_x/y/z.
- RX_STATIC_TESTABLE rx_err_t internal_read_gyro (bno055_data_t *out)

  Read BNO055 gyroscope data (raw, 16-bit little-endian).
- RX_STATIC_TESTABLE rx_err_t internal_init_reset_and_wait (void)

  BNO055 init steps 1-2: software reset, POR delay, CONFIG mode, config delay.
- RX_STATIC_TESTABLE rx_err_t internal_init_configure (void)

  BNO055 init steps 3-6: configure power mode, units, axis map, clear trigger.
- RX_STATIC_TESTABLE rx_err_t internal_init_enter_ndof (void)

  BNO055 init step 7: enter NDOF fusion mode and wait for transition.
- RX_STATIC_TESTABLE rx_err_t internal_verify_chip_id (void)

  BNO055 init step 8: read CHIP_ID register and verify it is 0xA0.
- RX_STATIC_TESTABLE rx_err_t internal_init_enable_interrupt (void)

  Configure BNO055 Page 1 interrupt engine to assert INT on each data sample.
- static rx_err_t internal_init_run_sequence (void)

  Execute the multi-step BNO055 initialization sequence.
- rx_err_t rx_bno055_init (rx_bus_manager_t *manager, const bno055_config_t *config)

  Initialize BNO055 sensor and configure for NDOF fusion mode.
- rx_err_t rx_bno055_read (bno055_data_t *out)

  Read current BNO055 fusion output data.
- rx_err_t rx_bno055_is_calibrated (bool *out_calibrated)

  Check whether all BNO055 subsystems are fully calibrated.
- rx_err_t rx_bno055_clear_int (void)

  Clear the BNO055 INT pin by reading the INT_STA register (Page 0, 0x37).

Variables

- static const char *const s_tag = "BNO055"

  Log tag for this module.
- static rx_bus_manager_t * s_manager = NULL

  Bus manager pointer stored during rx_bno055_init(), used by all I2C operations.
- static bool s_initialized = false

  Guard flag: true only after successful rx_bno055_init().
- static bno055_mode_t s_mode = k_bno055_mode_poll

  Operating mode stored during rx_bno055_init(), used by diagnostic queries.
- static const char *const s_bus_name = "i2c1_imu"

  I2C bus name used by the BNO055 driver to look up the bus manager.

4.5.1 Detailed Description

BNO055 9-DOF Absolute Orientation Sensor Driver Implementation.

4.5.2 Overview

Complete driver implementation for the Bosch BNO055 sensor connected via I2C on RIIC1 (SCL1=P2.1, SDA1=P2.0). The sensor is initialized in NDOF (Nine Degrees of Freedom) sensor fusion mode which provides:

- Absolute Euler angles (heading, roll, pitch)
- Unit quaternion for 3D rotation
- Gravity-free linear acceleration
- On-chip temperature
- Per-subsystem calibration status

4.5.3 Module Architecture

The driver uses two internal helper functions for all I2C communication:

- internal_write_reg(): Writes a single byte to a register
- internal_read_regs(): Burst reads N consecutive registers

Both helpers use the bus manager abstraction with bus name "i2c1_imu". The device address 0x28 is embedded in the "i2c1_imu" bus configuration registered by main.c.

4.5.4 Data Flow

```
imu_task -----> rx_bno055_read()
                    |
                internal_read_regs()
                    |
                rx_bus_i2c_write_read()
                    |
                RIIC1 HAL
                    |
                BNO055 sensor
```

4.5.5 NASA Power of 10 Compliance

Rule	Status	Notes
1. No goto	[PASS]	Structured if/while only
2. Bounded loops	[PASS]	No loops in this driver
3. No dynamic memory	[PASS]	All buffers on stack
4. Short functions	[PASS]	Max function ~50 lines
5. Assertions	[PASS]	2+ checks per function
6. Data scope	[PASS]	All locals at point of use
7. Check returns	[PASS]	All I2C returns validated
8. Limit preprocessor	[PASS]	C23 typed enums only
9. Pointer restrictions	[WARN]	bus_manager function pointers (DIP)
10. Compile warnings	[PASS]	-Wall -Wextra -Werror

See also

rx_bno055.h Public API

rx_bno055_regs.h Register definitions

Author

STAR Team

Date

2026-03-04

Copyright

Definition in file rx_bno055.c.

4.5.6 Enumeration Type Documentation

4.5.6.1 bno055_assert_val_t

enum bno055_assert_val_t : uint8_t

Compile-time assertion reference values for internal_assemble_int16_le.

Named constants used in static_assert comparisons to avoid magic numbers. k_bno055_expected_msb_shift verifies that k_bno055_shift_msb == 8 (correct for little-endian byte assembly). k_bno055_expected_uint16_bits verifies that uint16_t is exactly 16 bits wide (required for the assembly logic).

Since

Version 1.0.0

Enumerator

k_bno055_expected_msb_shift	Expected value of k_bno055_shift_msb
k_bno055_expected_uint16_bits	Expected bit width of uint16_t

Definition at line 117 of file rx_bno055.c.

```
00117             : uint8_t {
00118   k_bno055_expected_msb_shift   = 8U,  /**< Expected value of k_bno055_shift_msb */
00119   k_bno055_expected_uint16_bits = 16U, /**< Expected bit width of uint16_t */
00120 } bno055_assert_val_t;
```

4.5.6.2 bno055_euler_size_t

enum bno055_euler_size_t : uint8_t

Expected byte count for an Euler angle burst read.

A BNO055 Euler reading consists of heading, roll, pitch each as a 16-bit little-endian value: 3 components x 2 bytes = 6 bytes total. Used in static_assert to verify the buffer constant is large enough.

Since

Version 1.0.0

Enumerator

k_bno055_euler_expected_bytes	Minimum bytes needed for Euler heading/roll/pitch (3 x 2 bytes)

Definition at line 338 of file rx_bno055.c.

```
00338             : uint8_t {
00339   k_bno055_euler_expected_bytes =
00340     6U, /**< Minimum bytes needed for Euler heading/roll/pitch (3 x 2 bytes) */
00341 } bno055_euler_size_t;
```

4.5.6.3 bno055_gyro_size_t

enum bno055_gyro_size_t : uint8_t

Expected byte count for a gyroscope burst read.

A BNO055 gyroscope reading consists of X, Y, Z each as a 16-bit little-endian value: 3 components x 2 bytes = 6 bytes total. Used in static_assert to verify the buffer constant is large enough.

Since

Version 1.0.0

Enumerator

k_bno055_gyro_expected_bytes	Minimum bytes needed for gyroscope X/Y/Z (3 x 2 bytes)

Definition at line 307 of file rx_bno055.c.

```
00307             : uint8_t {
00308   k_bno055_gyro_expected_bytes = 6U, /**< Minimum bytes needed for gyroscope X/Y/Z (3 x 2 bytes) */
00309 } bno055_gyro_size_t;
```

4.5.6.4 bno055_i2c_size_t

enum bno055_i2c_size_t : uint8_t

I2C write/read buffer size constants for register access.

The BNO055 register write protocol sends [register_address, data_byte] as a 2-byte I2C write transaction. Read commands send a 1-byte address.

Since

Version 1.0.0

Enumerator

k_bno055_write_buf_size	Size of [reg, val] write buffer
k_bno055_read_cmd_size	Size of register address read command

Definition at line 100 of file rx_bno055.c.

```
00100             : uint8_t {
00101   k_bno055_write_buf_size = 2, /**< Size of [reg, val] write buffer */
00102   k_bno055_read_cmd_size  = 1, /**< Size of register address read command */
00103 } bno055_i2c_size_t;
```

4.5.6.5 bno055_i2c_write_idx_t

enum bno055_i2c_write_idx_t : uint8_t

Byte indices into the 2-byte I2C write buffer.

The write buffer layout is [register_address, data_byte]. These indices name each position to prevent magic-number indexing into the buffer.

Since

Version 1.0.0

Enumerator

k_bno055_write_idx_reg	Index of register address in write buffer
k_bno055_write_idx_val	Index of register value in write buffer

Definition at line 132 of file rx_bno055.c.

```
00132             : uint8_t {
00133   k_bno055_write_idx_reg = 0, /**< Index of register address in write buffer */
00134   k_bno055_write_idx_val = 1, /**< Index of register value in write buffer */
00135 } bno055_i2c_write_idx_t;
```

4.5.6.6 bno055_lia_size_t

enum bno055_lia_size_t : uint8_t

Expected byte count for a linear acceleration burst read.

A BNO055 linear acceleration reading consists of X, Y, Z each as a 16-bit little-endian value: 3 components x 2 bytes = 6 bytes total. Used in static_assert to verify the buffer constant is large enough.

Since

Version 1.0.0

Enumerator

k_bno055_lia_expected_bytes	Minimum bytes needed for linear accel X/Y/Z (3 x 2 bytes)

Definition at line 354 of file rx_bno055.c.

```
00354             : uint8_t {
00355   k_bno055_lia_expected_bytes =
00356     6U, /**< Minimum bytes needed for linear accel X/Y/Z (3 x 2 bytes) */
00357 } bno055_lia_size_t;
```

4.5.6.7 bno055_quat_size_t

enum bno055_quat_size_t : uint8_t

Expected byte count for a quaternion burst read.

A BNO055 quaternion reading consists of W, X, Y, Z each as a 16-bit little-endian value: 4 components x 2 bytes = 8 bytes total. Used in static_assert to verify the buffer constant is large enough.

Since

Version 1.0.0

Enumerator

k_bno055_quat_expected_bytes	Minimum bytes needed for quaternion W/X/Y/Z (4 x 2 bytes)

Definition at line 322 of file rx_bno055.c.

```
00322             : uint8_t {
00323   k_bno055_quat_expected_bytes =
00324     8U, /**< Minimum bytes needed for quaternion W/X/Y/Z (4 x 2 bytes) */
00325 } bno055_quat_size_t;
```

4.5.6.8 bno055_single_byte_size_t

enum bno055_single_byte_size_t : uint8_t

Single byte read/write sizes.

Used for reading single registers such as temperature and calib_stat.

Since

Version 1.0.0

Enumerator

k_bno055_single_byte	Read/write size for single byte operations

Definition at line 146 of file rx_bno055.c.

```
00146             : uint8_t {
00147   k_bno055_single_byte = 1, /**< Read/write size for single byte operations */
00148 } bno055_single_byte_size_t;
```

4.5.6.9 euler_offsets_t

enum euler_offsets_t : uint8_t

Byte offsets in the 6-byte Euler angle burst-read buffer.

The BNO055 Euler output registers (0x1A-0x1F) are read as a 6-byte burst. Each 16-bit field occupies two consecutive bytes in little-endian order.

Since

Version 1.0.0

Enumerator

k_eul_h_lsb_off	Byte offset of heading LSB in euler_buf

k_eul_h_msb_off	Byte offset of heading MSB in euler_buf
k_eul_r_lsb_off	Byte offset of roll LSB in euler_buf
k_eul_r_msb_off	Byte offset of roll MSB in euler_buf
k_eul_p_lsb_off	Byte offset of pitch LSB in euler_buf
k_eul_p_msb_off	Byte offset of pitch MSB in euler_buf

Definition at line 228 of file rx_bno055.c.

```
00228             : uint8_t {
00229   k_eul_h_lsb_off = 0, /**< Byte offset of heading LSB in euler_buf */
00230   k_eul_h_msb_off = 1, /**< Byte offset of heading MSB in euler_buf */
00231   k_eul_r_lsb_off = 2, /**< Byte offset of roll LSB in euler_buf */
00232   k_eul_r_msb_off = 3, /**< Byte offset of roll MSB in euler_buf */
00233   k_eul_p_lsb_off = 4, /**< Byte offset of pitch LSB in euler_buf */
00234   k_eul_p_msb_off = 5, /**< Byte offset of pitch MSB in euler_buf */
00235 } euler_offsets_t;
```

4.5.6.10 gyro_offsets_t

enum gyro_offsets_t : uint8_t

Byte offsets within a gyroscope burst read buffer (6 bytes).

Maps byte positions to X/Y/Z LSB and MSB within the 6-byte buffer read from BNO055 registers GYR_DATA_X_LSB (0x14) through GYR_DATA_Z_MSB (0x19).

Since

Version 1.0.0

Enumerator

k_gyr_x_lsb_off	Byte offset of gyro X LSB in gyro_buf
k_gyr_x_msb_off	Byte offset of gyro X MSB in gyro_buf
k_gyr_y_lsb_off	Byte offset of gyro Y LSB in gyro_buf
k_gyr_y_msb_off	Byte offset of gyro Y MSB in gyro_buf
k_gyr_z_lsb_off	Byte offset of gyro Z LSB in gyro_buf
k_gyr_z_msb_off	Byte offset of gyro Z MSB in gyro_buf

Definition at line 287 of file rx_bno055.c.

```
00287             : uint8_t {
00288   k_gyr_x_lsb_off = 0, /**< Byte offset of gyro X LSB in gyro_buf */
00289   k_gyr_x_msb_off = 1, /**< Byte offset of gyro X MSB in gyro_buf */
00290   k_gyr_y_lsb_off = 2, /**< Byte offset of gyro Y LSB in gyro_buf */
00291   k_gyr_y_msb_off = 3, /**< Byte offset of gyro Y MSB in gyro_buf */
00292   k_gyr_z_lsb_off = 4, /**< Byte offset of gyro Z LSB in gyro_buf */
00293   k_gyr_z_msb_off = 5, /**< Byte offset of gyro Z MSB in gyro_buf */
00294 } gyro_offsets_t;
```

4.5.6.11 lia_offsets_t

enum lia_offsets_t : uint8_t

Byte offsets in the 6-byte linear acceleration burst-read buffer.

The BNO055 linear acceleration registers (0x28-0x2D) are read as a 6-byte burst. Each 16-bit field occupies two consecutive bytes in little-endian order.

Since

Version 1.0.0

Enumerator

k_lia_x_lsb_off	Byte offset of linear accel X LSB in lia_buf
k_lia_x_msb_off	Byte offset of linear accel X MSB in lia_buf
k_lia_y_lsb_off	Byte offset of linear accel Y LSB in lia_buf
k_lia_y_msb_off	Byte offset of linear accel Y MSB in lia_buf
k_lia_z_lsb_off	Byte offset of linear accel Z LSB in lia_buf
k_lia_z_msb_off	Byte offset of linear accel Z MSB in lia_buf

Definition at line 268 of file rx_bno055.c.

```
00268             : uint8_t {
00269   k_lia_x_lsb_off = 0, /**< Byte offset of linear accel X LSB in lia_buf */
00270   k_lia_x_msb_off = 1, /**< Byte offset of linear accel X MSB in lia_buf */
00271   k_lia_y_lsb_off = 2, /**< Byte offset of linear accel Y LSB in lia_buf */
00272   k_lia_y_msb_off = 3, /**< Byte offset of linear accel Y MSB in lia_buf */
00273   k_lia_z_lsb_off = 4, /**< Byte offset of linear accel Z LSB in lia_buf */
00274   k_lia_z_msb_off = 5, /**< Byte offset of linear accel Z MSB in lia_buf */
00275 } lia_offsets_t;
```

4.5.6.12 quat_offsets_t

enum quat_offsets_t : uint8_t

Byte offsets in the 8-byte quaternion burst-read buffer.

The BNO055 quaternion output registers (0x20-0x27) are read as an 8-byte burst. Each 16-bit field occupies two consecutive bytes in little-endian order.

Since

Version 1.0.0

Enumerator

k_qua_w_lsb_off	Byte offset of quaternion W LSB in quat_buf
k_qua_w_msb_off	Byte offset of quaternion W MSB in quat_buf
k_qua_x_lsb_off	Byte offset of quaternion X LSB in quat_buf
k_qua_x_msb_off	Byte offset of quaternion X MSB in quat_buf

k_qua_y_lsb_off	Byte offset of quaternion Y LSB in quat_buf
k_qua_y_msb_off	Byte offset of quaternion Y MSB in quat_buf
k_qua_z_lsb_off	Byte offset of quaternion Z LSB in quat_buf
k_qua_z_msb_off	Byte offset of quaternion Z MSB in quat_buf

Definition at line 247 of file rx_bno055.c.

```
00247             : uint8_t {
00248   k_qua_w_lsb_off = 0, /**< Byte offset of quaternion W LSB in quat_buf */
00249   k_qua_w_msb_off = 1, /**< Byte offset of quaternion W MSB in quat_buf */
00250   k_qua_x_lsb_off = 2, /**< Byte offset of quaternion X LSB in quat_buf */
00251   k_qua_x_msb_off = 3, /**< Byte offset of quaternion X MSB in quat_buf */
00252   k_qua_y_lsb_off = 4, /**< Byte offset of quaternion Y LSB in quat_buf */
00253   k_qua_y_msb_off = 5, /**< Byte offset of quaternion Y MSB in quat_buf */
00254   k_qua_z_lsb_off = 6, /**< Byte offset of quaternion Z LSB in quat_buf */
00255   k_qua_z_msb_off = 7, /**< Byte offset of quaternion Z MSB in quat_buf */
00256 } quat_offsets_t;
```

4.5.7 Function Documentation

4.5.7.1 internal_assemble_int16_le()

```
int16_t internal_assemble_int16_le (
                uint8_t low,
                uint8_t high)   [inline], [static]
```

Assemble a little-endian 16-bit signed integer from two bytes.

The BNO055 outputs all multi-byte values in little-endian format (LSB first). This helper combines a low byte and a high byte into a signed int16_t value using unsigned arithmetic to avoid implementation-defined behaviour.

Precondition

low and high are valid bytes read from BNO055 registers

uint16_t is exactly 16 bits (verified by static_assert below)

Postcondition

Return value has correct sign via two's complement reinterpretation

Two static_asserts below enforce compile-time invariants on shift and width

Note

Inline function; zero overhead after compiler optimization

See also

bno055_byte_idx_t LSB/MSB index constants

NASA Power of 10 Rule 5 Compliance:

This single-expression inline helper has no runtime state or I/O side effects. Both preconditions and postconditions are enforced by the two static_asserts inside the function body. Runtime assertion overhead on uint8_t parameters is not warranted; callers are responsible for providing valid register bytes.

Since

Version 1.0.0

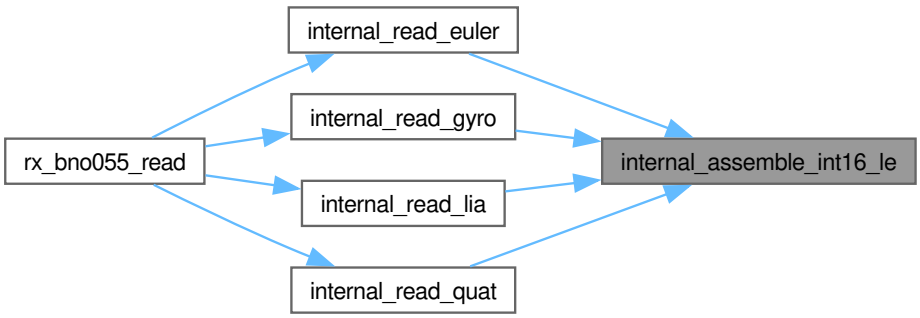
Definition at line 476 of file rx_bno055.c.

```
00477 {
00478   static_assert(
00479     (bool)(((unsigned)k_bno055_shift_msb == (unsigned)k_bno055_expected_msb_shift) != 0),
00480     "MSB shift must be 8 for little-endian assembly");
00481   static_assert(
00482     (bool)((sizeof(uint16_t) * CHAR_BIT == (unsigned)k_bno055_expected_uint16_bits) != 0),
00483     "uint16_t must be 16 bits for assembly to be well-defined");
00484   return (int16_t)((uint16_t)low | ((uint16_t)high « k_bno055_shift_msb));
00485 }
```

References k_bno055_expected_msb_shift, k_bno055_expected_uint16_bits, and k_bno055_shift_msb.

Referenced by internal_read_euler(), internal_read_gyro(), internal_read_lia(), and internal_read_quat().

Here is the caller graph for this function:



4.5.7.2 internal_init_configure()

```
RX_STATIC_TESTABLE rx_err_t internal_init_configure (
                void )
```

BNO055 init steps 3-6: configure power mode, units, axis map, clear trigger.

Writes power mode (Normal), unit selection (degrees/m/s^2/dps/Celsius), axis remapping (standard orientation), and clears the system trigger register. All writes are performed in CONFIG mode (entered by internal_init_reset_and_wait).

Precondition

s_manager non-NULL

BNO055 in CONFIG mode (internal_init_reset_and_wait succeeded)

Postcondition

Power mode set to Normal

Units set to degrees, m/s^2, dps, Celsius

Axis map set to standard orientation

System trigger cleared

Note

Not thread-safe; called only from rx_bno055_init()

See also

bno055_pwr_mode_t Power mode constants

bno055_axis_map_t Axis map constants

Since

Version 1.0.0

Definition at line 568 of file rx_bno055.c.

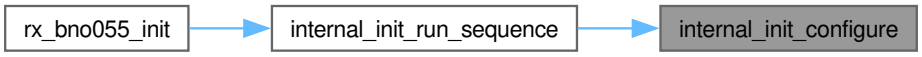
```
00569 {
00570   /* Pre-conditions: called only after rx_bno055_init sets a valid manager;
00571    * s_bus_name is a compile-time string constant (never NULL). */
00572   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for configure");
00573   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00574   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for configure");
00575   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00576
00577   /* Step 3: Set normal power mode */
00578   rx_err_t err = internal_write_reg(k_bno055_reg_pwr_mode, k_bno055_pwr_normal);
00579   if (err != k_rx_ok) {
00580     rx_log_error(s_tag, "Set power mode failed");
00581     return err;
00582   }
00583
00584   /* Step 4: Set default measurement units (degrees, m/s^2, dps, Celsius) */
00585   err = internal_write_reg(k_bno055_reg_unit_sel, k_bno055_unit_sel_default);
00586   if (err != k_rx_ok) {
00587     rx_log_error(s_tag, "Set unit sel failed");
00588     return err;
00589   }
00590
00591   /* Step 5: Set default axis remapping */
00592   err = internal_write_reg(k_bno055_reg_axis_map_cfg, k_bno055_axis_map_default);
00593   if (err != k_rx_ok) {
00594     rx_log_error(s_tag, "Set axis map cfg failed");
00595     return err;
00596   }
00597
00598   err = internal_write_reg(k_bno055_reg_axis_map_sgn, k_bno055_axis_map_sign_pos);
00599   if (err != k_rx_ok) {
00600     rx_log_error(s_tag, "Set axis map sign failed");
00601     return err;
00602   }
00603
00604   /* Step 6: Clear system trigger register */
00605   err = internal_write_reg(k_bno055_reg_sys_trigger, k_bno055_sys_trigger_clear);
00606   if (err != k_rx_ok) {
00607     rx_log_error(s_tag, "Clear sys trigger failed");
00608     return err;
00609   }
00610
00611   return k_rx_ok;
00612 }
```

References internal_write_reg(), k_bno055_axis_map_default, k_bno055_axis_map_sign_pos, k_bno055_pwr_normal, k_bno055_reg_axis_map_cfg, k_bno055_reg_axis_map_sgn, k_bno055_reg_pwr_mode, k_bno055_reg_sys_trigger, k_bno055_reg_unit_sel, k_bno055_sys_trigger_clear, k_bno055_unit_sel_default, s_bus_name, s_manager, and s_tag.

Referenced by internal_init_run_sequence().

Here is the call graph for this function:

Here is the caller graph for this function:



4.5.7.3 internal_init_enable_interrupt()

RX_STATIC_TESTABLE rx_err_t internal_init_enable_interrupt (
void)

Configure BNO055 Page 1 interrupt engine to assert INT on each data sample.

Writes to the BNO055 Page 1 interrupt registers to enable the accelerometer BSX data-ready interrupt (ACC_BSX_DRDY, INT_EN/INT_MSK bit 0). This causes the INT pin to assert on every new accelerometer sample (~100 Hz in NDOF mode), providing a hardware data-ready signal without any threshold configuration.

Register write sequence (BNO055 must be in CONFIG mode):

1. PAGE_ID = 1 (0x07) – switch to Page 1
2. INT_MSK = 0x01 (Page 1, 0x0F) – route ACC_BSX_DRDY to INT pin (bit 0)
3. INT_EN = 0x01 (Page 1, 0x10) – enable ACC_BSX_DRDY interrupt source (bit 0)
4. PAGE_ID = 0 (0x07) – restore Page 0 for normal sensor reads

Must be called while the sensor is in CONFIG mode (before internal_init_enter_ndof) so the interrupt registers accept writes per BNO055 datasheet section 4.3.

If any write fails, a best-effort restore of PAGE_ID=0 is attempted so the sensor is left in a known page before returning the error. On error, the caller (rx_bno055_init) will clear s_manager and return the error.

Precondition

s_manager non-NULL (set by rx_bno055_init before calling this helper)

BNO055 in CONFIG mode – call before internal_init_enter_ndof() per BNO055 datasheet

Postcondition

INT pin will assert on each BNO055 accelerometer data-ready event (on k_rx_ok)

PAGE_ID restored to 0 (attempted even on error)

Note

Not thread-safe; called only from rx_bno055_init()

See also

bno055_int_reg_t Page 1 register address constants

bno055_int_en_t INT_EN enable bit value (k_bno055_int_en_acc_bsx_drdy)

bno055_int_msk_t INT_MSK route bit value (k_bno055_int_msk_acc_bsx_drdy)

Since

Version 1.0.0

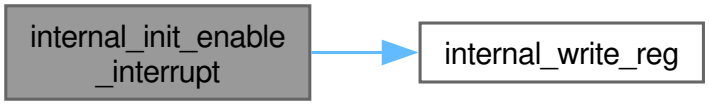
Definition at line 738 of file rx_bno055.c.

```
00739 {
00740   /* Pre-conditions: called only after rx_bno055_init sets a valid manager;
00741    * s_bus_name is a compile-time string constant (never NULL). */
00742   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for interrupt enable");
00743   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00744   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for interrupt enable");
00745   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00746
00747   /* Switch to Page 1 to access interrupt engine registers */
00748   rx_err_t err = internal_write_reg(k_bno055_reg_page_id, k_bno055_page1);
00749   if (err != k_rx_ok) {
00750     rx_log_error(s_tag, "INT init: page 1 switch failed");
00751     return err;
00752   }
00753
00754   /* Route ACC_BSX_DRDY interrupt source to INT pin (INT_MSK bit 0) */
00755   err = internal_write_reg(k_bno055_reg_int_msk, k_bno055_int_msk_acc_bsx_drdy);
00756   if (err != k_rx_ok) {
00757     rx_log_error(s_tag, "INT init: INT_MSK write failed");
00758     (void)internal_write_reg(k_bno055_reg_page_id, k_bno055_page0);
00759     return err;
00760   }
00761
00762   /* Enable accelerometer BSX data-ready as an interrupt source (INT_EN bit 0) */
00763   err = internal_write_reg(k_bno055_reg_int_en, k_bno055_int_en_acc_bsx_drdy);
00764   if (err != k_rx_ok) {
00765     rx_log_error(s_tag, "INT init: INT_EN write failed");
00766     (void)internal_write_reg(k_bno055_reg_page_id, k_bno055_page0);
00767     return err;
00768   }
00769
00770   /* Restore Page 0 for normal sensor output reads */
00771   err = internal_write_reg(k_bno055_reg_page_id, k_bno055_page0);
00772   if (err != k_rx_ok) {
00773     rx_log_error(s_tag, "INT init: page 0 restore failed");
00774     return err;
00775   }
00776
00777   return k_rx_ok;
00778 }
```

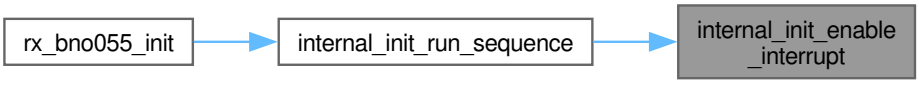
References internal_write_reg(), k_bno055_int_en_acc_bsx_drdy, k_bno055_int_msk_acc_bsx_drdy, k_bno055_page0, k_bno055_page1, k_bno055_reg_int_en, k_bno055_reg_int_msk, k_bno055_reg_page_id, s_bus_name, s_manager, and s_tag.

Referenced by internal_init_run_sequence().

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.7.4 internal_init_enter_ndof()

```
RX_STATIC_TESTABLE rx_err_t internal_init_enter_ndof (
                void )
```

BNO055 init step 7: enter NDOF fusion mode and wait for transition.

Writes OPR_MODE = NDOF (0x0C) to activate full 9-DOF sensor fusion (accelerometer + gyroscope + magnetometer). Waits 10 ms (1 tick) for the CONFIG -> NDOF transition to complete per BNO055 datasheet.

Precondition

s_manager non-NULL

BNO055 configured (internal_init_configure succeeded)

Postcondition

OPR_MODE register set to NDOF (0x0C)

~10 ms elapsed (1 tick) for mode transition to settle

Note

Not thread-safe; called only from rx_bno055_init()

Blocking: 1 tick @ 10 ms/tick = 10 ms (rounds up from 7 ms minimum)

See also

bno055_opr_mode_t Operating mode constants

bno055_delay_ticks_t Tick delay constants

Since

Version 1.0.0

Definition at line 636 of file rx_bno055.c.

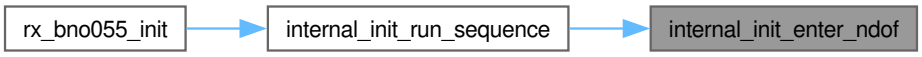
```
00637 {
00638   /* Pre-conditions: called only after rx_bno055_init sets a valid manager;
00639    * s_bus_name is a compile-time string constant (never NULL). */
00640   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for NDOF entry");
00641   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00642   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL");
00643   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00644
00645   /* Step 7: Enter NDOF fusion mode (full 9-DOF sensor fusion) */
00646   const rx_err_t err = internal_write_reg(k_bno055_reg_opr_mode, k_bno055_opr_ndof);
00647   if (err != k_rx_ok) {
00648     rx_log_error(s_tag, "Set NDOF mode failed");
00649     return err;
00650   }
00651   (void)tx_thread_sleep(
00652     (ULONG)k_bno055_delay_ndof_ticks); /* 1 tick @ 10 ms/tick = 10 ms (rounds up from 7 ms) */
00653
00654   return k_rx_ok;
00655 }
```

References internal_write_reg(), k_bno055_delay_ndof_ticks, k_bno055_opr_ndof, k_bno055_reg_opr_mode, s_bus_name, s_manager, and s_tag.

Referenced by internal_init_run_sequence().

Here is the call graph for this function:

Here is the caller graph for this function:



4.5.7.5 internal_init_reset_and_wait()

RX_STATIC_TESTABLE rx_err_t internal_init_reset_and_wait (
void)

BNO055 init steps 1-2: software reset, POR delay, CONFIG mode, config delay.

Writes SYS_TRIGGER reset command and waits 650 ms for POR sequence to complete. Then enters CONFIG mode and waits 20 ms for the transition. Must be called first in the init sequence before any register writes.

Precondition

s_manager non-NULL

"i2c1_imu" bus initialized

Postcondition

Sensor in CONFIG mode, ready for configuration register writes

~670 ms elapsed (650 ms POR + 20 ms CONFIG transition)

Note

Not thread-safe; called only from rx_bno055_init()

Blocking: 65 ticks @ 10 ms/tick = 650 ms POR, then 2 ticks = 20 ms CONFIG

See also

bno055_delay_ms_t Timing constants

bno055_sys_trigger_t Reset command values

Since

Version 1.0.0

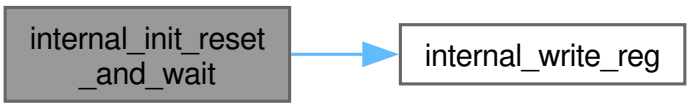
Definition at line 514 of file rx_bno055.c.

```
00515 {
00516   /* Pre-conditions: called only from rx_bno055_init after s_manager is set
00517    * and s_bus_name is a compile-time string constant (never NULL). */
00518   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for reset");
00519   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00520   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for reset sequence");
00521   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00522
00523   /* Step 1: Software reset, then wait for POR sequence */
00524   rx_err_t err = internal_write_reg(k_bno055_reg_sys_trigger, k_bno055_sys_trigger_rst);
00525   if (err != k_rx_ok) {
00526     rx_log_error(s_tag, "Software reset failed");
00527     return err;
00528   }
00529   (void)tx_thread_sleep(
00530     (ULONG)(k_bno055_delay_por_ms / k_bno055_ms_per_tick)); /* 65 ticks @ 10 ms/tick = 650 ms */
00531
00532   /* Step 2: Enter CONFIG mode (required for configuration register writes) */
00533   err = internal_write_reg(k_bno055_reg_opr_mode, k_bno055_opr_config);
00534   if (err != k_rx_ok) {
00535     rx_log_error(s_tag, "Set CONFIG mode failed");
00536     return err;
00537   }
00538   (void)tx_thread_sleep(
00539     (ULONG)k_bno055_delay_config_ms / (ULONG)k_bno055_ms_per_tick +
00540     (ULONG)k_bno055_tick_round_up); /* 2 ticks @ 10 ms/tick = 20 ms (round up for 19 ms) */
00541
00542   return k_rx_ok;
00543 }
```

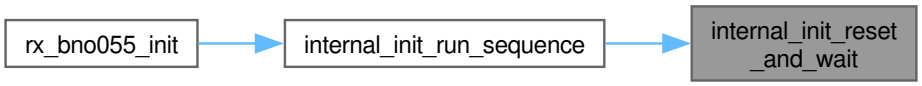
References internal_write_reg(), k_bno055_delay_config_ms, k_bno055_delay_por_ms, k_bno055_ms_per_tick, k_bno055_opr_config, k_bno055_reg_opr_mode, k_bno055_reg_sys_trigger, k_bno055_sys_trigger_rst, k_bno055_tick_round_up, s_bus_name, s_manager, and s_tag.

Referenced by internal_init_run_sequence().

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.7.6 internal_init_run_sequence()

```
rx_err_t internal_init_run_sequence (
                void ) [static]
```

Execute the multi-step BNO055 initialization sequence.

Runs reset, configure, optional interrupt setup, NDOF mode entry, and chip ID verification. Extracted from rx_bno055_init() to keep that function below the readability-function-size statement threshold.

Precondition

s_manager non-NULL (set by caller before invoking)

s_mode set to desired operating mode by caller

Postcondition

All BNO055 init registers written; NDOF mode active on k_rx_ok

Chip ID verified as 0xA0 on k_rx_ok

Note

Not thread-safe; called only from rx_bno055_init()

See also

internal_init_reset_and_wait() Steps 1-2

internal_init_configure() Steps 3-6

internal_init_enter_ndof() Step 7

internal_verify_chip_id() Step 8

internal_init_enable_interrupt() Interrupt engine (interrupt mode only)

Since

Version 1.0.0

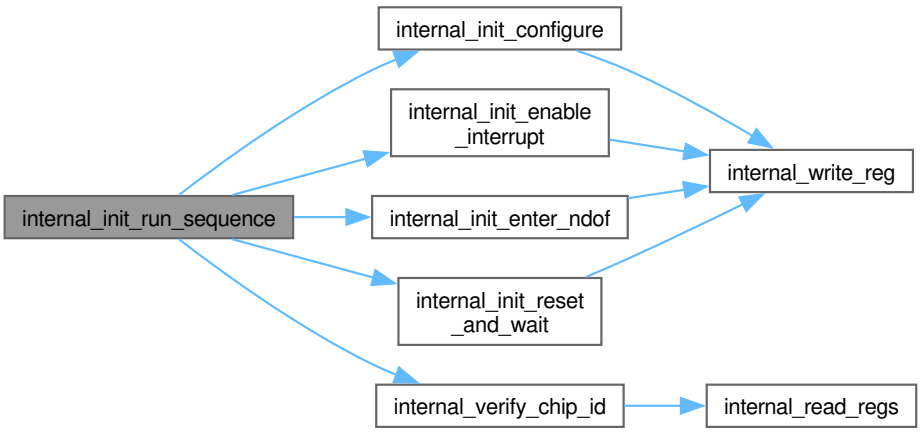
Definition at line 804 of file rx_bno055.c.

```
00805 {
00806   rx_err_t err = internal_init_reset_and_wait();
00807   if (err != k_rx_ok) {
00808     return err;
00809   }
00810
00811   err = internal_init_configure();
00812   if (err != k_rx_ok) {
00813     return err;
00814   }
00815
00816   /* Configure interrupt engine while still in CONFIG mode (BNO055 requires this) */
00817   if (s_mode == k_bno055_mode_interrupt) {
00818     err = internal_init_enable_interrupt();
00819     if (err != k_rx_ok) {
00820       return err;
00821     }
00822   }
00823
00824   err = internal_init_enter_ndof();
00825   if (err != k_rx_ok) {
00826     return err;
00827   }
00828
00829   return internal_verify_chip_id();
00830 }
```

References internal_init_configure(), internal_init_enable_interrupt(), internal_init_enter_ndof(), internal_init_reset_and_wait(), internal_verify_chip_id(), k_bno055_mode_interrupt, and s_mode.

Referenced by rx_bno055_init().

Here is the call graph for this function:



Here is the caller graph for this function:

4.5.7.7 internal_read_euler()

RX_STATIC_TESTABLE rx_err_t internal_read_euler (
bno055_data_t * out)

Read Euler angle registers from BNO055 and populate heading/roll/pitch.

Burst-reads 6 bytes from register 0x1A (EUL_H_LSB) and assembles three 16-bit little-endian values into out->heading_deg16, roll_deg16, pitch_deg16.

Precondition

s_initialized == true

out non-NULL

Postcondition

out->heading_deg16, roll_deg16, pitch_deg16 updated

Note

Called only from rx_bno055_read()

Since

Version 1.0.0

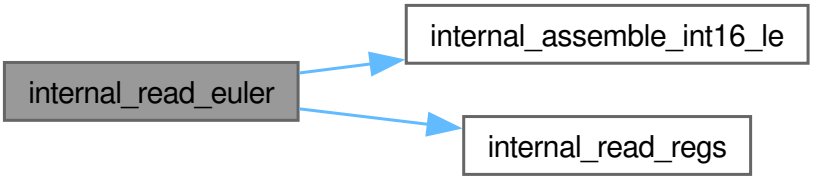
Definition at line 919 of file rx_bno055.c.

```
00920 {
00921   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
00922    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
00923   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading Euler angles");
00924   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
00925   static_assert(
00926     (bool)(((unsigned)k_bno055_euler_bytes >= (unsigned)k_bno055_euler_expected_bytes) != 0),
00927     "euler buffer must hold 6 bytes");
00928
00929   uint8_t  euler_buf[k_bno055_euler_bytes];
00930   rx_err_t err = internal_read_regs(k_bno055_reg_eul_h_lsb, euler_buf, k_bno055_euler_bytes);
00931   if (err != k_rx_ok) {
00932     rx_log_error(s_tag, "Euler read failed");
00933     return err;
00934   }
00935
00936   out->heading_deg16 =
00937     internal_assemble_int16_le(euler_buf[k_eul_h_lsb_off], euler_buf[k_eul_h_msb_off]);
00938   out->roll_deg16 =
00939     internal_assemble_int16_le(euler_buf[k_eul_r_lsb_off], euler_buf[k_eul_r_msb_off]);
00940   out->pitch_deg16 =
00941     internal_assemble_int16_le(euler_buf[k_eul_p_lsb_off], euler_buf[k_eul_p_msb_off]);
00942   return k_rx_ok;
00943 }
```

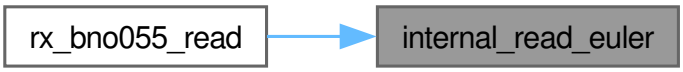
References bno055_data_t::heading_deg16, internal_assemble_int16_le(), internal_read_regs(), k_bno055_euler_bytes, k_bno055_euler_expected_bytes, k_bno055_reg_eul_h_lsb, k_eul_h_lsb_off, k_eul_h_msb_off, k_eul_p_lsb_off, k_eul_p_msb_off, k_eul_r_lsb_off, k_eul_r_msb_off, bno055_data_t::pitch_deg16, bno055_data_t::roll_deg16, s_initialized, and s_tag.

Referenced by rx_bno055_read().

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.7.8 internal_read_gyro()

```
RX_STATIC_TESTABLE rx_err_t internal_read_gyro (
                bno055_data_t * out)
```

Read BNO055 gyroscope data (raw, 16-bit little-endian).

Performs a 6-byte I2C burst read from GYR_DATA_X_LSB (0x14) through GYR_DATA_Z_MSB (0x19) and assembles three signed 16-bit values. Scale: 1 LSB = 1/16 deg/s (k_bno055_scale_gyro_lsb_per_dps = 16) when k_bno055_unit_sel_default is configured (dps mode, default).

Precondition

s_initialized must be true (call rx_bno055_init() first)

out must not be NULL

Postcondition

out->gyro_x_dps16, gyro_y_dps16, gyro_z_dps16 populated on k_rx_ok

Note

Not thread-safe; called only from rx_bno055_read()

See also

rx_bno055_read() Caller

k_bno055_scale_gyro_lsb_per_dps Scale factor (16 LSB per deg/s)

Since

Version 1.0.0

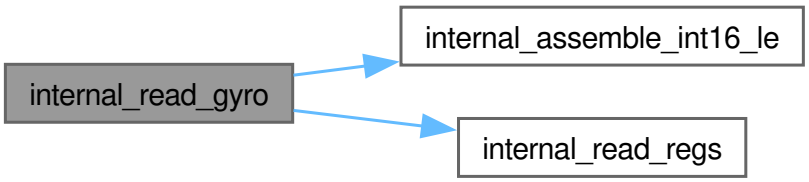
Definition at line 1048 of file rx_bno055.c.

```
01049 {
01050   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
01051    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
01052   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading gyro");
01053   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
01054   static_assert(
01055     (bool)(((unsigned)k_bno055_gyro_bytes >= (unsigned)k_bno055_gyro_expected_bytes) != 0),
01056     "gyro buffer must hold 6 bytes");
01057
01058   uint8_t  gyro_buf[k_bno055_gyro_bytes];
01059   rx_err_t err = internal_read_regs(k_bno055_reg_gyr_x_lsb, gyro_buf, k_bno055_gyro_bytes);
01060   if (err != k_rx_ok) {
01061     rx_log_error(s_tag, "Gyro read failed");
01062     return err;
01063   }
01064
01065   out->gyro_x_dps16 =
01066     internal_assemble_int16_le(gyro_buf[k_gyr_x_lsb_off], gyro_buf[k_gyr_x_msb_off]);
01067   out->gyro_y_dps16 =
01068     internal_assemble_int16_le(gyro_buf[k_gyr_y_lsb_off], gyro_buf[k_gyr_y_msb_off]);
01069   out->gyro_z_dps16 =
01070     internal_assemble_int16_le(gyro_buf[k_gyr_z_lsb_off], gyro_buf[k_gyr_z_msb_off]);
01071   return k_rx_ok;
01072 }
```

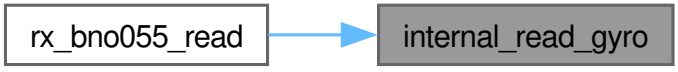
References bno055_data_t::gyro_x_dps16, bno055_data_t::gyro_y_dps16, bno055_data_t::gyro_z_dps16, internal_assemble_int16_le(), internal_read_regs(), k_bno055_gyro_bytes, k_bno055_gyro_expected_bytes, k_bno055_reg_gyr_x_lsb, k_gyr_x_lsb_off, k_gyr_x_msb_off, k_gyr_y_lsb_off, k_gyr_y_msb_off, k_gyr_z_lsb_off, k_gyr_z_msb_off, s_initialized, and s_tag.

Referenced by rx_bno055_read().

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.7.9 internal_read_lia()

```
RX_STATIC_TESTABLE rx_err_t internal_read_lia (
        bno055_data_t * out)
```

Read linear acceleration registers from BNO055 and populate lin_acc_x/y/z.

Burst-reads 6 bytes from register 0x28 (LIA_X_LSB) and assembles three 16-bit little-endian values into the lin_acc_x/y/z fields.

Precondition

s_initialized == true

out non-NULL

Postcondition

out->lin_acc_x/y/z updated

Note

Called only from rx_bno055_read()

Since

Version 1.0.0

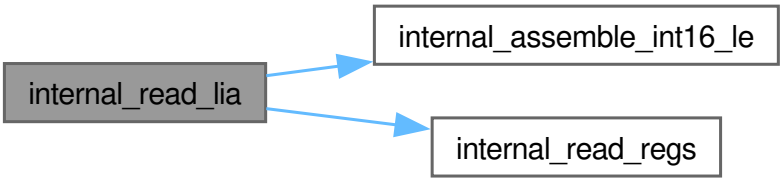
Definition at line 1003 of file rx_bno055.c.

```
01004 {
01005   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
01006    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
01007   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading linear acceleration");
01008   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
01009   static_assert(
01010     (bool)((unsigned)k_bno055_lia_bytes >= (unsigned)k_bno055_lia_expected_bytes) != 0),
01011     "lia buffer must hold 6 bytes");
01012
01013   uint8_t  lia_buf[k_bno055_lia_bytes];
01014   rx_err_t err = internal_read_regs(k_bno055_reg_lia_x_lsb, lia_buf, k_bno055_lia_bytes);
01015   if (err != k_rx_ok) {
01016     rx_log_error(s_tag, "Linear accel read failed");
01017     return err;
01018   }
01019
01020   out->lin_acc_x = internal_assemble_int16_le(lia_buf[k_lia_x_lsb_off], lia_buf[k_lia_x_msb_off]);
01021   out->lin_acc_y = internal_assemble_int16_le(lia_buf[k_lia_y_lsb_off], lia_buf[k_lia_y_msb_off]);
01022   out->lin_acc_z = internal_assemble_int16_le(lia_buf[k_lia_z_lsb_off], lia_buf[k_lia_z_msb_off]);
01023   return k_rx_ok;
01024 }
```

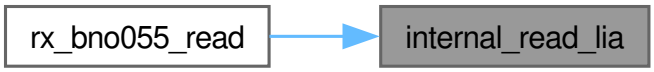
References internal_assemble_int16_le(), internal_read_regs(), k_bno055_lia_bytes, k_bno055_lia_expected_bytes, k_bno055_reg_lia_x_lsb, k_lia_x_lsb_off, k_lia_x_msb_off, k_lia_y_lsb_off, k_lia_y_msb_off, k_lia_z_lsb_off, k_lia_z_msb_off, bno055_data_t::lin_acc_x, bno055_data_t::lin_acc_y, bno055_data_t::lin_acc_z, s_initialized, and s_tag.

Referenced by rx_bno055_read().

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.7.10 internal_read_quat()

```
RX_STATIC_TESTABLE rx_err_t internal_read_quat (
            bno055_data_t * out)
```

Read quaternion registers from BNO055 and populate quat_w/x/y/z.

Burst-reads 8 bytes from register 0x20 (QUA_W_LSB) and assembles four 16-bit little-endian values into the quat_w/x/y/z fields.

Precondition

s_initialized == true

out non-NULL

Postcondition

out->quat_w/x/y/z updated

Note

Called only from rx_bno055_read()

Since

Version 1.0.0

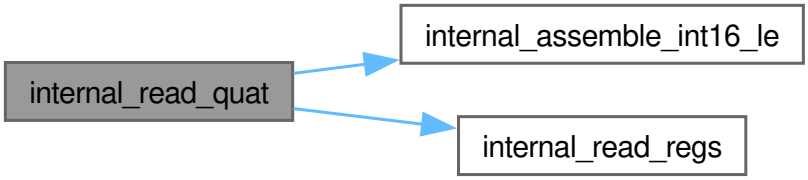
Definition at line 962 of file rx_bno055.c.

```
00963 {
00964   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
00965    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
00966   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading quaternion");
00967   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
00968   static_assert(
00969     (bool)(((unsigned)k_bno055_quat_bytes >= (unsigned)k_bno055_quat_expected_bytes) != 0),
00970     "quat buffer must hold 8 bytes");
00971
00972   uint8_t  quat_buf[k_bno055_quat_bytes];
00973   rx_err_t err = internal_read_regs(k_bno055_reg_qua_w_lsb, quat_buf, k_bno055_quat_bytes);
00974   if (err != k_rx_ok) {
00975     rx_log_error(s_tag, "Quaternion read failed");
00976     return err;
00977   }
00978
00979   out->quat_w = internal_assemble_int16_le(quat_buf[k_qua_w_lsb_off], quat_buf[k_qua_w_msb_off]);
00980   out->quat_x = internal_assemble_int16_le(quat_buf[k_qua_x_lsb_off], quat_buf[k_qua_x_msb_off]);
00981   out->quat_y = internal_assemble_int16_le(quat_buf[k_qua_y_lsb_off], quat_buf[k_qua_y_msb_off]);
00982   out->quat_z = internal_assemble_int16_le(quat_buf[k_qua_z_lsb_off], quat_buf[k_qua_z_msb_off]);
00983   return k_rx_ok;
00984 }
```

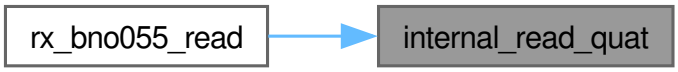
References internal_assemble_int16_le(), internal_read_regs(), k_bno055_quat_bytes, k_bno055_quat_expected_byt
k_bno055_reg_qua_w_lsb, k_qua_w_lsb_off, k_qua_w_msb_off, k_qua_x_lsb_off, k_qua_x_msb_off,
k_qua_y_lsb_off, k_qua_y_msb_off, k_qua_z_lsb_off, k_qua_z_msb_off, bno055_data_t::quat_w,
bno055_data_t::quat_x, bno055_data_t::quat_y, bno055_data_t::quat_z, s_initialized, and s_tag.

Referenced by rx_bno055_read().

Here is the call graph for this function:



Here is the caller graph for this function:



4.5.7.11 internal_read_regs()

```
RX_STATIC_TESTABLE rx_err_t internal_read_regs (
                uint8_t reg,
                uint8_t * buf,
                uint8_t len)
```

Burst-read consecutive registers from BNO055 via I2C write-read.

Issues an I2C combined write-read (repeated start) transaction:

1. Write the starting register address (1 byte)
2. Read len bytes of register data

The BNO055 auto-increments the register address for consecutive reads.

Precondition

s_manager must be non-NULL (set by rx_bno055_init)
"i2c1_imu" bus must be initialized
buf must have capacity for len bytes

Postcondition

buf[0..len-1] contain register data starting at reg (if k_rx_ok)
Bus transaction completes before function returns

Note

Not thread-safe; single-task access assumed

See also

rx_bus_i2c_write_read() Underlying I2C combined transaction

Since

Version 1.0.0

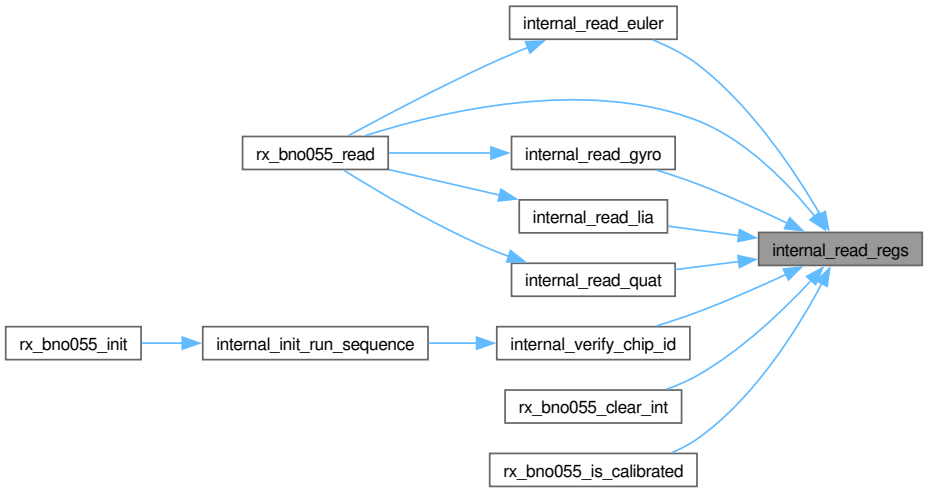
Definition at line 440 of file rx_bno055.c.

```
00441 {
00442   /* Pre-conditions: s_manager is non-NULL (only called after successful init);
00443    * buf is a local stack buffer (never NULL); len is an enum constant (always > 0). */
00444   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before read");
00445   RX_ASSERT_PRE(buf != NULL, "buf must be non-NULL for read operation");
00446   RX_ASSERT_PRE(len > 0U, "len must be positive for read operation");
00447   return rx_bus_i2c_write_read(s_manager, s_bus_name, &reg, k_bno055_read_cmd_size, buf, len);
00448 }
```

References k_bno055_read_cmd_size, s_bus_name, and s_manager.

Referenced by internal_read_euler(), internal_read_gyro(), internal_read_lia(), internal_read_quat(), internal_verify_chip_id(), rx_bno055_clear_int(), rx_bno055_is_calibrated(), and rx_bno055_read().

Here is the caller graph for this function:



4.5.7.12 internal_verify_chip_id()

```
RX_STATIC_TESTABLE rx_err_t internal_verify_chip_id (
                void )
```

BNO055 init step 8: read CHIP_ID register and verify it is 0xA0.

Reads the CHIP_ID register (0x00) and compares against k_bno055_chip_id_expected (0xA0). A mismatch indicates a wrong device, wiring fault, or I2C address collision. Returns k_rx_err_invalid_state if the chip ID does not match.

Precondition

s_manager non-NULL

BNO055 in NDOF mode (internal_init_enter_ndof succeeded)

Postcondition

CHIP_ID register confirmed == 0xA0 on k_rx_ok

Note

Not thread-safe; called only from rx_bno055_init()

See also

bno055_chip_id_t Expected chip ID constant

rx_bno055_init() Calls this as the final verification step

Since

Version 1.0.0

Definition at line 677 of file rx_bno055.c.

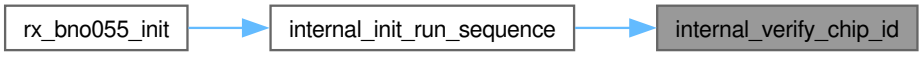
```
00678 {
00679   /* Pre-conditions: called only after rx_bno055_init sets s_manager;
00680    * s_bus_name is a compile-time string constant (never NULL). */
00681   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for chip ID verify");
00682   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00683   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for chip ID verification");
00684   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00685
00686   /* Step 8: Verify chip ID */
00687   uint8_t  chip_id = 0;
00688   rx_err_t err     = internal_read_regs(k_bno055_reg_chip_id, &chip_id, k_bno055_single_byte);
00689   if (err != k_rx_ok) {
00690     rx_log_error(s_tag, "Chip ID read failed");
00691     return err;
00692   }
00693
00694   if (chip_id != k_bno055_chip_id_expected) {
00695     rx_log_error_val(s_tag, "Unexpected chip ID", (uint32_t)chip_id);
00696     return k_rx_err_invalid_state;
00697   }
00698
00699   return k_rx_ok;
00700 }
```

References internal_read_regs(), k_bno055_chip_id_expected, k_bno055_reg_chip_id, k_bno055_single_byte, s_bus_name, s_manager, and s_tag.

Referenced by internal_init_run_sequence().

Here is the call graph for this function:

Here is the caller graph for this function:



4.5.7.13 internal_write_reg()

```
RX_STATIC_TESTABLE rx_err_t internal_write_reg (
                uint8_t reg,
                uint8_t val)
```

Write a single byte to a BNO055 register via I2C.

Sends a 2-byte I2C write transaction [reg, val] to the BNO055 at device address 0x28 (embedded in the "i2c1_imu" bus configuration).

Precondition

s_manager must be non-NULL (set by rx_bno055_init)

"i2c1_imu" bus must be initialized

Postcondition

Register at address reg contains value val (if k_rx_ok)

Bus transaction completes before function returns

Note

Not thread-safe; single-task access assumed

See also

rx_bus_i2c_write() Underlying I2C write function

Since

Version 1.0.0

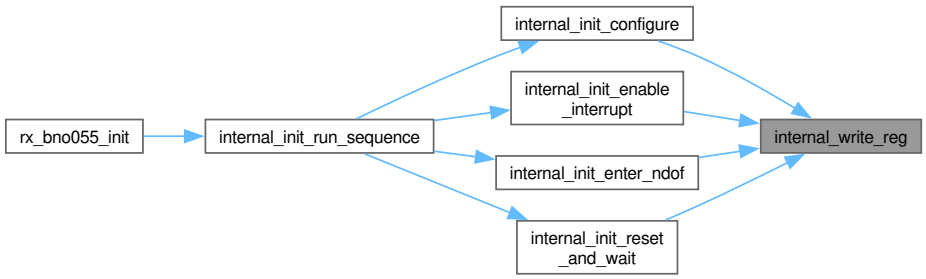
Definition at line 402 of file rx_bno055.c.

```
00403 {
00404   /* Pre-conditions: s_manager and s_bus_name are non-NULL by construction;
00405    * called only after rx_bno055_init stores a valid manager and s_bus_name
00406    * is a compile-time string constant. */
00407   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before write");
00408   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00409   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL before write");
00410   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00411   uint8_t buf[k_bno055_write_buf_size];
00412   buf[k_bno055_write_idx_reg] = reg;
00413   buf[k_bno055_write_idx_val] = val;
00414   return rx_bus_i2c_write(s_manager, s_bus_name, buf, k_bno055_write_buf_size);
00415 }
```

References k_bno055_write_buf_size, k_bno055_write_idx_reg, k_bno055_write_idx_val, s_bus_name, and s_manager.

Referenced by internal_init_configure(), internal_init_enable_interrupt(), internal_init_enter_ndof(), and internal_init_reset_and_wait().

Here is the caller graph for this function:



4.5.7.14 rx_bno055_clear_int()

```
rx_err_t rx_bno055_clear_int (
                void ) [nodiscard]
```

Clear the BNO055 INT pin by reading the INT_STA register (Page 0, 0x37).

Per BNO055 datasheet section 3.6, the INT output pin is asserted when an enabled-and-unmasked interrupt source fires and stays asserted until firmware reads the INT_STA register. Without this read, the pin latches high after the first interrupt and never produces another edge, so edge-triggered external IRQs on the host MCU fire exactly once.

Call this after every sensor read (or at any other convenient cadence) so the pin re-triggers on the next ACC_BSX_DRDY event.

Returns

rx_err_t Operation result

Return values

k_rx_ok	INT_STA read successfully (pin cleared)
k_rx_err_invalid_state	Driver not yet initialized
k_rx_err_timeout	/ k_rx_err_nack I2C failure

Precondition

rx_bno055_init() completed with k_rx_ok

Postcondition

BNO055 INT pin is low; pending-interrupt bits in INT_STA are cleared

Note

Safe to call with no pending interrupts – the read is idempotent.

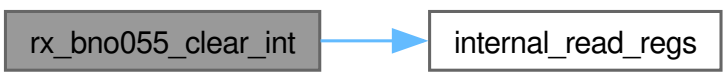
Not thread-safe.

Definition at line 1203 of file rx_bno055.c.

```
01204 {
01205   if (!s_initialized) {
01206     return k_rx_err_not_initialized;
01207   }
01208   uint8_t int_sta = 0U;
01209   return internal_read_regs(k_bno055_reg_int_sta, &int_sta, k_bno055_single_byte);
01210 }
```

References internal_read_regs(), k_bno055_reg_int_sta, k_bno055_single_byte, and s_initialized.

Here is the call graph for this function:



4.5.7.15 rx_bno055_init()

```
rx_err_t rx_bno055_init (
        rx_bus_manager_t * manager,
        const bno055_config_t * config)   [nodiscard]
```

Initialize BNO055 sensor and configure for NDOF fusion mode.

Validates parameters, stores module state, then delegates the complete 8-step initialization sequence (BNO055 datasheet section 3.3.1) to internal_init_run_sequence(). Any failure sets s_manager = NULL before returning so the module is re-initializable.

Delay rationale:

- 650 ms after reset: BNO055 internal boot sequence (ARM Cortex-M0 startup)
- 19 ms after config mode: Fusion -> CONFIG mode transition
- 7 ms after NDOF mode: CONFIG -> NDOF mode transition

ThreadX tick conversion: 1 tick = 10 ms at 100 Hz tick rate (k_bno055_ms_per_tick = 10).

- 650 ms POR: 650/10 = 65 ticks
- 19 ms config: 19/10 + 1 = 2 ticks (20 ms, rounded up for margin)
- 7 ms NDOF: 1 tick (10 ms, rounded up for margin)

Precondition

manager non-NULL with "i2c1_imu" bus registered and initialized
config non-NULL with a valid bno055_mode_t value in config->mode

Postcondition

s_initialized == true on success
Sensor running NDOF fusion
s_manager == NULL on any failure path
INT pin asserts at accelerometer data rate when config->mode == k_bno055_mode_interrupt

Note

Not thread-safe
Blocks ~700 ms total

See also

internal_init_run_sequence() Delegated initialization sequence
bno055_delay_ms_t Timing constants
bno055_config_t Configuration struct

Since

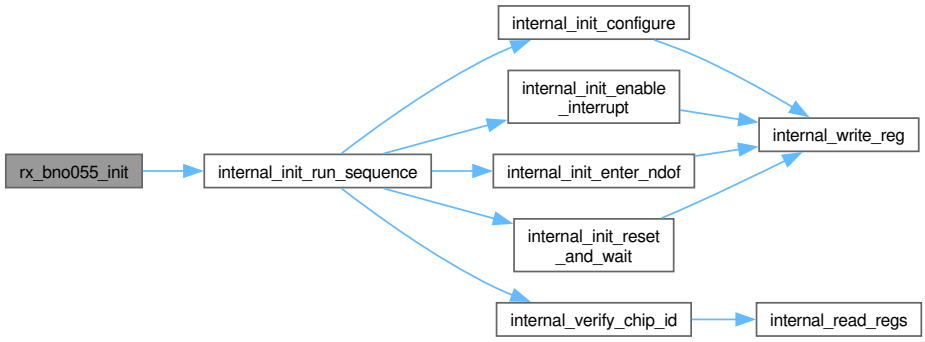
Version 1.0.0

Definition at line 874 of file rx_bno055.c.

```
00875 {
00876   static_assert((bool)((k_bno055_ms_per_tick != 0) != 0),
00877                 "k_bno055_ms_per_tick must be non-zero to avoid division by zero in tick delays");
00878   RX_CHECK_NULL_PTR(manager, s_tag, "Bus manager is NULL");
00879   RX_CHECK_NULL_PTR(config, s_tag, "Config is NULL");
00880
00881   if (s_initialized) {
00882     rx_log_error(s_tag, "BNO055 already initialized");
00883     return k_rx_err_invalid_state;
00884   }
00885
00886   s_manager     = manager;
00887   s_mode        = config->mode;
00888   s_initialized = false;
00889
00890   const rx_err_t err = internal_init_run_sequence();
00891   if (err != k_rx_ok) {
00892     s_manager = NULL;
00893     return err;
00894   }
00895
00896   s_initialized = true;
00897   rx_log_info(s_tag, "BNO055 initialized in NDOF mode");
00898
00899   return k_rx_ok;
00900 }
```

References internal_init_run_sequence(), k_bno055_ms_per_tick, bno055_config_t::mode, s_initialized, s_manager, s_mode, and s_tag.

Here is the call graph for this function:



4.5.7.16 rx_bno055_is_calibrated()

```
rx_err_t rx_bno055_is_calibrated (
                bool * out_calibrated)   [nodiscard]
```

Check whether all BNO055 subsystems are fully calibrated.

Reads CALIB_STAT register and checks all four 2-bit fields are at level 3 (fully calibrated).

Precondition

rx_bno055_init() succeeded

out_calibrated non-NULL

Postcondition

*out_calibrated valid only on k_rx_ok

CALIB_STAT register read (no state modified)

Note

Not thread-safe

See also

bno055_calib_shift_t Calibration shift constants

bno055_calib_mask_t Calibration mask and full-calibration sentinel

Since

Version 1.0.0

Definition at line 1173 of file rx_bno055.c.

```
01174 {
01175   RX_CHECK_NULL_PTR(out_calibrated, s_tag, "Output calibrated pointer is NULL");
01176
01177   if (!s_initialized) {
01178     return k_rx_err_not_initialized;
01179   }
01180
01181   uint8_t  calib_raw = 0;
01182   rx_err_t err = internal_read_regs(k_bno055_reg_calib_stat, &calib_raw, k_bno055_single_byte);
01183   if (err != k_rx_ok) {
01184     return err;
01185   }
01186
01187   const uint8_t mask = k_bno055_calib_level_mask;
01188   const uint8_t full = k_bno055_calib_full;
01189
01190   const uint8_t sys_cal = (calib_raw >> k_bno055_calib_sys_shift) & mask;
01191   const uint8_t gyr_cal = (calib_raw >> k_bno055_calib_gyr_shift) & mask;
01192   const uint8_t acc_cal = (calib_raw >> k_bno055_calib_acc_shift) & mask;
01193   const uint8_t mag_cal = (calib_raw >> k_bno055_calib_mag_shift) & mask;
01194
01195   /* Use bitwise & to evaluate all sub-expressions without short-circuit,
01196    * ensuring full branch coverage. */
01197   *out_calibrated =
01198     (bool)(((sys_cal == full) & (gyr_cal == full) & (acc_cal == full) & (mag_cal == full)) != 0);
01199
01200   return k_rx_ok;
01201 }
```

References internal_read_regs(), k_bno055_calib_acc_shift, k_bno055_calib_full, k_bno055_calib_gyr_shift, k_bno055_calib_level_mask, k_bno055_calib_mag_shift, k_bno055_calib_sys_shift, k_bno055_reg_calib_stat, k_bno055_single_byte, s_initialized, and s_tag.

Here is the call graph for this function:

4.5.7.17 rx_bno055_read()

```
rx_err_t rx_bno055_read (
            bno055_data_t * out)   [nodiscard]
```

Read current BNO055 fusion output data.

Performs six sequential I2C burst reads to collect all output data. All 16-bit values are assembled in little-endian format as specified in the BNO055 datasheet (LSB register first, MSB register second).

Register read sequence:

1. 0x14: 6 bytes -> Gyroscope X[2], Y[2], Z[2]
2. 0x1A: 6 bytes -> Euler heading[2], roll[2], pitch[2]
3. 0x20: 8 bytes -> Quaternion W[2], X[2], Y[2], Z[2]
4. 0x28: 6 bytes -> Linear accel X[2], Y[2], Z[2]
5. 0x34: 1 byte -> Temperature
6. 0x35: 1 byte -> Calibration status

Precondition

rx_bno055_init() succeeded
out non-NULL

Postcondition

All fields in *out populated on k_rx_ok
out->calib_stat reflects current calibration quality

Note

Not thread-safe

See also

bno055_scale_t Conversion factors for physical units

Since

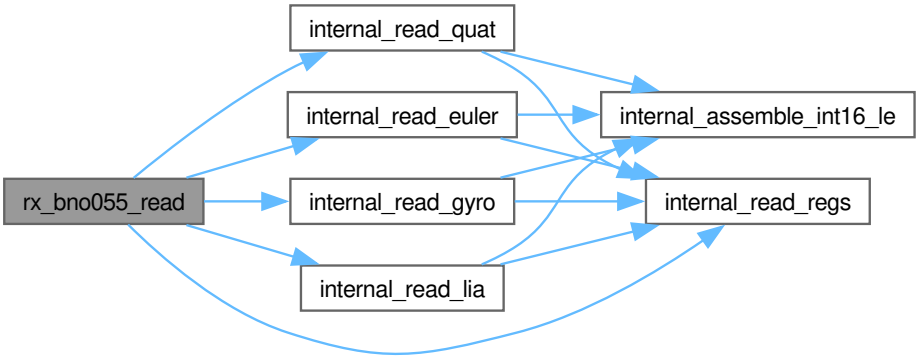
Version 1.0.0

Definition at line 1103 of file rx_bno055.c.

```
01104 {
01105   RX_CHECK_NULL_PTR(out, s_tag, "Output data pointer is NULL");
01106
01107   if (!s_initialized) {
01108     return k_rx_err_not_initialized;
01109   }
01110
01111   rx_err_t err = internal_read_gyro(out);
01112   if (err != k_rx_ok) {
01113     return err;
01114   }
01115
01116   err = internal_read_euler(out);
01117   if (err != k_rx_ok) {
01118     return err;
01119   }
01120
01121   err = internal_read_quat(out);
01122   if (err != k_rx_ok) {
01123     return err;
01124   }
01125
01126   err = internal_read_lia(out);
01127   if (err != k_rx_ok) {
01128     return err;
01129   }
01130
01131   /* Read Temperature: 1 byte from 0x34 */
01132   uint8_t temp_raw = 0;
01133   err              = internal_read_regs(k_bno055_reg_temp, &temp_raw, k_bno055_single_byte);
01134   if (err != k_rx_ok) {
01135     rx_log_error(s_tag, "Temperature read failed");
01136     return err;
01137   }
01138
01139   /* Read Calibration status: 1 byte from 0x35 */
01140   uint8_t calib_raw = 0;
01141   err               = internal_read_regs(k_bno055_reg_calib_stat, &calib_raw, k_bno055_single_byte);
01142   if (err != k_rx_ok) {
01143     rx_log_error(s_tag, "Calib status read failed");
01144     return err;
01145   }
01146
01147   out->temp_degc  = (int8_t)temp_raw;
01148   out->calib_stat = calib_raw;
01149
01150   return k_rx_ok;
01151 }
```

References bno055_data_t::calib_stat, internal_read_euler(), internal_read_gyro(), internal_read_lia(), internal_read_quat(), internal_read_regs(), k_bno055_reg_calib_stat, k_bno055_reg_temp, k_bno055_single_byte, s_initialized, s_tag, and bno055_data_t::temp_degc.

Here is the call graph for this function:



4.5.8 Variable Documentation

4.5.8.1 s_bus_name

const char* const s_bus_name = "i2c1_imu" [static]

I2C bus name used by the BNO055 driver to look up the bus manager.

The name must match the bus name registered in main.c via rx_bus_manager_register(). Passed to rx_bus_manager_get_i2c() to obtain the I2C bus handle.

Note

Must match the name registered in main.c (currently "i2c1_imu")

Warning

Do not modify; changing the name at runtime will cause bus lookup failures

Since

Version 1.0.0

Definition at line 211 of file rx_bno055.c.

Referenced by internal_init_configure(), internal_init_enable_interrupt(), internal_init_enter_ndof(), internal_init_reset_and_wait(), internal_read_regs(), internal_verify_chip_id(), and internal_write_reg().

4.5.8.2 s_initialized

bool s_initialized = false [static]

Guard flag: true only after successful rx_bno055_init().

Set to true at the end of rx_bno055_init() when all init steps succeed, including the chip ID verification. Remains false on any failure path. Checked at the top of rx_bno055_read() and rx_bno055_is_calibrated().

Invariant

s_initialized == true implies s_manager is non-NULL

Note

Only modified in rx_bno055_init()

Since

Version 1.0.0

Definition at line 186 of file rx_bno055.c.

Referenced by internal_read_euler(), internal_read_gyro(), internal_read_lia(), internal_read_quat(), rx_bno055_clear_int(), rx_bno055_init(), rx_bno055_is_calibrated(), and rx_bno055_read().

4.5.8.3 s_manager

rx_bus_manager_t* s_manager = NULL [static]

Bus manager pointer stored during rx_bno055_init(), used by all I2C operations.

Set to the caller-provided manager in rx_bno055_init(). Set back to NULL on all failure paths within rx_bno055_init() so a failed init leaves the module in a safe, re-initializable state.

Note

NULL before successful init and on any init failure path

Warning

Must not be accessed directly outside of this module

Since

Version 1.0.0

Definition at line 174 of file rx_bno055.c.

Referenced by internal_init_configure(), internal_init_enable_interrupt(), internal_init_enter_ndof(), internal_init_reset_and_wait(), internal_read_regs(), internal_verify_chip_id(), internal_write_reg(), and rx_bno055_init().

4.5.8.4 s_mode

bno055_mode_t s_mode = k_bno055_mode_poll [static]

Operating mode stored during rx_bno055_init(), used by diagnostic queries.

Set to config->mode during rx_bno055_init(). Reset to k_bno055_mode_poll by rx_bno055_test_reset_state() (UNIT_TEST builds only).

Note

Valid only after successful rx_bno055_init()

Since

Version 1.0.0

Definition at line 196 of file rx_bno055.c.

Referenced by internal_init_run_sequence(), and rx_bno055_init().

4.5.8.5 s_tag

const char* const s_tag = "BNO055" [static]

Log tag for this module.

Used as prefix in all rx_log_* calls to identify the BNO055 driver.

Note

Constant; never modified after initialization

Since

Version 1.0.0

Definition at line 162 of file rx_bno055.c.

Referenced by internal_init_configure(), internal_init_enable_interrupt(), internal_init_enter_ndof(), internal_init_reset_and_wait(), internal_read_euler(), internal_read_gyro(), internal_read_lia(), internal_read_quat(), internal_verify_chip_id(), rx_bno055_init(), rx_bno055_is_calibrated(), and rx_bno055_read().

4.6 rx_bno055.c

Go to the documentation of this file.

```
00001 /**
00002  * @file rx_bno055.c
00003  * @brief BNO055 9-DOF Absolute Orientation Sensor Driver Implementation
00004  *
00005  * @details
00006  * # Overview
00007  *
00008  * Complete driver implementation for the Bosch BNO055 sensor connected
00009  * via I2C on RIIC1 (SCL1=P2.1, SDA1=P2.0). The sensor is initialized in
00010  * NDOF (Nine Degrees of Freedom) sensor fusion mode which provides:
00011  * - Absolute Euler angles (heading, roll, pitch)
00012  * - Unit quaternion for 3D rotation
00013  * - Gravity-free linear acceleration
00014  * - On-chip temperature
00015  * - Per-subsystem calibration status
00016  *
00017  * # Module Architecture
00018  *
00019  * The driver uses two internal helper functions for all I2C communication:
00020  * - internal_write_reg(): Writes a single byte to a register
00021  * - internal_read_regs(): Burst reads N consecutive registers
00022  *
00023  * Both helpers use the bus manager abstraction with bus name "i2c1_imu".
00024  * The device address 0x28 is embedded in the "i2c1_imu" bus configuration
00025  * registered by main.c.
00026  *
00027  * # Data Flow
00028  *
00029  * @code{.unparsed}
00030  * imu_task -----> rx_bno055_read()
00031  *                     |
00032  *                internal_read_regs()
00033  *                     |
00034  *                rx_bus_i2c_write_read()
00035  *                     |
00036  *                RIIC1 HAL
00037  *                     |
00038  *                BNO055 sensor
00039  * @endcode
00040  *
00041  * # NASA Power of 10 Compliance
```

```
00042  *
00043  * | Rule | Status | Notes |
00044  * |------|--------|-------|
00045  * | 1. No goto | [PASS] | Structured if/while only |
00046  * | 2. Bounded loops | [PASS] | No loops in this driver |
00047  * | 3. No dynamic memory | [PASS] | All buffers on stack |
00048  * | 4. Short functions | [PASS] | Max function ~50 lines |
00049  * | 5. Assertions | [PASS] | 2+ checks per function |
00050  * | 6. Data scope | [PASS] | All locals at point of use |
00051  * | 7. Check returns | [PASS] | All I2C returns validated |
00052  * | 8. Limit preprocessor | [PASS] | C23 typed enums only |
00053  * | 9. Pointer restrictions | [WARN] | bus_manager function pointers (DIP) |
00054  * | 10. Compile warnings | [PASS] | -Wall -Wextra -Werror |
00055  *
00056  * @see rx_bno055.h Public API
00057  * @see rx_bno055_regs.h Register definitions
00058  *
00059  * @author STAR Team
00060  * @date 2026-03-04
00061  * @copyright Copyright (c) 2026 Locked Inc.
00062  * SPDX-License-Identifier: MIT
00063  */
00064
00065 #include "rx_bno055.h"
00066
00067 #include <limits.h>
00068 #include <stddef.h>
00069
00070 #include "rx_bno055_regs.h"
00071 #include "rx_bus_i2c.h"
00072 #include "rx_check.h"
00073 #include "rx_log.h"
00074 #include "tx_api.h"
00075
00076 /* Validate k_bno055_ms_per_tick is consistent with the actual ThreadX tick rate.
00077  * If TX_TIMER_TICKS_PER_SECOND changes, this assert catches the mismatch at compile time. */
00078 static_assert(
00079   (bool)(((k_bno055_ms_per_second % TX_TIMER_TICKS_PER_SECOND) == 0U) != 0),
00080   "TX_TIMER_TICKS_PER_SECOND must evenly divide k_bno055_ms_per_second (no truncation)");
00081 static_assert((bool)((k_bno055_ms_per_tick ==
00082                       (k_bno055_ms_per_second / TX_TIMER_TICKS_PER_SECOND)) != 0),
00083               "k_bno055_ms_per_tick must equal k_bno055_ms_per_second/TX_TIMER_TICKS_PER_SECOND");
00084
00085 /*
      ================================================================================
00086  * Constants
00087  *
      ================================================================================
00088  */
00089
00090 /**
00091  * @enum bno055_i2c_size_t
00092  * @brief I2C write/read buffer size constants for register access
00093  *
00094  * @details
00095  * The BNO055 register write protocol sends [register_address, data_byte]
00096  * as a 2-byte I2C write transaction. Read commands send a 1-byte address.
00097  *
00098  * @since Version 1.0.0
00099  */
00100 typedef enum : uint8_t {
00101   k_bno055_write_buf_size = 2, /**< Size of [reg, val] write buffer */
00102   k_bno055_read_cmd_size  = 1, /**< Size of register address read command */
00103 } bno055_i2c_size_t;
00104
00105 /**
00106  * @enum bno055_assert_val_t
00107  * @brief Compile-time assertion reference values for internal_assemble_int16_le
00108  *
00109  * @details
00110  * Named constants used in static_assert comparisons to avoid magic numbers.
00111  * k_bno055_expected_msb_shift verifies that k_bno055_shift_msb == 8 (correct
00112  * for little-endian byte assembly). k_bno055_expected_uint16_bits verifies
00113  * that uint16_t is exactly 16 bits wide (required for the assembly logic).
00114  *
00115  * @since Version 1.0.0
00116  */
00117 typedef enum : uint8_t {
00118   k_bno055_expected_msb_shift   = 8U,  /**< Expected value of k_bno055_shift_msb */
00119   k_bno055_expected_uint16_bits = 16U, /**< Expected bit width of uint16_t */
00120 } bno055_assert_val_t;
00121
00122 /**
00123  * @enum bno055_i2c_write_idx_t
00124  * @brief Byte indices into the 2-byte I2C write buffer
00125  *
00126  * @details
```

```
00127  * The write buffer layout is [register_address, data_byte]. These indices
00128  * name each position to prevent magic-number indexing into the buffer.
00129  *
00130  * @since Version 1.0.0
00131  */
00132 typedef enum : uint8_t {
00133   k_bno055_write_idx_reg = 0, /**< Index of register address in write buffer */
00134   k_bno055_write_idx_val = 1, /**< Index of register value in write buffer */
00135 } bno055_i2c_write_idx_t;
00136
00137 /**
00138  * @enum bno055_single_byte_size_t
00139  * @brief Single byte read/write sizes
00140  *
00141  * @details
00142  * Used for reading single registers such as temperature and calib_stat.
00143  *
00144  * @since Version 1.0.0
00145  */
00146 typedef enum : uint8_t {
00147   k_bno055_single_byte = 1, /**< Read/write size for single byte operations */
00148 } bno055_single_byte_size_t;
00149
00150 /*
      ============================================================================
00151  * Module-Static State
00152  *
      ============================================================================
00153  */
00154
00155 /**
00156  * @var s_tag
00157  * @brief Log tag for this module
00158  * @details Used as prefix in all rx_log_* calls to identify the BNO055 driver.
00159  * @note Constant; never modified after initialization
00160  * @since Version 1.0.0
00161  */
00162 static const char* const s_tag = "BNO055";
00163
00164 /**
00165  * @var s_manager
00166  * @brief Bus manager pointer stored during rx_bno055_init(), used by all I2C operations
00167  * @details Set to the caller-provided manager in rx_bno055_init(). Set back to NULL
00168  *          on all failure paths within rx_bno055_init() so a failed init leaves the
00169  *          module in a safe, re-initializable state.
00170  * @note NULL before successful init and on any init failure path
00171  * @warning Must not be accessed directly outside of this module
00172  * @since Version 1.0.0
00173  */
00174 static rx_bus_manager_t* s_manager = NULL;
00175
00176 /**
00177  * @var s_initialized
00178  * @brief Guard flag: true only after successful rx_bno055_init()
00179  * @details Set to true at the end of rx_bno055_init() when all init steps succeed,
00180  *          including the chip ID verification. Remains false on any failure path.
00181  *          Checked at the top of rx_bno055_read() and rx_bno055_is_calibrated().
00182  * @invariant s_initialized == true implies s_manager is non-NULL
00183  * @note Only modified in rx_bno055_init()
00184  * @since Version 1.0.0
00185  */
00186 static bool s_initialized = false;
00187
00188 /**
00189  * @var s_mode
00190  * @brief Operating mode stored during rx_bno055_init(), used by diagnostic queries
00191  * @details Set to config->mode during rx_bno055_init(). Reset to k_bno055_mode_poll
00192  *          by rx_bno055_test_reset_state() (UNIT_TEST builds only).
00193  * @note Valid only after successful rx_bno055_init()
00194  * @since Version 1.0.0
00195  */
00196 static bno055_mode_t s_mode = k_bno055_mode_poll;
00197
00198 /**
00199  * @var s_bus_name
00200  * @brief I2C bus name used by the BNO055 driver to look up the bus manager
00201  *
00202  * @details
00203  * The name must match the bus name registered in main.c via rx_bus_manager_register().
00204  * Passed to rx_bus_manager_get_i2c() to obtain the I2C bus handle.
00205  *
00206  * @note Must match the name registered in main.c (currently "i2c1_imu")
00207  * @warning Do not modify; changing the name at runtime will cause bus lookup failures
00208  *
00209  * @since Version 1.0.0
00210  */
00211 static const char* const s_bus_name = "i2c1_imu";
```

```
00212
00213 /*
      ==========================================================================
00214  * Module-level Offset Constants
00215  *
      ==========================================================================
00216  */
00217
00218 /**
00219  * @enum euler_offsets_t
00220  * @brief Byte offsets in the 6-byte Euler angle burst-read buffer
00221  *
00222  * @details
00223  * The BNO055 Euler output registers (0x1A-0x1F) are read as a 6-byte burst.
00224  * Each 16-bit field occupies two consecutive bytes in little-endian order.
00225  *
00226  * @since Version 1.0.0
00227  */
00228 typedef enum : uint8_t {
00229   k_eul_h_lsb_off = 0, /**< Byte offset of heading LSB in euler_buf */
00230   k_eul_h_msb_off = 1, /**< Byte offset of heading MSB in euler_buf */
00231   k_eul_r_lsb_off = 2, /**< Byte offset of roll LSB in euler_buf */
00232   k_eul_r_msb_off = 3, /**< Byte offset of roll MSB in euler_buf */
00233   k_eul_p_lsb_off = 4, /**< Byte offset of pitch LSB in euler_buf */
00234   k_eul_p_msb_off = 5, /**< Byte offset of pitch MSB in euler_buf */
00235 } euler_offsets_t;
00236
00237 /**
00238  * @enum quat_offsets_t
00239  * @brief Byte offsets in the 8-byte quaternion burst-read buffer
00240  *
00241  * @details
00242  * The BNO055 quaternion output registers (0x20-0x27) are read as an 8-byte burst.
00243  * Each 16-bit field occupies two consecutive bytes in little-endian order.
00244  *
00245  * @since Version 1.0.0
00246  */
00247 typedef enum : uint8_t {
00248   k_qua_w_lsb_off = 0, /**< Byte offset of quaternion W LSB in quat_buf */
00249   k_qua_w_msb_off = 1, /**< Byte offset of quaternion W MSB in quat_buf */
00250   k_qua_x_lsb_off = 2, /**< Byte offset of quaternion X LSB in quat_buf */
00251   k_qua_x_msb_off = 3, /**< Byte offset of quaternion X MSB in quat_buf */
00252   k_qua_y_lsb_off = 4, /**< Byte offset of quaternion Y LSB in quat_buf */
00253   k_qua_y_msb_off = 5, /**< Byte offset of quaternion Y MSB in quat_buf */
00254   k_qua_z_lsb_off = 6, /**< Byte offset of quaternion Z LSB in quat_buf */
00255   k_qua_z_msb_off = 7, /**< Byte offset of quaternion Z MSB in quat_buf */
00256 } quat_offsets_t;
00257
00258 /**
00259  * @enum lia_offsets_t
00260  * @brief Byte offsets in the 6-byte linear acceleration burst-read buffer
00261  *
00262  * @details
00263  * The BNO055 linear acceleration registers (0x28-0x2D) are read as a 6-byte burst.
00264  * Each 16-bit field occupies two consecutive bytes in little-endian order.
00265  *
00266  * @since Version 1.0.0
00267  */
00268 typedef enum : uint8_t {
00269   k_lia_x_lsb_off = 0, /**< Byte offset of linear accel X LSB in lia_buf */
00270   k_lia_x_msb_off = 1, /**< Byte offset of linear accel X MSB in lia_buf */
00271   k_lia_y_lsb_off = 2, /**< Byte offset of linear accel Y LSB in lia_buf */
00272   k_lia_y_msb_off = 3, /**< Byte offset of linear accel Y MSB in lia_buf */
00273   k_lia_z_lsb_off = 4, /**< Byte offset of linear accel Z LSB in lia_buf */
00274   k_lia_z_msb_off = 5, /**< Byte offset of linear accel Z MSB in lia_buf */
00275 } lia_offsets_t;
00276
00277 /**
00278  * @enum gyro_offsets_t
00279  * @brief Byte offsets within a gyroscope burst read buffer (6 bytes)
00280  *
00281  * @details
00282  * Maps byte positions to X/Y/Z LSB and MSB within the 6-byte buffer
00283  * read from BNO055 registers GYR_DATA_X_LSB (0x14) through GYR_DATA_Z_MSB (0x19).
00284  *
00285  * @since Version 1.0.0
00286  */
00287 typedef enum : uint8_t {
00288   k_gyr_x_lsb_off = 0, /**< Byte offset of gyro X LSB in gyro_buf */
00289   k_gyr_x_msb_off = 1, /**< Byte offset of gyro X MSB in gyro_buf */
00290   k_gyr_y_lsb_off = 2, /**< Byte offset of gyro Y LSB in gyro_buf */
00291   k_gyr_y_msb_off = 3, /**< Byte offset of gyro Y MSB in gyro_buf */
00292   k_gyr_z_lsb_off = 4, /**< Byte offset of gyro Z LSB in gyro_buf */
00293   k_gyr_z_msb_off = 5, /**< Byte offset of gyro Z MSB in gyro_buf */
00294 } gyro_offsets_t;
00295
00296 /**
```

```
00297  * @enum bno055_gyro_size_t
00298  * @brief Expected byte count for a gyroscope burst read
00299  *
00300  * @details
00301  * A BNO055 gyroscope reading consists of X, Y, Z each as a 16-bit
00302  * little-endian value: 3 components x 2 bytes = 6 bytes total.
00303  * Used in static_assert to verify the buffer constant is large enough.
00304  *
00305  * @since Version 1.0.0
00306  */
00307 typedef enum : uint8_t {
00308   k_bno055_gyro_expected_bytes = 6U, /**< Minimum bytes needed for gyroscope X/Y/Z (3 x 2 bytes) */
00309 } bno055_gyro_size_t;
00310
00311 /**
00312  * @enum bno055_quat_size_t
00313  * @brief Expected byte count for a quaternion burst read
00314  *
00315  * @details
00316  * A BNO055 quaternion reading consists of W, X, Y, Z each as a 16-bit
00317  * little-endian value: 4 components x 2 bytes = 8 bytes total.
00318  * Used in static_assert to verify the buffer constant is large enough.
00319  *
00320  * @since Version 1.0.0
00321  */
00322 typedef enum : uint8_t {
00323   k_bno055_quat_expected_bytes =
00324     8U, /**< Minimum bytes needed for quaternion W/X/Y/Z (4 x 2 bytes) */
00325 } bno055_quat_size_t;
00326
00327 /**
00328  * @enum bno055_euler_size_t
00329  * @brief Expected byte count for an Euler angle burst read
00330  *
00331  * @details
00332  * A BNO055 Euler reading consists of heading, roll, pitch each as a 16-bit
00333  * little-endian value: 3 components x 2 bytes = 6 bytes total.
00334  * Used in static_assert to verify the buffer constant is large enough.
00335  *
00336  * @since Version 1.0.0
00337  */
00338 typedef enum : uint8_t {
00339   k_bno055_euler_expected_bytes =
00340     6U, /**< Minimum bytes needed for Euler heading/roll/pitch (3 x 2 bytes) */
00341 } bno055_euler_size_t;
00342
00343 /**
00344  * @enum bno055_lia_size_t
00345  * @brief Expected byte count for a linear acceleration burst read
00346  *
00347  * @details
00348  * A BNO055 linear acceleration reading consists of X, Y, Z each as a 16-bit
00349  * little-endian value: 3 components x 2 bytes = 6 bytes total.
00350  * Used in static_assert to verify the buffer constant is large enough.
00351  *
00352  * @since Version 1.0.0
00353  */
00354 typedef enum : uint8_t {
00355   k_bno055_lia_expected_bytes =
00356     6U, /**< Minimum bytes needed for linear accel X/Y/Z (3 x 2 bytes) */
00357 } bno055_lia_size_t;
00358
00359 /*
      ============================================================================
00360  * Forward Declarations
00361  *
      ============================================================================
00362  */
00363
00364 RX_STATIC_TESTABLE rx_err_t internal_write_reg(uint8_t reg, uint8_t val);
00365 RX_STATIC_TESTABLE rx_err_t internal_read_regs(uint8_t reg, uint8_t* buf, uint8_t len);
00366 static inline int16_t      internal_assemble_int16_le(uint8_t low, uint8_t high);
00367 RX_STATIC_TESTABLE rx_err_t internal_read_euler(bno055_data_t* out);
00368 RX_STATIC_TESTABLE rx_err_t internal_read_quat(bno055_data_t* out);
00369 RX_STATIC_TESTABLE rx_err_t internal_read_lia(bno055_data_t* out);
00370 RX_STATIC_TESTABLE rx_err_t internal_read_gyro(bno055_data_t* out);
00371 RX_STATIC_TESTABLE rx_err_t internal_init_reset_and_wait(void);
00372 RX_STATIC_TESTABLE rx_err_t internal_init_configure(void);
00373 RX_STATIC_TESTABLE rx_err_t internal_init_enter_ndof(void);
00374 RX_STATIC_TESTABLE rx_err_t internal_verify_chip_id(void);
00375 RX_STATIC_TESTABLE rx_err_t internal_init_enable_interrupt(void);
00376 static rx_err_t            internal_init_run_sequence(void);
00377
00378 /*
      ============================================================================
00379  * Internal Helpers
00380  *
```

```
    ========================================================================
00381  */
00382
00383 /**
00384  * @brief Write a single byte to a BNO055 register via I2C
00385  *
00386  * @details
00387  * Sends a 2-byte I2C write transaction [reg, val] to the BNO055 at
00388  * device address 0x28 (embedded in the "i2c1_imu" bus configuration).
00389  *
00390  *
00391  *
00392  * @pre s_manager must be non-NULL (set by rx_bno055_init)
00393  * @pre "i2c1_imu" bus must be initialized
00394  * @post Register at address reg contains value val (if k_rx_ok)
00395  * @post Bus transaction completes before function returns
00396  *
00397  * @note Not thread-safe; single-task access assumed
00398  * @see rx_bus_i2c_write() Underlying I2C write function
00399  *
00400  * @since Version 1.0.0
00401  */
00402 RX_STATIC_TESTABLE rx_err_t internal_write_reg(uint8_t reg, uint8_t val)
00403 {
00404   /* Pre-conditions: s_manager and s_bus_name are non-NULL by construction;
00405    * called only after rx_bno055_init stores a valid manager and s_bus_name
00406    * is a compile-time string constant. */
00407   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before write");
00408   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00409   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL before write");
00410   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00411   uint8_t buf[k_bno055_write_buf_size];
00412   buf[k_bno055_write_idx_reg] = reg;
00413   buf[k_bno055_write_idx_val] = val;
00414   return rx_bus_i2c_write(s_manager, s_bus_name, buf, k_bno055_write_buf_size);
00415 }
00416
00417 /**
00418  * @brief Burst-read consecutive registers from BNO055 via I2C write-read
00419  *
00420  * @details
00421  * Issues an I2C combined write-read (repeated start) transaction:
00422  * 1. Write the starting register address (1 byte)
00423  * 2. Read len bytes of register data
00424  *
00425  * The BNO055 auto-increments the register address for consecutive reads.
00426  *
00427  *
00428  *
00429  * @pre s_manager must be non-NULL (set by rx_bno055_init)
00430  * @pre "i2c1_imu" bus must be initialized
00431  * @pre buf must have capacity for len bytes
00432  * @post buf[0..len-1] contain register data starting at reg (if k_rx_ok)
00433  * @post Bus transaction completes before function returns
00434  *
00435  * @note Not thread-safe; single-task access assumed
00436  * @see rx_bus_i2c_write_read() Underlying I2C combined transaction
00437  *
00438  * @since Version 1.0.0
00439  */
00440 RX_STATIC_TESTABLE rx_err_t internal_read_regs(uint8_t reg, uint8_t* buf, uint8_t len)
00441 {
00442   /* Pre-conditions: s_manager is non-NULL (only called after successful init);
00443    * buf is a local stack buffer (never NULL); len is an enum constant (always > 0). */
00444   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL before read");
00445   RX_ASSERT_PRE(buf != NULL, "buf must be non-NULL for read operation");
00446   RX_ASSERT_PRE(len > 0U, "len must be positive for read operation");
00447   return rx_bus_i2c_write_read(s_manager, s_bus_name, &reg, k_bno055_read_cmd_size, buf, len);
00448 }
00449
00450 /**
00451  * @brief Assemble a little-endian 16-bit signed integer from two bytes
00452  *
00453  * @details
00454  * The BNO055 outputs all multi-byte values in little-endian format (LSB first).
00455  * This helper combines a low byte and a high byte into a signed int16_t value
00456  * using unsigned arithmetic to avoid implementation-defined behaviour.
00457  *
00458  *
00459  *
00460  * @pre low and high are valid bytes read from BNO055 registers
00461  * @pre uint16_t is exactly 16 bits (verified by static_assert below)
00462  * @post Return value has correct sign via two's complement reinterpretation
00463  * @post Two static_asserts below enforce compile-time invariants on shift and width
00464  *
00465  * @note Inline function; zero overhead after compiler optimization
00466  * @see bno055_byte_idx_t LSB/MSB index constants
```

```
00467  *
00468  * @par NASA Power of 10 Rule 5 Compliance:
00469  * This single-expression inline helper has no runtime state or I/O side effects.
00470  * Both preconditions and postconditions are enforced by the two static_asserts
00471  * inside the function body. Runtime assertion overhead on uint8_t parameters
00472  * is not warranted; callers are responsible for providing valid register bytes.
00473  *
00474  * @since Version 1.0.0
00475  */
00476 static inline int16_t internal_assemble_int16_le(uint8_t low, uint8_t high)
00477 {
00478   static_assert(
00479     (bool)(((unsigned)k_bno055_shift_msb == (unsigned)k_bno055_expected_msb_shift) != 0),
00480     "MSB shift must be 8 for little-endian assembly");
00481   static_assert(
00482     (bool)((sizeof(uint16_t) * CHAR_BIT == (unsigned)k_bno055_expected_uint16_bits) != 0),
00483     "uint16_t must be 16 bits for assembly to be well-defined");
00484   return (int16_t)((uint16_t)low | ((uint16_t)high << k_bno055_shift_msb));
00485 }
00486
00487 /*
     ============================================================================
00488  * Init Helpers
00489  *
     ============================================================================
00490  */
00491
00492 /**
00493  * @brief BNO055 init steps 1-2: software reset, POR delay, CONFIG mode, config delay
00494  *
00495  * @details
00496  * Writes SYS_TRIGGER reset command and waits 650 ms for POR sequence to
00497  * complete. Then enters CONFIG mode and waits 20 ms for the transition.
00498  * Must be called first in the init sequence before any register writes.
00499  *
00500  *
00501  * @pre s_manager non-NULL
00502  * @pre "i2c1_imu" bus initialized
00503  * @post Sensor in CONFIG mode, ready for configuration register writes
00504  * @post ~670 ms elapsed (650 ms POR + 20 ms CONFIG transition)
00505  *
00506  * @note Not thread-safe; called only from rx_bno055_init()
00507  * @note Blocking: 65 ticks @ 10 ms/tick = 650 ms POR, then 2 ticks = 20 ms CONFIG
00508  *
00509  * @see bno055_delay_ms_t Timing constants
00510  * @see bno055_sys_trigger_t Reset command values
00511  *
00512  * @since Version 1.0.0
00513  */
00514 RX_STATIC_TESTABLE rx_err_t internal_init_reset_and_wait(void)
00515 {
00516   /* Pre-conditions: called only from rx_bno055_init after s_manager is set
00517    * and s_bus_name is a compile-time string constant (never NULL). */
00518   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for reset");
00519   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00520   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for reset sequence");
00521   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00522
00523   /* Step 1: Software reset, then wait for POR sequence */
00524   rx_err_t err = internal_write_reg(k_bno055_reg_sys_trigger, k_bno055_sys_trigger_rst);
00525   if (err != k_rx_ok) {
00526     rx_log_error(s_tag, "Software reset failed");
00527     return err;
00528   }
00529   (void)tx_thread_sleep(
00530     (ULONG)(k_bno055_delay_por_ms / k_bno055_ms_per_tick)); /* 65 ticks @ 10 ms/tick = 650 ms */
00531
00532   /* Step 2: Enter CONFIG mode (required for configuration register writes) */
00533   err = internal_write_reg(k_bno055_reg_opr_mode, k_bno055_opr_config);
00534   if (err != k_rx_ok) {
00535     rx_log_error(s_tag, "Set CONFIG mode failed");
00536     return err;
00537   }
00538   (void)tx_thread_sleep(
00539     (ULONG)k_bno055_delay_config_ms / (ULONG)k_bno055_ms_per_tick +
00540     (ULONG)k_bno055_tick_round_up); /* 2 ticks @ 10 ms/tick = 20 ms (round up for 19 ms) */
00541
00542   return k_rx_ok;
00543 }
00544
00545 /**
00546  * @brief BNO055 init steps 3-6: configure power mode, units, axis map, clear trigger
00547  *
00548  * @details
00549  * Writes power mode (Normal), unit selection (degrees/m/s^2/dps/Celsius),
00550  * axis remapping (standard orientation), and clears the system trigger register.
00551  * All writes are performed in CONFIG mode (entered by internal_init_reset_and_wait).
```

```
00552  *
00553  *
00554  * @pre s_manager non-NULL
00555  * @pre BNO055 in CONFIG mode (internal_init_reset_and_wait succeeded)
00556  * @post Power mode set to Normal
00557  * @post Units set to degrees, m/s^2, dps, Celsius
00558  * @post Axis map set to standard orientation
00559  * @post System trigger cleared
00560  *
00561  * @note Not thread-safe; called only from rx_bno055_init()
00562  *
00563  * @see bno055_pwr_mode_t Power mode constants
00564  * @see bno055_axis_map_t Axis map constants
00565  *
00566  * @since Version 1.0.0
00567  */
00568 RX_STATIC_TESTABLE rx_err_t internal_init_configure(void)
00569 {
00570   /* Pre-conditions: called only after rx_bno055_init sets a valid manager;
00571    * s_bus_name is a compile-time string constant (never NULL). */
00572   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for configure");
00573   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00574   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for configure");
00575   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00576
00577   /* Step 3: Set normal power mode */
00578   rx_err_t err = internal_write_reg(k_bno055_reg_pwr_mode, k_bno055_pwr_normal);
00579   if (err != k_rx_ok) {
00580     rx_log_error(s_tag, "Set power mode failed");
00581     return err;
00582   }
00583
00584   /* Step 4: Set default measurement units (degrees, m/s^2, dps, Celsius) */
00585   err = internal_write_reg(k_bno055_reg_unit_sel, k_bno055_unit_sel_default);
00586   if (err != k_rx_ok) {
00587     rx_log_error(s_tag, "Set unit sel failed");
00588     return err;
00589   }
00590
00591   /* Step 5: Set default axis remapping */
00592   err = internal_write_reg(k_bno055_reg_axis_map_cfg, k_bno055_axis_map_default);
00593   if (err != k_rx_ok) {
00594     rx_log_error(s_tag, "Set axis map cfg failed");
00595     return err;
00596   }
00597
00598   err = internal_write_reg(k_bno055_reg_axis_map_sgn, k_bno055_axis_map_sign_pos);
00599   if (err != k_rx_ok) {
00600     rx_log_error(s_tag, "Set axis map sign failed");
00601     return err;
00602   }
00603
00604   /* Step 6: Clear system trigger register */
00605   err = internal_write_reg(k_bno055_reg_sys_trigger, k_bno055_sys_trigger_clear);
00606   if (err != k_rx_ok) {
00607     rx_log_error(s_tag, "Clear sys trigger failed");
00608     return err;
00609   }
00610
00611   return k_rx_ok;
00612 }
00613
00614 /**
00615  * @brief BNO055 init step 7: enter NDOF fusion mode and wait for transition
00616  *
00617  * @details
00618  * Writes OPR_MODE = NDOF (0x0C) to activate full 9-DOF sensor fusion
00619  * (accelerometer + gyroscope + magnetometer). Waits 10 ms (1 tick) for
00620  * the CONFIG -> NDOF transition to complete per BNO055 datasheet.
00621  *
00622  *
00623  * @pre s_manager non-NULL
00624  * @pre BNO055 configured (internal_init_configure succeeded)
00625  * @post OPR_MODE register set to NDOF (0x0C)
00626  * @post ~10 ms elapsed (1 tick) for mode transition to settle
00627  *
00628  * @note Not thread-safe; called only from rx_bno055_init()
00629  * @note Blocking: 1 tick @ 10 ms/tick = 10 ms (rounds up from 7 ms minimum)
00630  *
00631  * @see bno055_opr_mode_t Operating mode constants
00632  * @see bno055_delay_ticks_t Tick delay constants
00633  *
00634  * @since Version 1.0.0
00635  */
00636 RX_STATIC_TESTABLE rx_err_t internal_init_enter_ndof(void)
00637 {
00638   /* Pre-conditions: called only after rx_bno055_init sets a valid manager;
```

```
00639   * s_bus_name is a compile-time string constant (never NULL). */
00640  RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for NDOF entry");
00641  /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00642  RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL");
00643  /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00644
00645  /* Step 7: Enter NDOF fusion mode (full 9-DOF sensor fusion) */
00646  const rx_err_t err = internal_write_reg(k_bno055_reg_opr_mode, k_bno055_opr_ndof);
00647  if (err != k_rx_ok) {
00648    rx_log_error(s_tag, "Set NDOF mode failed");
00649    return err;
00650  }
00651  (void)tx_thread_sleep(
00652    (ULONG)k_bno055_delay_ndof_ticks); /* 1 tick @ 10 ms/tick = 10 ms (rounds up from 7 ms) */
00653
00654  return k_rx_ok;
00655 }
00656
00657 /**
00658  * @brief BNO055 init step 8: read CHIP_ID register and verify it is 0xA0
00659  *
00660  * @details
00661  * Reads the CHIP_ID register (0x00) and compares against k_bno055_chip_id_expected (0xA0).
00662  * A mismatch indicates a wrong device, wiring fault, or I2C address collision.
00663  * Returns k_rx_err_invalid_state if the chip ID does not match.
00664  *
00665  *
00666  * @pre s_manager non-NULL
00667  * @pre BNO055 in NDOF mode (internal_init_enter_ndof succeeded)
00668  * @post CHIP_ID register confirmed == 0xA0 on k_rx_ok
00669  *
00670  * @note Not thread-safe; called only from rx_bno055_init()
00671  *
00672  * @see bno055_chip_id_t Expected chip ID constant
00673  * @see rx_bno055_init() Calls this as the final verification step
00674  *
00675  * @since Version 1.0.0
00676  */
00677 RX_STATIC_TESTABLE rx_err_t internal_verify_chip_id(void)
00678 {
00679  /* Pre-conditions: called only after rx_bno055_init sets s_manager;
00680   * s_bus_name is a compile-time string constant (never NULL). */
00681  RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for chip ID verify");
00682  /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00683  RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for chip ID verification");
00684  /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00685
00686  /* Step 8: Verify chip ID */
00687  uint8_t  chip_id = 0;
00688  rx_err_t err     = internal_read_regs(k_bno055_reg_chip_id, &chip_id, k_bno055_single_byte);
00689  if (err != k_rx_ok) {
00690    rx_log_error(s_tag, "Chip ID read failed");
00691    return err;
00692  }
00693
00694  if (chip_id != k_bno055_chip_id_expected) {
00695    rx_log_error_val(s_tag, "Unexpected chip ID", (uint32_t)chip_id);
00696    return k_rx_err_invalid_state;
00697  }
00698
00699  return k_rx_ok;
00700 }
00701
00702 /**
00703  * @brief Configure BNO055 Page 1 interrupt engine to assert INT on each data sample
00704  *
00705  * @details
00706  * Writes to the BNO055 Page 1 interrupt registers to enable the accelerometer
00707  * BSX data-ready interrupt (ACC_BSX_DRDY, INT_EN/INT_MSK bit 0). This causes
00708  * the INT pin to assert on every new accelerometer sample (~100 Hz in NDOF mode),
00709  * providing a hardware data-ready signal without any threshold configuration.
00710  *
00711  * Register write sequence (BNO055 must be in CONFIG mode):
00712  * 1. PAGE_ID = 1 (0x07) -- switch to Page 1
00713  * 2. INT_MSK = 0x01 (Page 1, 0x0F) -- route ACC_BSX_DRDY to INT pin (bit 0)
00714  * 3. INT_EN  = 0x01 (Page 1, 0x10) -- enable ACC_BSX_DRDY interrupt source (bit 0)
00715  * 4. PAGE_ID = 0 (0x07) -- restore Page 0 for normal sensor reads
00716  *
00717  * Must be called while the sensor is in CONFIG mode (before internal_init_enter_ndof)
00718  * so the interrupt registers accept writes per BNO055 datasheet section 4.3.
00719  *
00720  * If any write fails, a best-effort restore of PAGE_ID=0 is attempted so the
00721  * sensor is left in a known page before returning the error. On error, the caller
00722  * (rx_bno055_init) will clear s_manager and return the error.
00723  *
00724  *
00725  * @pre s_manager non-NULL (set by rx_bno055_init before calling this helper)
```

```
00726  * @pre BNO055 in CONFIG mode -- call before internal_init_enter_ndof() per BNO055 datasheet
00727  * @post INT pin will assert on each BNO055 accelerometer data-ready event (on k_rx_ok)
00728  * @post PAGE_ID restored to 0 (attempted even on error)
00729  *
00730  * @note Not thread-safe; called only from rx_bno055_init()
00731  *
00732  * @see bno055_int_reg_t Page 1 register address constants
00733  * @see bno055_int_en_t INT_EN enable bit value (k_bno055_int_en_acc_bsx_drdy)
00734  * @see bno055_int_msk_t INT_MSK route bit value (k_bno055_int_msk_acc_bsx_drdy)
00735  *
00736  * @since Version 1.0.0
00737  */
00738 RX_STATIC_TESTABLE rx_err_t internal_init_enable_interrupt(void)
00739 {
00740   /* Pre-conditions: called only after rx_bno055_init sets a valid manager;
00741    * s_bus_name is a compile-time string constant (never NULL). */
00742   RX_ASSERT_PRE(s_manager != NULL, "s_manager must be non-NULL for interrupt enable");
00743   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(s_bus_name is a compile-time constant) */
00744   RX_ASSERT_PRE(s_bus_name != NULL, "s_bus_name must be non-NULL for interrupt enable");
00745   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
00746
00747   /* Switch to Page 1 to access interrupt engine registers */
00748   rx_err_t err = internal_write_reg(k_bno055_reg_page_id, k_bno055_page1);
00749   if (err != k_rx_ok) {
00750     rx_log_error(s_tag, "INT init: page 1 switch failed");
00751     return err;
00752   }
00753
00754   /* Route ACC_BSX_DRDY interrupt source to INT pin (INT_MSK bit 0) */
00755   err = internal_write_reg(k_bno055_reg_int_msk, k_bno055_int_msk_acc_bsx_drdy);
00756   if (err != k_rx_ok) {
00757     rx_log_error(s_tag, "INT init: INT_MSK write failed");
00758     (void)internal_write_reg(k_bno055_reg_page_id, k_bno055_page0);
00759     return err;
00760   }
00761
00762   /* Enable accelerometer BSX data-ready as an interrupt source (INT_EN bit 0) */
00763   err = internal_write_reg(k_bno055_reg_int_en, k_bno055_int_en_acc_bsx_drdy);
00764   if (err != k_rx_ok) {
00765     rx_log_error(s_tag, "INT init: INT_EN write failed");
00766     (void)internal_write_reg(k_bno055_reg_page_id, k_bno055_page0);
00767     return err;
00768   }
00769
00770   /* Restore Page 0 for normal sensor output reads */
00771   err = internal_write_reg(k_bno055_reg_page_id, k_bno055_page0);
00772   if (err != k_rx_ok) {
00773     rx_log_error(s_tag, "INT init: page 0 restore failed");
00774     return err;
00775   }
00776
00777   return k_rx_ok;
00778 }
00779
00780 /**
00781  * @brief Execute the multi-step BNO055 initialization sequence
00782  *
00783  * @details
00784  * Runs reset, configure, optional interrupt setup, NDOF mode entry, and chip
00785  * ID verification. Extracted from rx_bno055_init() to keep that function below
00786  * the readability-function-size statement threshold.
00787  *
00788  *
00789  * @pre s_manager non-NULL (set by caller before invoking)
00790  * @pre s_mode set to desired operating mode by caller
00791  * @post All BNO055 init registers written; NDOF mode active on k_rx_ok
00792  * @post Chip ID verified as 0xA0 on k_rx_ok
00793  *
00794  * @note Not thread-safe; called only from rx_bno055_init()
00795  *
00796  * @see internal_init_reset_and_wait() Steps 1-2
00797  * @see internal_init_configure() Steps 3-6
00798  * @see internal_init_enter_ndof() Step 7
00799  * @see internal_verify_chip_id() Step 8
00800  * @see internal_init_enable_interrupt() Interrupt engine (interrupt mode only)
00801  *
00802  * @since Version 1.0.0
00803  */
00804 static rx_err_t internal_init_run_sequence(void)
00805 {
00806   rx_err_t err = internal_init_reset_and_wait();
00807   if (err != k_rx_ok) {
00808     return err;
00809   }
00810
00811   err = internal_init_configure();
00812   if (err != k_rx_ok) {
```

```
00813     return err;
00814   }
00815
00816   /* Configure interrupt engine while still in CONFIG mode (BNO055 requires this) */
00817   if (s_mode == k_bno055_mode_interrupt) {
00818     err = internal_init_enable_interrupt();
00819     if (err != k_rx_ok) {
00820       return err;
00821     }
00822   }
00823
00824   err = internal_init_enter_ndof();
00825   if (err != k_rx_ok) {
00826     return err;
00827   }
00828
00829   return internal_verify_chip_id();
00830 }
00831
00832 /*
      ============================================================================
00833  * Public API Implementation
00834  *
      ============================================================================
00835  */
00836
00837 /**
00838  * @brief Initialize BNO055 sensor and configure for NDOF fusion mode
00839  *
00840  * @details
00841  * Validates parameters, stores module state, then delegates the complete
00842  * 8-step initialization sequence (BNO055 datasheet section 3.3.1) to
00843  * internal_init_run_sequence(). Any failure sets s_manager = NULL before
00844  * returning so the module is re-initializable.
00845  *
00846  * Delay rationale:
00847  * - 650 ms after reset: BNO055 internal boot sequence (ARM Cortex-M0 startup)
00848  * - 19 ms after config mode: Fusion -> CONFIG mode transition
00849  * - 7 ms after NDOF mode: CONFIG -> NDOF mode transition
00850  *
00851  * ThreadX tick conversion: 1 tick = 10 ms at 100 Hz tick rate (k_bno055_ms_per_tick = 10).
00852  * - 650 ms POR: 650/10 = 65 ticks
00853  * - 19 ms config: 19/10 + 1 = 2 ticks (20 ms, rounded up for margin)
00854  * - 7 ms NDOF: 1 tick (10 ms, rounded up for margin)
00855  *
00856  *
00857  *
00858  * @pre manager non-NULL with "i2c1_imu" bus registered and initialized
00859  * @pre config non-NULL with a valid bno055_mode_t value in config->mode
00860  * @post s_initialized == true on success
00861  * @post Sensor running NDOF fusion
00862  * @post s_manager == NULL on any failure path
00863  * @post INT pin asserts at accelerometer data rate when config->mode == k_bno055_mode_interrupt
00864  *
00865  * @note Not thread-safe
00866  * @note Blocks ~700 ms total
00867  *
00868  * @see internal_init_run_sequence() Delegated initialization sequence
00869  * @see bno055_delay_ms_t Timing constants
00870  * @see bno055_config_t Configuration struct
00871  *
00872  * @since Version 1.0.0
00873  */
00874 rx_err_t rx_bno055_init(rx_bus_manager_t* manager, const bno055_config_t* config)
00875 {
00876   static_assert((bool)((k_bno055_ms_per_tick != 0) != 0),
00877                 "k_bno055_ms_per_tick must be non-zero to avoid division by zero in tick delays");
00878   RX_CHECK_NULL_PTR(manager, s_tag, "Bus manager is NULL");
00879   RX_CHECK_NULL_PTR(config, s_tag, "Config is NULL");
00880
00881   if (s_initialized) {
00882     rx_log_error(s_tag, "BNO055 already initialized");
00883     return k_rx_err_invalid_state;
00884   }
00885
00886   s_manager     = manager;
00887   s_mode        = config->mode;
00888   s_initialized = false;
00889
00890   const rx_err_t err = internal_init_run_sequence();
00891   if (err != k_rx_ok) {
00892     s_manager = NULL;
00893     return err;
00894   }
00895
00896   s_initialized = true;
00897   rx_log_info(s_tag, "BNO055 initialized in NDOF mode");
```

```
00898
00899   return k_rx_ok;
00900 }
00901
00902 /**
00903  * @brief Read Euler angle registers from BNO055 and populate heading/roll/pitch
00904  *
00905  * @details
00906  * Burst-reads 6 bytes from register 0x1A (EUL_H_LSB) and assembles
00907  * three 16-bit little-endian values into out->heading_deg16, roll_deg16, pitch_deg16.
00908  *
00909  *
00910  *
00911  * @pre s_initialized == true
00912  * @pre out non-NULL
00913  * @post out->heading_deg16, roll_deg16, pitch_deg16 updated
00914  *
00915  * @note Called only from rx_bno055_read()
00916  *
00917  * @since Version 1.0.0
00918  */
00919 RX_STATIC_TESTABLE rx_err_t internal_read_euler(bno055_data_t* out)
00920 {
00921   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
00922    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
00923   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading Euler angles");
00924   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
00925   static_assert(
00926     (bool)(((unsigned)k_bno055_euler_bytes >= (unsigned)k_bno055_euler_expected_bytes) != 0),
00927     "euler buffer must hold 6 bytes");
00928
00929   uint8_t  euler_buf[k_bno055_euler_bytes];
00930   rx_err_t err = internal_read_regs(k_bno055_reg_eul_h_lsb, euler_buf, k_bno055_euler_bytes);
00931   if (err != k_rx_ok) {
00932     rx_log_error(s_tag, "Euler read failed");
00933     return err;
00934   }
00935
00936   out->heading_deg16 =
00937     internal_assemble_int16_le(euler_buf[k_eul_h_lsb_off], euler_buf[k_eul_h_msb_off]);
00938   out->roll_deg16 =
00939     internal_assemble_int16_le(euler_buf[k_eul_r_lsb_off], euler_buf[k_eul_r_msb_off]);
00940   out->pitch_deg16 =
00941     internal_assemble_int16_le(euler_buf[k_eul_p_lsb_off], euler_buf[k_eul_p_msb_off]);
00942   return k_rx_ok;
00943 }
00944
00945 /**
00946  * @brief Read quaternion registers from BNO055 and populate quat_w/x/y/z
00947  *
00948  * @details
00949  * Burst-reads 8 bytes from register 0x20 (QUA_W_LSB) and assembles
00950  * four 16-bit little-endian values into the quat_w/x/y/z fields.
00951  *
00952  *
00953  *
00954  * @pre s_initialized == true
00955  * @pre out non-NULL
00956  * @post out->quat_w/x/y/z updated
00957  *
00958  * @note Called only from rx_bno055_read()
00959  *
00960  * @since Version 1.0.0
00961  */
00962 RX_STATIC_TESTABLE rx_err_t internal_read_quat(bno055_data_t* out)
00963 {
00964   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
00965    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
00966   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading quaternion");
00967   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
00968   static_assert(
00969     (bool)(((unsigned)k_bno055_quat_bytes >= (unsigned)k_bno055_quat_expected_bytes) != 0),
00970     "quat buffer must hold 8 bytes");
00971
00972   uint8_t  quat_buf[k_bno055_quat_bytes];
00973   rx_err_t err = internal_read_regs(k_bno055_reg_qua_w_lsb, quat_buf, k_bno055_quat_bytes);
00974   if (err != k_rx_ok) {
00975     rx_log_error(s_tag, "Quaternion read failed");
00976     return err;
00977   }
00978
00979   out->quat_w = internal_assemble_int16_le(quat_buf[k_qua_w_lsb_off], quat_buf[k_qua_w_msb_off]);
00980   out->quat_x = internal_assemble_int16_le(quat_buf[k_qua_x_lsb_off], quat_buf[k_qua_x_msb_off]);
00981   out->quat_y = internal_assemble_int16_le(quat_buf[k_qua_y_lsb_off], quat_buf[k_qua_y_msb_off]);
00982   out->quat_z = internal_assemble_int16_le(quat_buf[k_qua_z_lsb_off], quat_buf[k_qua_z_msb_off]);
00983   return k_rx_ok;
00984 }
```

```
00985
00986 /**
00987  * @brief Read linear acceleration registers from BNO055 and populate lin_acc_x/y/z
00988  *
00989  * @details
00990  * Burst-reads 6 bytes from register 0x28 (LIA_X_LSB) and assembles
00991  * three 16-bit little-endian values into the lin_acc_x/y/z fields.
00992  *
00993  *
00994  *
00995  * @pre s_initialized == true
00996  * @pre out non-NULL
00997  * @post out->lin_acc_x/y/z updated
00998  *
00999  * @note Called only from rx_bno055_read()
01000  *
01001  * @since Version 1.0.0
01002  */
01003 RX_STATIC_TESTABLE rx_err_t internal_read_lia(bno055_data_t* out)
01004 {
01005   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
01006    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
01007   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading linear acceleration");
01008   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
01009   static_assert(
01010     (bool)(((unsigned)k_bno055_lia_bytes >= (unsigned)k_bno055_lia_expected_bytes) != 0),
01011     "lia buffer must hold 6 bytes");
01012
01013   uint8_t  lia_buf[k_bno055_lia_bytes];
01014   rx_err_t err = internal_read_regs(k_bno055_reg_lia_x_lsb, lia_buf, k_bno055_lia_bytes);
01015   if (err != k_rx_ok) {
01016     rx_log_error(s_tag, "Linear accel read failed");
01017     return err;
01018   }
01019
01020   out->lin_acc_x = internal_assemble_int16_le(lia_buf[k_lia_x_lsb_off], lia_buf[k_lia_x_msb_off]);
01021   out->lin_acc_y = internal_assemble_int16_le(lia_buf[k_lia_y_lsb_off], lia_buf[k_lia_y_msb_off]);
01022   out->lin_acc_z = internal_assemble_int16_le(lia_buf[k_lia_z_lsb_off], lia_buf[k_lia_z_msb_off]);
01023   return k_rx_ok;
01024 }
01025
01026 /**
01027  * @brief Read BNO055 gyroscope data (raw, 16-bit little-endian)
01028  *
01029  * @details
01030  * Performs a 6-byte I2C burst read from GYR_DATA_X_LSB (0x14) through
01031  * GYR_DATA_Z_MSB (0x19) and assembles three signed 16-bit values.
01032  * Scale: 1 LSB = 1/16 deg/s (k_bno055_scale_gyro_lsb_per_dps = 16)
01033  * when k_bno055_unit_sel_default is configured (dps mode, default).
01034  *
01035  *
01036  *
01037  * @pre s_initialized must be true (call rx_bno055_init() first)
01038  * @pre out must not be NULL
01039  * @post out->gyro_x_dps16, gyro_y_dps16, gyro_z_dps16 populated on k_rx_ok
01040  *
01041  * @note Not thread-safe; called only from rx_bno055_read()
01042  *
01043  * @see rx_bno055_read() Caller
01044  * @see k_bno055_scale_gyro_lsb_per_dps Scale factor (16 LSB per deg/s)
01045  *
01046  * @since Version 1.0.0
01047  */
01048 RX_STATIC_TESTABLE rx_err_t internal_read_gyro(bno055_data_t* out)
01049 {
01050   /* Pre-conditions: s_initialized is true (rx_bno055_read checks via
01051    * RX_CHECK_INITIALIZED); out is a local stack variable (never NULL). */
01052   RX_ASSERT_PRE(s_initialized, "driver must be initialized before reading gyro");
01053   RX_ASSERT_PRE(out != NULL, "out must not be NULL");
01054   static_assert(
01055     (bool)(((unsigned)k_bno055_gyro_bytes >= (unsigned)k_bno055_gyro_expected_bytes) != 0),
01056     "gyro buffer must hold 6 bytes");
01057
01058   uint8_t  gyro_buf[k_bno055_gyro_bytes];
01059   rx_err_t err = internal_read_regs(k_bno055_reg_gyr_x_lsb, gyro_buf, k_bno055_gyro_bytes);
01060   if (err != k_rx_ok) {
01061     rx_log_error(s_tag, "Gyro read failed");
01062     return err;
01063   }
01064
01065   out->gyro_x_dps16 =
01066     internal_assemble_int16_le(gyro_buf[k_gyr_x_lsb_off], gyro_buf[k_gyr_x_msb_off]);
01067   out->gyro_y_dps16 =
01068     internal_assemble_int16_le(gyro_buf[k_gyr_y_lsb_off], gyro_buf[k_gyr_y_msb_off]);
01069   out->gyro_z_dps16 =
01070     internal_assemble_int16_le(gyro_buf[k_gyr_z_lsb_off], gyro_buf[k_gyr_z_msb_off]);
01071   return k_rx_ok;
```

```
01072 }
01073
01074 /**
01075  * @brief Read current BNO055 fusion output data
01076  *
01077  * @details
01078  * Performs six sequential I2C burst reads to collect all output data.
01079  * All 16-bit values are assembled in little-endian format as specified
01080  * in the BNO055 datasheet (LSB register first, MSB register second).
01081  *
01082  * Register read sequence:
01083  * 1. 0x14: 6 bytes -> Gyroscope X[2], Y[2], Z[2]
01084  * 2. 0x1A: 6 bytes -> Euler heading[2], roll[2], pitch[2]
01085  * 3. 0x20: 8 bytes -> Quaternion W[2], X[2], Y[2], Z[2]
01086  * 4. 0x28: 6 bytes -> Linear accel X[2], Y[2], Z[2]
01087  * 5. 0x34: 1 byte  -> Temperature
01088  * 6. 0x35: 1 byte  -> Calibration status
01089  *
01090  *
01091  *
01092  * @pre rx_bno055_init() succeeded
01093  * @pre out non-NULL
01094  * @post All fields in *out populated on k_rx_ok
01095  * @post out->calib_stat reflects current calibration quality
01096  *
01097  * @note Not thread-safe
01098  *
01099  * @see bno055_scale_t Conversion factors for physical units
01100  *
01101  * @since Version 1.0.0
01102  */
01103 rx_err_t rx_bno055_read(bno055_data_t* out)
01104 {
01105   RX_CHECK_NULL_PTR(out, s_tag, "Output data pointer is NULL");
01106
01107   if (!s_initialized) {
01108     return k_rx_err_not_initialized;
01109   }
01110
01111   rx_err_t err = internal_read_gyro(out);
01112   if (err != k_rx_ok) {
01113     return err;
01114   }
01115
01116   err = internal_read_euler(out);
01117   if (err != k_rx_ok) {
01118     return err;
01119   }
01120
01121   err = internal_read_quat(out);
01122   if (err != k_rx_ok) {
01123     return err;
01124   }
01125
01126   err = internal_read_lia(out);
01127   if (err != k_rx_ok) {
01128     return err;
01129   }
01130
01131   /* Read Temperature: 1 byte from 0x34 */
01132   uint8_t temp_raw = 0;
01133   err              = internal_read_regs(k_bno055_reg_temp, &temp_raw, k_bno055_single_byte);
01134   if (err != k_rx_ok) {
01135     rx_log_error(s_tag, "Temperature read failed");
01136     return err;
01137   }
01138
01139   /* Read Calibration status: 1 byte from 0x35 */
01140   uint8_t calib_raw = 0;
01141   err               = internal_read_regs(k_bno055_reg_calib_stat, &calib_raw, k_bno055_single_byte);
01142   if (err != k_rx_ok) {
01143     rx_log_error(s_tag, "Calib status read failed");
01144     return err;
01145   }
01146
01147   out->temp_degc  = (int8_t)temp_raw;
01148   out->calib_stat = calib_raw;
01149
01150   return k_rx_ok;
01151 }
01152
01153 /**
01154  * @brief Check whether all BNO055 subsystems are fully calibrated
01155  *
01156  * @details
01157  * Reads CALIB_STAT register and checks all four 2-bit fields are at
01158  * level 3 (fully calibrated).
```

```
01159  *
01160  *
01161  *
01162  * @pre rx_bno055_init() succeeded
01163  * @pre out_calibrated non-NULL
01164  * @post *out_calibrated valid only on k_rx_ok
01165  * @post CALIB_STAT register read (no state modified)
01166  *
01167  * @note Not thread-safe
01168  * @see bno055_calib_shift_t Calibration shift constants
01169  * @see bno055_calib_mask_t Calibration mask and full-calibration sentinel
01170  *
01171  * @since Version 1.0.0
01172  */
01173 rx_err_t rx_bno055_is_calibrated(bool* out_calibrated)
01174 {
01175   RX_CHECK_NULL_PTR(out_calibrated, s_tag, "Output calibrated pointer is NULL");
01176
01177   if (!s_initialized) {
01178     return k_rx_err_not_initialized;
01179   }
01180
01181   uint8_t  calib_raw = 0;
01182   rx_err_t err = internal_read_regs(k_bno055_reg_calib_stat, &calib_raw, k_bno055_single_byte);
01183   if (err != k_rx_ok) {
01184     return err;
01185   }
01186
01187   const uint8_t mask = k_bno055_calib_level_mask;
01188   const uint8_t full = k_bno055_calib_full;
01189
01190   const uint8_t sys_cal = (calib_raw >> k_bno055_calib_sys_shift) & mask;
01191   const uint8_t gyr_cal = (calib_raw >> k_bno055_calib_gyr_shift) & mask;
01192   const uint8_t acc_cal = (calib_raw >> k_bno055_calib_acc_shift) & mask;
01193   const uint8_t mag_cal = (calib_raw >> k_bno055_calib_mag_shift) & mask;
01194
01195   /* Use bitwise & to evaluate all sub-expressions without short-circuit,
01196    * ensuring full branch coverage. */
01197   *out_calibrated =
01198     (bool)(((sys_cal == full) & (gyr_cal == full) & (acc_cal == full) & (mag_cal == full)) != 0);
01199
01200   return k_rx_ok;
01201 }
01202
01203 rx_err_t rx_bno055_clear_int(void)
01204 {
01205   if (!s_initialized) {
01206     return k_rx_err_not_initialized;
01207   }
01208   uint8_t int_sta = 0U;
01209   return internal_read_regs(k_bno055_reg_int_sta, &int_sta, k_bno055_single_byte);
01210 }
01211
01212 #ifdef UNIT_TEST
01213 /**
01214  * @brief Reset all driver static state to uninitialized (unit test only)
01215  *
01216  * @details
01217  * Clears s_initialized and s_manager so that each unit test starts from a
01218  * clean, uninitialized state. Compiled only when UNIT_TEST is defined.
01219  * Call from setUp() in test files that exercise the BNO055 driver.
01220  *
01221  * @pre None
01222  * @post s_initialized == false
01223  * @post s_manager == NULL
01224  * @post s_mode == k_bno055_mode_poll
01225  *
01226  * @note For unit tests only; not compiled in firmware builds
01227  * @since Version 1.0.0
01228  */
01229 void rx_bno055_test_reset_state(void)
01230 {
01231   s_initialized = false;
01232   s_manager     = NULL;
01233   s_mode        = k_bno055_mode_poll;
01234   /* Post-conditions: s_initialized==false, s_manager==NULL, s_mode==poll
01235    * are guaranteed by the direct assignments above. */
01236   /* GCOVR_EXCL_BR_START LCOV_EXCL_BR_START(values assigned on lines above) */
01237   RX_ASSERT_POST(!s_initialized, "s_initialized must be false after reset");
01238   RX_ASSERT_POST(s_manager == NULL, "s_manager must be NULL after reset");
01239   /* GCOVR_EXCL_BR_STOP LCOV_EXCL_BR_STOP */
01240 }
01241 #endif /* UNIT_TEST */
```